



**National Technical
University of Athens**

**School of Electrical and
Computer Engineering**

Division of Signals, Control & Robotics

EPFL

**École Polytechnique
Fédérale de Lausanne**

**School of Computer and
Communication Sciences**

Machine Learning & Optimization Lab

**On Learning What to Learn:
Adaptive Data Mixtures for Robust Multi-Target
LLM Pretraining**

MASTER'S THESIS of
Maria Ios Glarou

Supervisors

Petros Maragos, Professor
NTUA

Martin Jaggi, Associate Professor
EPFL

Lausanne, October 2025

Abstract

Data selection during LLM pretraining is a major driver of downstream performance. Steering the training mixture with signals from target tasks can orient learning toward representations that better serve those objectives. However, most existing approaches tune mixtures for a single target, yielding narrow specialization and weak robustness across tasks. This thesis introduces GRAPE (Group-Robust Multi-target Adaptive PrEtraining) [30], a multi-source, multi-target reweighting framework that discovers effective training mixtures for multiple targets simultaneously. GRAPE maintains two sets of weights: task weights, encoding the relative priority of each target task, and source weights, specifying sampling proportions over source domains. Derived from a minimax formulation, the method couples two interleaved reweighting updates. First, a task-reweighting step—using group distributionally robust optimization (GDRO)—reallocates task weights toward targets showing the least progress, correspondingly easing emphasis on better-served tasks. Second, a source-reweighting step updates the sampling weights over source domains, shifting probability toward domains whose updates most effectively reduce loss on the targets in focus. Together, these updates instantiate the minimax design and dynamically steer data selection, closing performance gaps and yielding balanced, sample-efficient improvements across targets. Empirically, on ClimbLab, GRAPE outperforms strong baselines with higher average accuracy, superior data efficiency, and more balanced task-wise gains across six targets, while generalizing better to unseen reasoning benchmarks. On SlimPajama, across twelve multi-task suites, we observe consistent improvements over competing methods. In multilingual experiments on Wiki40B, GRAPE leverages six high-resource languages to improve low-resource suites of sizes 4 and 8, achieving faster convergence and lower final perplexity.

Keywords: Large Language Models, Adaptive Data Mixtures, Domain Reweighting, Distributionally Robust Optimization, Pretraining, Multi-Task Learning, Multilingual Language Modeling, Gradient Alignment, Minimax Optimization

Contents

Abstract	3
1 Introduction	13
1.1 Overview	13
1.2 Contributions	14
1.3 Organization	14
1.4 Notation	14
2 Background and Related Work	16
2.1 Overview	16
3 Methodology	18
3.1 GRAPE: A Group DRO Approach to Multi-Target Reweighting	19
3.2 Theoretical Properties	21
3.3 GRAPE Algorithm	22
4 Experimental Setup	23
4.1 Evaluation Scenarios	23
4.2 English Multi-task Reasoning	23
4.3 English Reasoning Suites	24
4.4 Datasets and Domain Partitions	26
4.5 Multilingual Transfer Learning	28
4.6 Models and Training Setup	29
4.6.1 Choice of Architecture	29
4.7 Baseline Methods	30
4.8 Evaluation Metrics	31
5 English Reasoning Experiments	33
5.1 Aggregate Performance: Data Efficiency and Convergence	33
5.2 Task-Level Performance Results	34
5.3 Analysis of Baseline Domain Weight Profiles	35
5.4 Weight Evolution: Emergent Task Curriculum and Domain Selection	36
5.5 Scaling Behavior at 0.7B Parameters	38
5.6 Impact of Task Diversity on Weight Dynamics	39
5.7 Adaptive vs. Static Reweighting on Matched Targets	42
5.8 Generalization Performance on Unseen Benchmarks	42
5.9 Robustness Across Target Combinations	43
5.10 Evaluating Pretrained Initialization Quality via Supervised Fine-Tuning	44
5.11 Ablation: Progress Signals for Target Reweighting	45
5.12 Ablation: Alignment Operators for Task Weight Updates	47

5.13	Ablation: Predictive vs. Realized Progress Signals	48
5.14	Ablation: Impact of Update Frequency on Adaptation	48
5.15	Impact of Domain Granularity on Performance	51
5.16	Validation on SlimPajama Across Multiple Task Suites	52
5.17	Gradient Aggregation Strategies for Multi-Task Pretraining	55
6	Multilingual Modeling Experiments	56
6.1	Aggregate Performance: Data Efficiency and Convergence	56
6.2	Task-Level Performance Results	56
6.3	Weight Evolution: Cross-Lingual Transfer and Linguistic Discovery	59
6.4	Ablation: Progress Signal Variants for Multilingual Learning	60
7	Discussion and Future Directions	61
8	Conclusion	64
8.1	Key Findings	64
8.2	Limitations and Future Work	64
8.3	Closing Remarks	65
A	Ablation Studies	66
B	Implementation Details and Baseline Specifications	94
B.1	Implementation Details for RegMix Baseline	94
B.2	Training Configuration for GRAPE	96
C	Theoretical Derivation and Analysis	97
C.1	Derivation of GRAPE	97
C.2	Theoretical Analysis	98
C.2.1	Proof of the Convergence Theorem	98
C.2.2	Proof of the Variance Reduction Theorem	100
	Bibliography	103

List of Figures

3.1	Overview of the GRAPE algorithm (shown here for the case $\Delta T_\alpha = \Delta T_z = \Delta T$). The workflow alternates between (Left) computing task Rate-of-Improvement (RoI) metrics and updating task weights $\mathbf{z}$, (Middle) updating domain weights α to align domain gradients with the aggregated target gradient, and (Right) standard model training with weighted data sampling for ΔT steps. .	21
5.1	125M, 1.6B tokens (ClimbLab), 6-task setting. Average 5-shot accuracy vs. training tokens. <i>GRAPE demonstrates superior data efficiency and final performance.</i>	33
5.2	0.7B, full runs, 6-task setting. Average 5-shot accuracy vs. training tokens across different datasets. <i>GRAPE maintains superior performance and data efficiency at larger scale on both corpora.</i>	34
5.3	125M, 1.6B tokens (ClimbLab), 6-task setting. End-of-training domain weights heatmap. <i>Adaptive methods converge to STEM-oriented clusters.</i>	35
5.4	125M, 1.6B tokens (ClimbLab), 6-task setting. Domain and target weight trajectories: (a) domain weight evolution, (b) final domain weight heatmap, (c) target task weight evolution. <i>STEM clusters dominate while target weights reveal an emergent curriculum.</i>	38
5.5	125M vs. 0.7B, full runs (ClimbLab), 6-task setting. Target-weight evolution comparison. . .	39
5.6	125M, 1.6B tokens (ClimbLab), 6-task (left) and 8-task (right) settings. (a) Target weight trajectories, (b) domain weight trajectories, (c) final domain heatmaps, (d) embedding projections with clusters having final weight above uniform emphasized. <i>Adding tasks concentrates domain allocation into a tighter STEM-centric mixture.</i>	41
5.7	125M, 0.66B tokens (ClimbLab), 3-task setting. (a) Target weight dynamics, (b) domain weight dynamics, (c) final domain allocations. <i>GRAPE achieves similar final domains but superior outcomes through dynamic adaptation.</i>	42
5.8	125M, 1.6B tokens (ClimbLab), 6-task setting. (a) Target weight evolution, (b) domain weight evolution across different ΔT values. <i>Optimal cadence maintains curriculum diversity while avoiding extreme bias and uniform distribution.</i>	50
5.9	125M, 0.64B tokens, 6-task setting. Target weight trajectories under topic-based (left) vs. source-based (right) domains. <i>Finer domains enable more adaptive curriculum evolution.</i>	52
5.10	125M, 0.66B tokens (SlimPajama), 6-task (left) and 8-task (right) settings. Target weight trajectories vs. training tokens. <i>GRAPE reallocates toward still-improving hard targets while maintaining coverage.</i>	53
5.11	125M, 0.66B tokens (SlimPajama), 6-task (a) and 8-task (b) settings. Final domain weight heatmaps. <i>Different target sets lead to distinct allocation patterns.</i>	53
6.1	125M, 1.6B tokens (Wiki40B), 8-task multilingual setting. Average log-perplexity evolution during training across all target languages. <i>GRAPE achieves 5.8x faster convergence and 10.5% better final performance compared to all baselines.</i>	56
6.2	125M, 1.6B tokens (Wiki40B), 4-task setting. Log-perplexity evolution during training for 4-task setting targets. <i>GRAPE consistently achieves lowest perplexity across all languages, with particularly strong gains on Romanian.</i>	57

6.3	125M, 1.6B tokens (Wiki40B), 8-task setting. Log-perplexity evolution during training for 8-task setting targets. <i>GRAPE maintains consistent advantage across all eight languages, with dramatic improvements on hardest targets like Romanian and Turkish.</i>	58
6.4	125M, 1.6B tokens (Wiki40B), 4-task (left) and 8-task (right) settings. (a) Target weight trajectories, (b) domain weight trajectories, (c) final domain heatmaps. <i>Weight evolution and transfer structure show GRAPE prioritizing laggards and their supporting sources across both task configurations.</i>	59
A.1	Target Tasks Set {ARC-C, ARC-E, GSM8K} : log-perplexity on target tasks across methods (20k steps, 0.66B tokens).	70
A.2	Target Tasks Set {ARC-C, ARC-E, GSM8K} : weight dynamics. Top—target-task weights z ; bottom—training-domain weights α . Larger weights indicate greater emphasis during training.	71
A.3	Target Tasks Set {GSM8K, PIQA} : log-perplexity on target tasks across methods (20k steps, 0.66B tokens).	72
A.4	Target Tasks Set {GSM8K, PIQA} : weight dynamics. Top—target-task weights z ; bottom—training-domain weights α . Larger weights indicate greater emphasis during training.	73
A.5	Target Tasks Set {GSM8K, HellaSwag} : log-perplexity on target tasks across methods (20k steps, 0.66B tokens).	74
A.6	Target Tasks Set {GSM8K, HellaSwag} : weight dynamics. Top—target-task weights z ; bottom—training-domain weights α . Larger weights indicate greater emphasis during training.	75
A.7	Target Tasks Set {GSM8K, LogiQA} : log-perplexity on target tasks across methods (20k steps, 0.66B tokens).	76
A.8	Target Tasks Set {GSM8K, LogiQA} : weight dynamics. Top—target-task weights z ; bottom—training-domain weights α . Larger weights indicate greater emphasis during training.	77
A.9	Target Tasks Set {GSM8K, SciQ} : log-perplexity on target tasks across methods (20k steps, 0.66B tokens).	78
A.10	Target Tasks Set {GSM8K, SciQ} : weight dynamics. Top—target-task weights z ; bottom—training-domain weights α . Larger weights indicate greater emphasis during training.	79
A.11	Target Tasks Set {ARC-C, ARC-E, GSM8K, KodCode} : log-perplexity on target tasks across methods (20k steps, 0.66B tokens).	80
A.12	Target Tasks Set {ARC-C, ARC-E, GSM8K, KodCode} : weight dynamics. Top—target-task weights z ; bottom—training-domain weights α . Larger weights indicate greater emphasis during training.	81
A.13	Target Tasks Set {GSM8K, HellaSwag, KodCode} : log-perplexity on target tasks across methods (20k steps, 0.66B tokens).	82
A.14	Target Tasks Set {GSM8K, HellaSwag, KodCode} : weight dynamics. Top—target-task weights z ; bottom—training-domain weights α . Larger weights indicate greater emphasis during training.	83
A.15	Target Tasks Set {ARC-C, ARC-E, GSM8K, HellaSwag, KodCode, LogiQA, PIQA, SciQ} : log-perplexity on target tasks across methods (20k steps, 0.66B tokens).	84
A.16	Target Tasks Set {ARC-C, ARC-E, GSM8K, HellaSwag, KodCode, LogiQA, PIQA, SciQ} : weight dynamics. Top—target-task weights z ; bottom—training-domain weights α . Larger weights indicate greater emphasis during training.	85
A.17	Target Tasks Set {ARC-C, ARC-E, HellaSwag, LogiQA, PIQA, SciQ} : log-perplexity on target tasks across methods (20k steps, 0.66B tokens).	86
A.18	Target Tasks Set {ARC-C, ARC-E, HellaSwag, LogiQA, PIQA, SciQ} : weight dynamics. Top—target-task weights z ; bottom—training-domain weights α . Larger weights indicate greater emphasis during training.	87
A.19	Target Tasks Set {ARC-C, ARC-E, HellaSwag, LogiQA, MathQA, MedQA, PIQA, SciQ} : log-perplexity on target tasks across methods (20k steps, 0.66B tokens).	88

A.20	Target Tasks Set {ARC-C, ARC-E, HellaSwag, LogiQA, MathQA, MedQA, PIQA, SciQ} : weight dynamics. Top—target-task weights z ; bottom—training-domain weights α . Larger weights indicate greater emphasis during training.	89
A.21	Target Tasks Set {ARC-C, ARC-E, HellaSwag, LogiQA, MathQA, MedQA} : log-perplexity on target tasks across methods (20k steps, 0.66B tokens).	90
A.22	Target Tasks Set {ARC-C, ARC-E, HellaSwag, LogiQA, MathQA, MedQA} : weight dynamics. Top—target-task weights z ; bottom—training-domain weights α . Larger weights indicate greater emphasis during training.	91
A.23	Target Tasks Set {HellaSwag, LogiQA, MathQA, MedQA} : log-perplexity on target tasks across methods (20k steps, 0.66B tokens).	92
A.24	Target Tasks Set {HellaSwag, LogiQA, MathQA, MedQA} : weight dynamics. Top—target-task weights z ; bottom—training-domain weights α . Larger weights indicate greater emphasis during training.	93
B.1	RegMix regression fits across three experimental settings. Each scatter plot shows predicted vs. true validation loss across 256 test proxy models, with Spearman correlation indicating prediction quality.	95

List of Tables

4.1	7 evaluation benchmarks with human performance baselines.	25
4.2	9 benchmark datasets with training and evaluation sample counts.	26
4.3	20 ClimbLab semantic clusters with topics and token distributions.	27
4.4	7 SlimPajama source domains and their characteristics.	28
4.5	6 source and 8 target language assignments for Wiki40B.	28
4.6	14 Wiki40B language families, scripts, and transfer relationships.	29
4.7	GPT-2 model architectures and parameter counts.	29
4.8	Compute overhead from GRAPE reweighting across 3 experimental settings.	30
4.9	Comparison of methods by task weights, domain weights, update frequency, and the signal used for α updates.	30
5.1	125M, 1.6B tokens (ClimbLab), 6-task setting. Task log-perplexity (lower is better). <i>GRAPE achieves the best average across all reasoning tasks.*</i>	35
5.2	125M, 1.6B tokens (ClimbLab), 6-task setting. Task 5-shot accuracy (higher is better). <i>GRAPE achieves balanced improvements across all reasoning tasks.*</i>	35
5.3	High-weight domain clusters identified by GRAPE emphasize STEM and procedural reasoning.	37
5.4	0.7B, full run (ClimbLab), 6-task setting. Task 5-shot accuracy at larger scale. <i>GRAPE improves on all tasks and achieves best average.*</i>	38
5.5	125M and 0.7B, ClimbLab, 6-task setting. Average 5-shot accuracy and relative gains. <i>GRAPE's advantage narrows as model scale increases.</i>	39
5.6	125M, 1.6B tokens (ClimbLab), 8-task setting. Perplexity and 5-shot accuracy. <i>GRAPE achieves the lowest average perplexity and accuracy.*</i>	40
5.7	125M, 0.66B tokens (ClimbLab), 3-task setting. Task 5-shot accuracy on matched targets. <i>GRAPE improves all targets and generalizes better to non-targets.*</i>	42
5.8	125M, 1.6B tokens (ClimbLab), 6-task setting. 5-shot accuracy $\uparrow$ (%) on unseen tasks. <i>GRAPE demonstrates strong generalization beyond training targets.*</i>	43
5.9	125M, 0.66B tokens (SlimPajama), T1–T12 settings. Average log-perplexity and 5-shot exact-match accuracy across 12 target configurations. <i>GRAPE achieves the best performance across all settings.*</i>	44
5.10	125M, 0.66B tokens (SlimPajama), T1–T12 settings. 5-shot exact-match accuracy on unseen tasks (not used during training) across 12 target configurations. <i>GRAPE consistently outperforms DoGE on held-out tasks, demonstrating strong generalization capabilities.*</i>	44
5.11	125M, 1.6B tokens (ClimbLab), 6-task setting. Pretrained vs. post-SFT accuracy. <i>GRAPE provides a stronger initialization for supervised fine-tuning.*</i>	45
5.12	125M, 0.66B tokens (SlimPajama), T1–T12 settings. Average test log-perplexity across 12 task combinations with summary statistics. <i>ROI attains the most wins across diverse multi-task configurations and delivers best average and worst-case perplexity.*</i>	46
5.13	125M, 0.66B tokens (SlimPajama), T1–T12 settings. Target vs. unseen accuracy (%). <i>Progress signal choice induces different generalization profiles.*</i>	47

5.14	125M, 573M tokens (ClimbLab), 6-task setting. Task log-perplexity under different alignment operators. <i>Dot product captures sign, magnitude, and direction simultaneously.*</i>	48
5.15	125M, 1.6B tokens (ClimbLab), 6-task setting. Task 5-shot accuracy under different progress signals. <i>Directional alignment outperforms realized loss differences.*</i>	48
5.16	125M, 1.6B tokens (ClimbLab), 6-task setting. Task 5-shot accuracy under different update cadences ΔT . <i>Mid-range cadence ($\Delta T=100$) balances adaptation speed with stability.*</i>	49
5.17	125M, 1.6B tokens (ClimbLab), 6-task setting. Runtime measurements across different update frequencies. <i>In practice, runtime differences are primarily driven by update frequency rather than method choice.</i>	51
5.18	125M, 0.64B tokens, 6-task setting. Overall accuracy under different domain granularities. <i>Finer-grained domains enable larger improvements.</i>	52
5.19	125M, 0.64B tokens, 6-task setting. Task-level accuracy percentage improvements over DoGE. <i>Topic-based domains benefit knowledge-intensive tasks most.</i>	52
5.20	125M, 0.66B tokens (SlimPajama), 6-task setting. Task log-perplexity. <i>GRAPE achieves best average and worst-case across all tasks.*</i>	54
5.21	125M and 0.7B, 0.66B tokens (SlimPajama), 6-task setting. Task 5-shot accuracy at two scales. <i>GRAPE achieves best average at both scales.*</i>	54
5.22	125M, 0.66B tokens (SlimPajama), 8-task setting. Task log-perplexity with average and worst-case. <i>GRAPE achieves best average and worst-case perplexity.*</i>	54
5.23	125M, 0.66B tokens (SlimPajama), 8-task setting. Task 5-shot accuracy. <i>GRAPE achieves best average with strong specialized-task performance.*</i>	54
5.24	125M, 0.96B tokens (ClimbLab), 6-task setting. Task 5-shot accuracy under different gradient aggregation methods. <i>Simple averaging with domain reweighting remains most effective.*</i>	55
6.1	125M, 1.6B tokens (Wiki40B), 4-task setting. Final log-perplexity on 4-task setting targets only. <i>GRAPE achieves best average across all baselines.*</i>	57
6.2	125M, 1.6B tokens (Wiki40B), 8-task setting. Final log-perplexity on 8-task setting targets only. <i>GRAPE achieves best average and leads on all individual languages.*</i>	57
6.3	125M, 1.6B tokens (Wiki40B), 8-task setting. GRAPE discovers and exploits linguistic family relationships for improved transfer. <i>All language families show consistent gains, with Romance languages achieving the strongest improvements.*</i>	60
6.4	125M, 1.6B tokens (Wiki40B), 4-task setting. GRAPE variants vs. DoGE and Uniform on 4-task setting. <i>All GRAPE variants substantially outperform domain-only and uniform baselines.*</i>	60
6.5	125M, 1.6B tokens (Wiki40B), 8-task setting. GRAPE variants vs. DoGE and Uniform on 8-task setting. <i>GRAPE-GAP delivers 16% improvement over uniform baseline.*</i>	60
A.1	All Target Tasks Sets (T1–T12) : mapping organized by domain focus. Each combination tests different aspects of multi-target adaptive pretraining across reasoning, mathematical, linguistic, logical, scientific, medical, and coding domains.	68
A.2	Log perplexity summary for all ablation settings (lower is better). Each cell shows avg min max (across target tasks within the combination) for each method. “Best (avg)” indicates which method achieved the lowest average perplexity per task set. “Wins (avg)” gives the total out of 12 ablations. Bold values denote best performance; underlined values denote second-best performance.	69
A.3	All Target Tasks Sets (T1–T12) : average 5-shot exact-match accuracy across (i) tasks set as targets in the method and (ii) unseen tasks (never used during training). “Wins (/12)” counts the number of times (out of 12) each method achieves the highest score per row. Bold values denote best performance; underlined values denote second-best performance.	69
A.4	Target Tasks Set {ARC–C, ARC–E, GSM8K} : log-perplexity ↓ and 5-shot exact-match accuracy ↑ at 0.66B training tokens.	70

A.5	Target Tasks Set {ARC-C, ARC-E, GSM8K} : 5-shot accuracy ↑ on held-out tasks (%).	71
A.6	Target Tasks Set {GSM8K, PIQA} : log-perplexity ↓ and 5-shot exact-match accuracy ↑ at 0.66B training tokens.	72
A.7	Target Tasks Set {GSM8K, PIQA} : 5-shot accuracy ↑ on held-out tasks (%).	73
A.8	Target Tasks Set {GSM8K, HellaSwag} : log-perplexity ↓ and 5-shot exact-match accuracy ↑ at 0.66B training tokens.	74
A.9	Target Tasks Set {GSM8K, HellaSwag} : 5-shot accuracy ↑ on held-out tasks (%).	75
A.10	Target Tasks Set {GSM8K, LogiQA} : log-perplexity ↓ and 5-shot exact-match accuracy ↑ at 0.66B training tokens.	76
A.11	Target Tasks Set {GSM8K, LogiQA} : 5-shot accuracy ↑ on held-out tasks (%).	77
A.12	Target Tasks Set {GSM8K, SciQ} : log-perplexity ↓ and 5-shot exact-match accuracy ↑ at 0.66B training tokens.	78
A.13	Target Tasks Set {GSM8K, SciQ} : 5-shot accuracy ↑ on held-out tasks (%).	79
A.14	Target Tasks Set {ARC-C, ARC-E, GSM8K, KodCode} : log-perplexity ↓ and 5-shot exact-match accuracy ↑ at 0.66B training tokens.	80
A.15	Target Tasks Set {ARC-C, ARC-E, GSM8K, KodCode} : 5-shot accuracy ↑ on held-out tasks (%).	81
A.16	Target Tasks Set {GSM8K, HellaSwag, KodCode} : log-perplexity ↓ and 5-shot exact-match accuracy ↑ at 0.66B training tokens.	82
A.17	Target Tasks Set {GSM8K, HellaSwag, KodCode} : 5-shot accuracy ↑ on held-out tasks (%).	83
A.18	Target Tasks Set {ARC-C, ARC-E, GSM8K, HellaSwag, KodCode, LogiQA, PIQA, SciQ} : log-perplexity ↓ and 5-shot exact-match accuracy ↑ at 0.66B training tokens.	84
A.19	Target Tasks Set {ARC-C, ARC-E, GSM8K, HellaSwag, KodCode, LogiQA, PIQA, SciQ} : 5-shot accuracy ↑ on held-out tasks (%).	85
A.20	Target Tasks Set {ARC-C, ARC-E, HellaSwag, LogiQA, PIQA, SciQ} : log-perplexity ↓ and 5-shot exact-match accuracy ↑ at 0.66B training tokens.	86
A.21	Target Tasks Set {ARC-C, ARC-E, HellaSwag, LogiQA, PIQA, SciQ} : 5-shot accuracy ↑ on held-out tasks (%).	87
A.22	Target Tasks Set {ARC-C, ARC-E, HellaSwag, LogiQA, MathQA, MedQA, PIQA, SciQ} : log-perplexity ↓ and 5-shot exact-match accuracy ↑ at 0.66B training tokens.	88
A.23	Target Tasks Set {ARC-C, ARC-E, HellaSwag, LogiQA, MathQA, MedQA, PIQA, SciQ} : 5-shot accuracy ↑ on held-out tasks (%).	89
A.24	Target Tasks Set {ARC-C, ARC-E, HellaSwag, LogiQA, MathQA, MedQA} : log-perplexity ↓ and 5-shot exact-match accuracy ↑ at 0.66B training tokens.	90
A.25	Target Tasks Set {ARC-C, ARC-E, HellaSwag, LogiQA, MathQA, MedQA} : 5-shot accuracy ↑ on held-out tasks (%).	91
A.26	Target Tasks Set {HellaSwag, LogiQA, MathQA, MedQA} : log-perplexity ↓ and 5-shot exact-match accuracy ↑ at 0.66B training tokens.	92
A.27	Target Tasks Set {HellaSwag, LogiQA, MathQA, MedQA} : 5-shot accuracy ↑ on held-out tasks (%).	93
B.1	Training configuration for Small and Large models.	96

Chapter 1

Introduction

1.1 Overview

Large language models (LLMs) are pre-trained on mixtures of heterogeneous data sources (domains) spanning topics, styles, and languages. The composition of this mixture strongly shapes the capabilities of the resulting model. Fixed, heuristic sampling strategies are simple but can be misaligned with downstream needs. Domain reweighting during pretraining offers a way to adapt the mixture as learning unfolds. However, existing approaches implicitly optimize for a single target objective or an average over tasks, which can mask conflicts between tasks and lead to uneven progress. In practice, we often care about maintaining robust performance across multiple targets (e.g., reasoning benchmarks and multilingual modeling) under strict compute and token budgets.

This work studies multi-source, multi-target domain reweighting: how to allocate pretraining tokens across source domains while balancing progress across several target tasks. Moving beyond single-target optimization raises distinct challenges:

Competing signals and evolving difficulty. Multi-task training often exhibits dominant or conflicting training signals; naive averaging can overfit to the easiest or loudest target while neglecting others. Difficulty is also nonstationary: some targets saturate while others remain challenging, so a mixture that helps early can later hinder overall progress. An effective strategy should surface targets that are falling behind and direct additional support from sources that tend to benefit them, while preserving gains elsewhere.

Constraints at scale. The scale of LLM pretraining imposes tight memory and compute constraints, which narrow the space of feasible interventions. Methods that require explicit per-target gradient operations or large auxiliary state can become impractical as model size and the number of targets grow. Practical mechanisms should therefore rely on inexpensive, incrementally updated signals and infrequent, scheduled reweighting that keeps overhead low yet remains responsive to changing training dynamics.

Bringing these considerations together, we identify three key requirements for a practical solution: (i) optimizing a target-aware objective that promotes balanced progress across all tasks; (ii) dynamically adapting allocation throughout training; and (iii) operating efficiently to meet LLM-scale constraints.

The approach we propose in this work is designed with these principles at its core. We introduce GRAPE (Group Robust Multi-target Adaptive Pretraining), which explicitly models the interplay between *what to improve* (task prioritization) and *how to improve it* (domain allocation). The core idea is a two-level feedback mechanism. First, we track how quickly each target task is improving by monitoring recent loss reductions—using the model’s own previous performance as a reference for measuring additional improvement. This progress-based signal naturally adapts attention: tasks that are advancing slowly receive higher priority, ensuring no target is left behind (inspired by Group-DRO). Second, we shift domain sampling toward sources whose gradients align with

progress on those emphasized tasks—domains that help struggling tasks gain more sampling weight.

We formalize this as a regularized minimax problem where task weights and domain weights co-evolve at scheduled update rounds. In practice, we estimate gradient alignments stochastically using small mini-batches sampled only at reweighting rounds, apply KL-regularized multiplicative updates, and resume standard training. Importantly, the overhead depends on update frequency and the numbers of tasks and domains, not model size, ensuring scalability.

1.2 Contributions

This thesis makes the following contributions:

- **Principled Formulation:** We provide a formal framework for multi-source, multi-target domain reweighting in LLM pretraining, focusing on adaptively balancing progress across all target tasks.
- **Practical Algorithm:** We introduce GRAPE, an efficient and scalable method that periodically updates task and domain weights using lightweight, batch-based gradient signals, ensuring negligible overhead and practical applicability.
- **Empirical Validation:** Through extensive experiments in both multi-task reasoning and multilingual language modeling, we show that our approach improves sample efficiency and achieves stronger, more balanced performance compared to strong baselines.
- **Key Insights:** We uncover and discuss several important findings: (i) GRAPE discovers automatic curricula, dynamically prioritizing harder skills as training evolves; (ii) improvements gained in pretraining transfer effectively to subsequent supervised fine-tuning; (iii) domain granularity matters—finer-grained domains yield more stable and effective reweighting.
- **Limitations and Discussion:** We analyze practical considerations such as sensitivity to scheduling, interactions with domain granularity and data quality, and highlight limitations and promising directions for future work.

1.3 Organization

Chapter 3 provides the formal analysis of the minimax objective for multi-source, multi-target domain reweighting, deriving the multiplicative update rules for task and domain weights and discussing practical scheduling. Chapter 4 details the experimental design and Chapters 5 and 6 present results on multi-task reasoning and multilingual modeling, as well as ablations on progress metrics, frequency of weight updates, and task configurations. Chapter 7 summarizes the findings discusses limitations and suggests directions for future research. The theoretical analysis with the method derivation and detailed proofs is provided in Appendix C, while additional ablations details are reported in Appendix A.

1.4 Notation

We use the following notation throughout this thesis:

- K, N : number of training domains and target tasks, respectively
- $\mathcal{D}_{\text{train}} \triangleq \{D_k\}_{k=1}^K$: pretraining corpus partitioned into K source domains
- $\mathcal{D}_{\text{tgt}} \triangleq \{T_n\}_{n=1}^N$: set of N target tasks or low-resource languages
- θ_t : model parameters at step t

- $\alpha \in \Delta_K$: domain weights (probability simplex in $\mathbb{R}^K$)
- $z \in \Delta_N$: task weights (probability simplex in $\mathbb{R}^N$)
- $\ell_n(\theta_t)$: loss on target task T_n at step t
- $g_k(\theta_t) \triangleq \nabla_{\theta} \ell(\theta_t; x_k)$: training gradient on mini-batch $x_k \sim D_k$
- μ_{α}, μ_z : KL regularization coefficients for domain and task weights
- γ_t : learning rate at step t

Chapter 2

Background and Related Work

2.1 Overview

Scaling Laws and the Centrality of Data Empirical scaling laws relate pretraining loss to model size, dataset size, and compute [43]. Under fixed compute, training smaller models on more tokens can be optimal, indicating that many historical models were under-trained [39]. Consequently, *data*—its scale, composition, and cleanliness—has become a primary driver of capability. Public corpora such as C4 [70], The Pile [32], and RefinedWeb [65] show that mixture design and deduplication materially affect downstream behavior [49], while concerns about bias, toxicity, and contamination [8] motivate principled selection and documentation.

Current Pretraining Practice Practice has shifted from indiscriminate web scraping to *curated, deduplicated, and objective-aware* mixtures with transparent pipelines and ablations. Beyond legacy web corpora, FineWeb and FineWeb-Edu exemplify high signal-to-noise filtering, near-duplicate removal, and provenance-aware sampling at scale [64]. Contemporary mixtures also integrate capability-focused sources (code, mathematics, scientific text), broaden multilingual coverage, and include safety/contamination controls. Training recipes increasingly emphasize long-context readiness and efficiency (continued pretraining, token packing, memory-efficient attention, low-precision kernels). This evolution raises the central question of the chapter: *how should data be selected and weighted so that pretraining aligns with evaluation objectives?*

Problem Setting and Organizing Axes Let D_{raw} be a heterogeneous corpus partitioned into domains by provenance, language, topic, or clustering, and let A denote the pretraining algorithm. The goal is to choose a subset or sampling distribution q over D_{raw} so that the resulting model $M = A(q)$ optimizes a target objective (e.g., held-out validation loss, downstream probes, or robustness). Exhaustive search is infeasible at LLM scale; practical methods estimate domain/example *utility* and enact it via filtering, thresholding, or importance sampling. It is useful to structure the space along three axes: *granularity* (domain vs. example), *timing* (offline vs. online), and *objective* (single surrogate vs. multiple, potentially conflicting targets). Domain-level control maps directly to sampler proportions and scales well; example-level control offers precision at higher computational cost.

Fixed Mixtures as Baselines Fixed mixtures remain strong baselines. Recipes such as C4 + Wikipedia + Books [70] and broader web-heavy blends like The Pile or RefinedWeb [32, 65] pair heuristic filtering with multi-stage deduplication [49]. FineWeb extends this direction with large-scale ablations on extraction, filtering, and deduplication choices [64]. These pipelines meaningfully affect generalization, multilingual coverage, and contamination, but they are *static*: they do not adapt to evaluation distributions or to the model’s evolving representation geometry.

From Static Recipes to Learned Mixtures To move beyond static recipes, learned methods control q according to an explicit objective. At *domain* granularity—the most practical knob at LLM scale—two timing regimes

dominate. *Offline* (proxy-based) approaches learn a *static* mixture before the large run, trading adaptability for scalability: RegMix fits a regressor over many lightweight proxy runs to predict an optimal mixture for a held-out surrogate (e.g., Pile-CC validation) and transfers the top mixture [54]; CLIMB first clusters the corpus into ~ 20 semantic “domains”, then performs iterative mixture search [24]; DoGE learns static weights by bilevel optimization with gradient alignment to a target [29]. *Online* approaches adapt weights *during* the large run in response to model feedback; DGA periodically recomputes gradient alignment on the current model to refresh domain weights [28]. Both regimes share the same aim—steering training toward a surrogate objective—but differ in when feedback is incorporated and how disruptive the mechanism is to data pipelines.

Going Finer: Sample-Level Selection Beyond domains, *sample*-level selection exploits finer signals such as influence and trajectory-based attribution (IF [46], TracIn [66], TRAK [63]) and coalitional valuation (Data Shapley [35, 41], Beta Shapley [47], Data Banzhaf [90]). These methods can deliver high precision but are computationally intensive at LLM scale and often rely on proxies, checkpoint ensembling, or denoising; in practice, they complement domain-level reweighting rather than replace it.

Multi-Target Reality and the Link to MTL Most domain-reweighting methods optimize a *single* surrogate (a specific validation set), which risks biasing mixtures and inducing negative transfer when evaluation spans multiple, competing targets. This setting mirrors multi-task learning (MTL): minimizing mean loss can be suboptimal when gradients conflict or when one task dominates by gradient magnitude [52, 97]. Gradient-surgery methods address conflicts by combining or projecting per-task gradients with adaptive weights—PCGrad ($\mathcal{O}(|\theta|N^2)$) [97], CAGrad (constrained $\mathcal{O}(N^3)$) [52]—while MGDA [22] and NASH-MTL [61] provide Pareto-style guarantees. However, all require storing task gradients ($\mathcal{O}(|\theta|N)$) and at least $\mathcal{O}(|\theta|N^2)$ work per step, which is prohibitive for LLMs with many targets. FAMO [51] improves efficiency by dynamically weighting task losses to equalize rates of decrease using $\mathcal{O}(1)$ space/time per iteration. This perspective suggests that mixture design should incorporate multi-target signals—e.g., gradient alignment against several targets or progress-based balancing—rather than optimizing a single proxy.

How Adaptive Methods Integrate with Current Practice Adaptive reweighting *layers on top of* curated pipelines; it does not replace them. A typical workflow is: (i) assemble a high-signal-to-noise, deduplicated, provenance-aware pool (e.g., RefinedWeb, FineWeb, in-domain additions) and set coarse static ratios for coverage, safety, and throughput; (ii) learn an *offline* warm start with a lightweight proxy (RegMix/CLIMB/DoGE) and use the resulting static mixture as the *initial* sampler for the large run; and (iii) apply *online* refinement during the run with low-frequency, bounded updates (e.g., DGA-style gradient alignment or progress signals) under guardrails such as entropy regularization, per-domain floors/ceilings, and momentum on weights. This pattern aligns with production constraints—predictability, throughput, and reproducibility—while injecting objective-awareness where it is most useful.

Curated web pipelines (e.g., FineWeb [64]) lift baseline quality but remain static. Learned domain-level reweighting provides a scalable control knob with a clear trade-off between offline scalability and online adaptivity; sample-level methods add precision where their cost is justified. Realistic evaluations are multi-target, so effective mixture optimization should balance progress across competing objectives instead of optimizing a single surrogate, and it should do so as a thin, well-guarded layer on top of today’s curated data pipelines.

Chapter 3

Methodology

In this chapter, we first formalize the multi-source-multi-target reweighting problem (Section 3) and present the complete GRAPE algorithm and the theoretical properties of our method.

Conceptual Overview GRAPE adaptively calibrates both task and domain sampling distributions during pre-training. Its central principle is to emphasize *lagging tasks* — those that improve more slowly — by adjusting the upstream data mixture.

We formalize this idea as a minimax optimization problem grounded in Group DRO. The inner problem highlights underperforming tasks via task reweighting, while the outer problem adjusts domain weights to accelerate progress on those prioritized tasks.

Together, these dynamics yield an *adaptive curriculum* that continuously redirects training toward the most challenging bottlenecks, ensuring that no task is persistently neglected.

Problem Formulation. Let $\mathcal{D}_{\text{train}} \triangleq \{D_k\}_{k=1}^K$ denote a pretraining corpus partitioned into K source domains based on meta-attributes (e.g., data source, topic, language, modality). Training draws minibatches from $\mathcal{D}_{\text{train}}$ according to a domain distribution $\alpha \in \Delta_K$, where Δ_K is the probability simplex in $\mathbb{R}^K$. Let $\mathcal{D}_{\text{tgt}} \triangleq \{T_n\}_{n=1}^N$ denote N downstream target tasks, where each T_n is a specific evaluation benchmark (e.g., mathematical reasoning, commonsense reasoning, scientific understanding). The overall objective of multi-target, multi-domain pretraining is to adapt the domain-sampling distribution $\alpha \in \Delta_K$ so that training achieves uniformly strong performance across all N target tasks. We denote:

- θ_t : model parameters at step t .
- γ_t : learning rate at step t .
- $\ell_n(\theta_t)$: stochastic loss of θ_t evaluated on target task T_n , for $n \in [N]$.
- $\mathbf{g}_k(\theta_t) \triangleq \nabla_{\theta} \ell(\theta_t; \mathbf{x}_k)$: training gradient on a mini-batch $\mathbf{x}_k \sim D_k$ from source domain k for $k \in [K]$.
- $\mathbf{d}_t$: *mixture gradient* overall update direction at step t .
- $\alpha^{(t)}$ is the vector of domain weights at step t ($\alpha_k^{(t)}$: domain weight for domain k at step t , i.e. the k -th element of $\alpha^{(t)}$)

We assume optimization proceeds via gradient descent: $\theta_{t+1} = \theta_t - \gamma_t \mathbf{d}_t$ (SGD). The overall update direction is a domain-weighted sum of gradients:

$$\mathbf{d}_t := \sum_{k=1}^K \alpha_k^{(t)} \mathbf{g}_k(\theta_t).$$

We additionally introduce *task weights* $\mathbf{z} \in \Delta_N \subset \mathbb{R}^N$ to represent the relative importance assigned to each target task, allowing these weights to be updated dynamically as learning progresses. The key idea is to update $\mathbf{z}$ in response to task improvement and to adapt α based on the current $\mathbf{z}$ to optimize overall training progress.

3.1 GRAPE: A Group DRO Approach to Multi-Target Reweighting

Rate of Improvement (RoI) Signal We define the *Rate of Improvement* (RoI) as the relative loss reduction between consecutive steps:

$$r_n^{(t)} = \frac{\ell_n(\boldsymbol{\theta}_t) - \ell_n(\boldsymbol{\theta}_{t+1})}{\ell_n(\boldsymbol{\theta}_t)}. \quad (3.1)$$

Normalization by $\ell_n(\boldsymbol{\theta}_t)$ prevents high-loss targets from dominating the signal, yielding a fair, scale-invariant measure of progress. A first-order Taylor expansion at $\boldsymbol{\theta}_t$ gives up to a second-order term $o(\|\mathbf{d}_t\|^2)$:

$$\ell_n(\boldsymbol{\theta}_{t+1}) \approx \ell_n(\boldsymbol{\theta}_t) + \langle \nabla_{\boldsymbol{\theta}} \ell_n(\boldsymbol{\theta}_t), \boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_t \rangle$$

So for small γ_t , RoI can be approximated as:

$$r_n^{(t)} \approx \gamma_t \langle \nabla_{\boldsymbol{\theta}} \log \ell_n(\boldsymbol{\theta}_t), \mathbf{d}_t \rangle, \quad (3.2)$$

where $\nabla_{\boldsymbol{\theta}} \log \ell_n(\boldsymbol{\theta}_t) = \frac{\nabla_{\boldsymbol{\theta}} \ell_n(\boldsymbol{\theta}_t)}{\ell_n(\boldsymbol{\theta}_t)}$.

Substituting the mixture form of $\mathbf{d}_t$ then gives:

$$r_n^{(t)} \approx \gamma_t \sum_{k=1}^K \alpha_k^{(t)} \langle \nabla_{\boldsymbol{\theta}} \log \ell_n(\boldsymbol{\theta}_t), \mathbf{g}_k(\boldsymbol{\theta}_t) \rangle \quad (3.3)$$

GDRO Formulation Following the principles of Group DRO [67, 73], we formulate a minimax objective to optimize the domain weights ($\boldsymbol{\alpha}$) for the worst-case (minimum) RoI score across tasks:

$$\begin{aligned} \max_{\boldsymbol{\alpha} \in \Delta^K} \min_{n \in [N]} \mathbb{E}[r_n^{(t)}] &\approx \max_{\boldsymbol{\alpha} \in \Delta^K} \min_{\mathbf{z} \in \Delta^N} \sum_{n=1}^N z_n \mathbb{E}[r_n^{(t)}] - \bar{h}(\boldsymbol{\alpha}) + \underline{h}(\mathbf{z}) \\ &\approx \max_{\boldsymbol{\alpha} \in \Delta^K} \min_{\mathbf{z} \in \Delta^N} \gamma_t \sum_{k=1}^K \alpha_k \sum_{n=1}^N z_n \mathbb{E}[\langle \nabla_{\boldsymbol{\theta}} \log \ell_n(\boldsymbol{\theta}_t), \mathbf{g}_k(\boldsymbol{\theta}_t) \rangle] - \bar{h}(\boldsymbol{\alpha}) + \underline{h}(\mathbf{z}). \end{aligned} \quad (3.4)$$

Here, the inner minimization finds the task weights $\mathbf{z}$ that concentrate on the tasks with the lowest expected RoI; the outer maximization finds the domain weights $\boldsymbol{\alpha}$ that maximize this minimum expected RoI. Following [59, 67], we introduce Bregman divergence regularization terms $\bar{h}(\boldsymbol{\alpha})$ and $\underline{h}(\mathbf{z})$ relative to the previous weights $\boldsymbol{\alpha}^{(t-1)}$ and $\mathbf{z}^{(t-1)}$ to stabilize the updates and manage stochastic variance:

$$\bar{h}(\boldsymbol{\alpha}) := \mu_{\boldsymbol{\alpha}} D_{\Psi}(\boldsymbol{\alpha} \| \boldsymbol{\alpha}^{(t-1)}), \quad \underline{h}(\mathbf{z}) := \mu_{\mathbf{z}} D_{\Psi}(\mathbf{z} \| \mathbf{z}^{(t-1)}),$$

where $\Psi(\mathbf{b}) = \sum_i b_i \log(b_i)$ is the negative entropy.

Update Rules. Solving this regularized minimax problem (see Appendix C.1) yields multiplicative update rules resembling online mirror descent. At each step t , the task weights are updated as:

$$\mathbf{z}^{(t)} = \frac{\hat{\mathbf{z}}^{(t)}}{\sum_{n \in [N]} \hat{z}_n^{(t)}}, \quad \text{with } \hat{z}_n^{(t)} \leftarrow z_n^{(t-1)} \cdot \exp \left(-\frac{\gamma_t}{\mu_{\mathbf{z}}} \mathbb{E}_{\mathbf{x} \sim \text{mix}(\boldsymbol{\alpha}^{(t-1)})} [\langle \nabla_{\boldsymbol{\theta}} \log \ell_n(\boldsymbol{\theta}_t), \nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}_t; \mathbf{x}) \rangle] \right), \quad (3.5)$$

and the domain weights are updated as:

$$\boldsymbol{\alpha}^{(t)} = \frac{\hat{\boldsymbol{\alpha}}^{(t)}}{\sum_{k \in [K]} \hat{\alpha}_k^{(t)}}, \quad \text{with } \hat{\alpha}_k^{(t)} \leftarrow \alpha_k^{(t-1)} \cdot \exp \left(\frac{\gamma_t}{\mu_{\boldsymbol{\alpha}}} \mathbb{E}_{\mathbf{y} \sim \text{mix}(\mathbf{z}^{(t-1)})} [\langle \mathbf{g}_k(\boldsymbol{\theta}_t), \nabla_{\boldsymbol{\theta}} \log \ell(\boldsymbol{\theta}_t; \mathbf{y}) \rangle] \right), \quad (3.6)$$

The inner product $\langle \nabla_{\boldsymbol{\theta}} \log \ell_n(\boldsymbol{\theta}_t), \mathbf{g}_k(\boldsymbol{\theta}_t) \rangle$ captures the alignment between the gradient direction of task $n \in [N]$ (normalized by loss) and the gradient direction of domain $k \in [K]$. Here, $\nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}_t; \mathbf{x})$ denotes the (training loss)

gradient from a training batch $\mathbf{x}$ sampled according to $\alpha^{(t-1)}$, and $\nabla_{\theta} \log \ell(\theta_t; \mathbf{y})$ denotes the task-weighted normalized validation gradient from a validation batch $\mathbf{y}$ sampled according to $\mathbf{z}^{(t-1)}$.

In practice, the expectations in Equation 3.5 and Equation 3.6 are replaced by stochastic estimates using mini-batches. Furthermore, these weights are updated periodically for two reasons. First, computing N target gradients and K domain gradients at every step is computationally expensive. By updating these weights periodically, we can amortize the cost of these gradient evaluations over multiple steps. Second, the model needs time to adapt to the new weights before they can be meaningfully re-evaluated.

Interpretation. The inner distribution $\mathbf{z} \in \Delta_N$ identifies which task or set of tasks are currently lagging, while the outer distribution $\alpha \in \Delta_K$ adjusts domain sampling to support them. Concretely, the inner minimization concentrates probability mass on the tasks with the lowest expected RoI at step t , i.e. whose (normalized) gradient, $\nabla_{\theta} \log \ell(\theta, \mathbf{y})$, has the lowest alignment with the overall update direction $\mathbf{d}_t$ (Eq. (3.6)). This step makes the optimization sensitive to underperforming tasks. Once the slowest-improving tasks are identified, the outer maximization adjusts the domain weights so that more probability mass is placed on those domains whose gradients, i.e. $\mathbf{g}_k$, are most aligned with the gradients of the identified tasks, i.e. $\nabla_{\theta} \log \ell(\theta, \mathbf{y})$ (Eq. (3.6)). In this way, α is shifted toward data sources that can contribute most to their improvement.

Role of Regularizers. The KL terms in Eq. (3.4) serve two purposes: (i) they keep the weights strictly inside the simplex, and (ii) they stabilize the alternating updates by damping noise-driven swings. Equivalently, μ_{α} and μ_z can be interpreted as inverse step sizes for mirror descent dynamics. Defining $\eta_{\alpha} = 1/\mu_{\alpha}$ and $\eta_z = 1/\mu_z$, the resulting updates correspond to *exponentiated-gradient steps* (also known as multiplicative weights) in log-space. Smaller values of μ_{α} and μ_z (larger step sizes) make the updates more responsive to recent task signals, while larger values (smaller step sizes) yield more conservative updates that improve stability by reducing variance. This trade-off is essential for balancing adaptivity with robustness.

Computational Overhead. The extra gradient evaluations required to update task and domain weights scale with the number of domains K and tasks N according to

$$\text{overhead} \approx \frac{N+1}{\Delta T_z} + \frac{K+1}{\Delta T_{\alpha}},$$

where ΔT_{α} and ΔT_z are the update frequencies for the domain and task weights, respectively. Compared to prior domain reweighting algorithm DoGE [29], our task reweighting step introduces an additional $(N+1)\Delta T_z^{-1}$ overhead in computations.

Memory Overhead. The memory overhead stems primarily from storing gradients for each domain/task during the update step. Reweighting requires storing additional gradient buffers of size $2 \times |\text{model parameters}|$ at update steps. This leads to a temporary peak memory overhead of roughly 50% compared to standard pretraining (with AdamW), as illustrated in Figure 3.1.

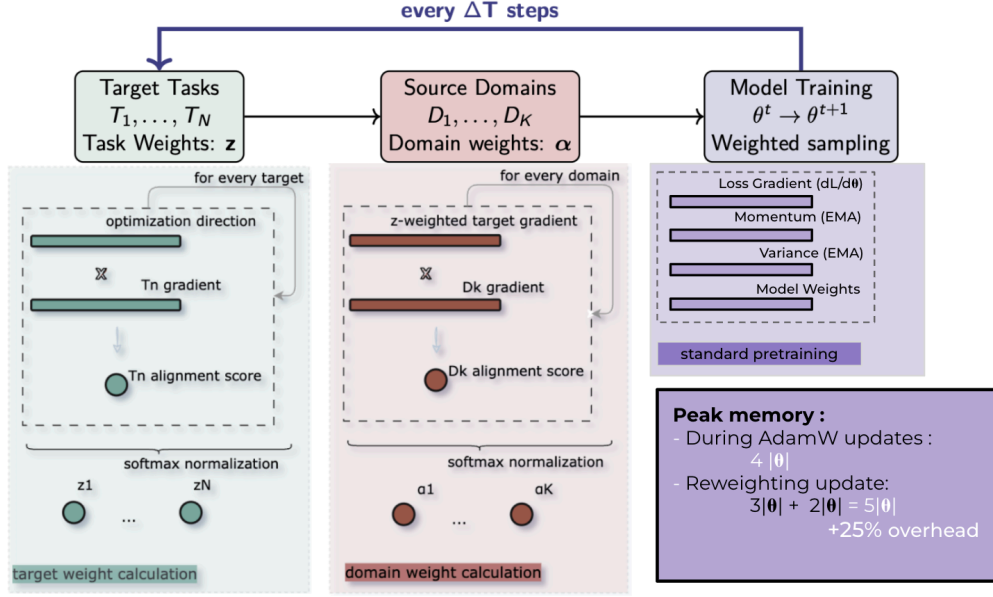


Figure 3.1: Overview of the GRAPE algorithm (shown here for the case $\Delta T_\alpha = \Delta T_z = \Delta T$). The workflow alternates between (Left) computing task Rate-of-Improvement (RoI) metrics and updating task weights $\mathbf{z}$, (Middle) updating domain weights α to align domain gradients with the aggregated target gradient, and (Right) standard model training with weighted data sampling for ΔT steps.

3.2 Theoretical Properties

GRAPE is a regularized minimax formulation optimized via multiplicative updates. The rules (Equation 3.5, Equation 3.6) instantiate online mirror descent on $(\alpha, \mathbf{z})$. Our main result, Theorem 3.1, assumes L -smooth and strongly convex losses—stricter than in LLM pretraining but standard in theory—and yields $\mathcal{O}(1/T)$ convergence to the Pareto set. Although strong convexity need not hold for LLM objectives, the analysis is informative and motivates testing; empirically we observe behavior consistent with the theory.

Beyond convergence, Theorem 3.2 establishes a monotone reduction in the variance of task losses under *smoothness and bounded-gradient* conditions. Concretely, if each l_n is L -smooth and stochastic gradients are bounded, then after an initial transient the variance $\sigma_t^2 = \text{Var}_{n \in [N]}(l_n(\theta_t))$ decreases monotonically. Thus, GRAPE actively mitigates divergence across tasks and promotes more uniform progress than static weighting or naive averaging—an effect that persists empirically even outside ideal conditions. Proofs of Theorem 3.1 and Theorem 3.2 are given in Appendix C.

Theorem 3.1 (Convergence of GRAPE). *Assume each loss $l_n(\theta)$ is L -smooth and μ -strongly convex for all $n \in [N]$, and that the stochastic gradients satisfy $\|\nabla l_n(\theta; x)\| \leq \mathcal{G}$. If the step sizes obey $0 < \gamma_t \leq 1/L$ and the regularization parameters satisfy $\mu_\alpha > 0$ and $\mu_z > 0$, then GRAPE with updates (Equation 3.5, Equation 3.6) converges to a neighborhood of the Pareto-optimal set at rate $\mathcal{O}(1/T)$, where T is the number of iterations. Specifically, for any $\epsilon > 0$ there exists T_0 such that for all $T > T_0$,*

$$\min_{t \in [T]} \left\{ \max_{n \in [N]} \mathbb{E}[l_n(\theta_t)] - \min_{\theta} \max_{n \in [N]} l_n(\theta) \right\} \leq \epsilon.$$

Theorem 3.2 (Monotone Reduction of Task-Performance Variance). *Let $\sigma_t^2 := \text{Var}_{n \in [N]}(l_n(\theta_t))$, the across-task variance at iteration t . Assume each $l_n(\theta)$ is L -smooth, and that the stochastic gradients satisfy $\|\nabla l_n(\theta; x)\| \leq \mathcal{G}$. Then there exists a finite burn-in T_0 such that the variance sequence is eventually monotone nonincreasing:*

$$\sigma_{t+1}^2 \leq \sigma_t^2 \quad \text{for all } t \geq T_0,$$

and hence converges to a minimal basin $\sigma_*^2 \geq 0$.

3.3 GRAPE Algorithm

The complete GRAPE algorithm integrates the multiplicative update rules with standard training is shown in Algorithm 3.1.

Algorithm 3.1 Group Robust Multi-target Adaptive PrEtraining (GRAPE)

```

1: Input:
    $\mathcal{D}_{\text{train}} \triangleq \{D_k\}_{k=1}^K$ ,  $\mathcal{D}_{\text{tgt}} \triangleq \{T_n\}_{n=1}^N$ ,
   optimizer Optimizer, loss function  $\ell(\cdot)$ ,
   initial parameters  $\theta_0$ , initial learning rate  $\gamma_0$ , total steps  $T$ ,
   initial domain weights  $\alpha_0$ , initial task weights  $z_0$ , update frequencies  $\Delta T_\alpha$ ,  $\Delta T_z$ ,
   regularization coefficients  $\mu_\alpha$ ,  $\mu_z$ ,

2: for  $t \in [T]$  do

3:                                     Standard training
4:   Sample from  $\mathcal{D}_{\text{train}}$  a batch  $x \sim \text{mix}(\alpha_t)$ 
5:    $\theta_{t+1} \leftarrow \text{Optimizer}(\theta_t, \nabla_\theta \ell(\theta_t; x))$ 

6:   if  $t \bmod \Delta T_z = 0$  then

7:                                     Task reweighting
8:   Sample from  $\mathcal{D}_{\text{train}}$  a batch  $x \sim \text{mix}(\alpha_t)$ 
9:   For each  $n \in [N]$ :
10:    Sample from  $T_n$  a batch  $y_n \sim T_n$ 
11:     $\alpha_n^{(t)} \leftarrow \langle \nabla_\theta \log \ell(\theta_{t+1}; y_n), \nabla_\theta \ell(\theta_{t+1}; x) \rangle$ 
12:     $\hat{z} \leftarrow z_t \odot \exp(-\frac{\gamma_t}{\mu_z} \alpha^{(t)})$ 
13:     $z_{t+1} \leftarrow \hat{z} / \sum_{n=1}^N \hat{z}_n$ 

14:   else
15:      $z_{t+1} \leftarrow z_t$ 
16:   end if

17:   if  $t \bmod \Delta T_\alpha = 0$  then

18:                                     Domain reweighting
19:   Sample from  $\mathcal{D}_{\text{tgt}}$  a batch  $y \sim \text{mix}(z_t)$ 
20:   For each  $k \in [K]$ :
21:    Sample from  $D_k$  a batch  $x_k \sim D_k$ ;
22:     $\alpha_k^{(t)} \leftarrow \langle \nabla_\theta \ell(\theta_{t+1}; x_k), \nabla_\theta \log \ell(\theta_{t+1}; y) \rangle$ 
23:     $\hat{\alpha} \leftarrow \alpha_t \odot \exp(\frac{\gamma_t}{\mu_\alpha} \alpha^{(t)})$ 
24:     $\alpha_{t+1} \leftarrow \hat{\alpha} / \sum_{k=1}^K \hat{\alpha}_k$ 

25:   else
26:      $\alpha_{t+1} \leftarrow \alpha_t$ 
27:   end if

28: end for

29: Return:  $\theta_T$ 

```

Chapter 4

Experimental Setup

This chapter presents the experimental setup used to evaluate GRAPE. We first introduce the evaluation scenarios and their associated targets, then describe the datasets and domain partitions. We next specify the models and training settings, present the baseline methods, and conclude with the metrics and evaluation protocol.

Notation. We denote the number of source domains by K and the number of target tasks by N . Domain weights are $\alpha \in \Delta_K$, task weights are $\mathbf{z} \in \Delta_N$, and weight updates are performed every ΔT_α and ΔT_z steps respectively.

4.1 Evaluation Scenarios

We evaluate GRAPE in two controlled scenarios that represent complementary challenges in multi-target pre-training. Each scenario defines a collection of source domains, a set of target tasks, and an evaluation objective. The aim is to test whether GRAPE can jointly adapt domain sampling and task weighting to balance performance across multiple targets in a single training run.

1. Domain Mixture for Multi-task Reasoning. In this scenario, the sources comprise diverse domains within English text, while the targets consist of reasoning benchmarks that require a range of knowledge types and cognitive skills. The primary objective is to assess whether adaptive reweighting can enhance overall reasoning performance by identifying beneficial domain–task relationships, while maintaining balanced performance across all tasks rather than concentrating on a select few. (Details in Section 4.2.)
2. Language Mixture for Multilingual Learning. In this scenario, the sources are high-resource languages and the targets are typologically diverse low-resource languages. The evaluation focuses on whether GRAPE can effectively allocate training effort across languages to support multiple targets concurrently, thereby avoiding over-optimization for those most similar to the sources. (Details in Section 4.5.)

These two scenarios are complementary in scope. The first probes *intra-lingual* multi-target balance across reasoning tasks, while the second probes *inter-lingual* multi-target balance across languages. Together, they provide a structured testbed for assessing whether adaptive data mixture optimization can generalize across different definitions of “domain” and different forms of target diversity.

4.2 English Multi-task Reasoning

This scenario evaluates GRAPE on English text, using reasoning benchmarks as targets (details in 4.3) and two complementary datasets for pretraining: semantically clustered domains (ClimbLab; details in 4.4) and source-defined domains (SlimPajama; details in 4.4).

4.3 English Reasoning Suites

We evaluate two benchmark suites that span a range of reasoning skills, knowledge domains, and difficulty levels, ensuring that GRAPE must balance performance across diverse and challenging task types rather than overfitting to any single category.

1. General Reasoning. ARC-E (ARC-Easy)/ARC-C (ARC-Challenge), SciQ, HellaSwag, PIQA, LogiQA.
2. General + Knowledge-Intensive Reasoning. The above plus two benchmarks from specific knowledge domains: MedQA (medical) and MathQA (mathematics).

The benchmarks span different types of reasoning. For each, we briefly describe the task, note relevant properties that make it challenging, and report the human reference population and their performance as indicators of difficulty (Table 4.1).

Scientific Knowledge.

- ARC-E / ARC-C [19]: Grade-school science multiple-choice questions. ARC-E emphasizes factual recall and is relatively accessible, whereas ARC-C was explicitly filtered to remove questions solvable by retrieval methods, making it substantially harder and requiring multi-step reasoning.
- SciQ [92]: Multiple-choice science questions paired with supporting passages. Questions are moderately challenging and test reasoning with textual context.

Commonsense Reasoning.

- HellaSwag [98]: A sentence-completion benchmark with four options. Distractors are adversarially generated to fool statistical models, making it difficult for LMs relying on surface patterns.
- PIQA [10]: Physical commonsense benchmark with binary choices. Questions require reasoning about object affordances and everyday physical interactions.

Logical Deduction.

- LogiQA [53]: Logical reasoning problems adapted from Chinese civil service exams. Designed to test deductive reasoning, making it one of the most difficult benchmarks.

Specific Knowledge Domains.

- MedQA [42]: USMLE-style medical exam questions. Requires specialized medical knowledge and professional training, which makes it difficult for general-purpose LMs.
- MathQA [4]: Math word problems across algebra, geometry, and probability. Solving requires symbolic reasoning and mathematical competence beyond surface understanding.

Ablation Benchmarks In addition to the main evaluation suites, we include two benchmarks that are structurally different from multiple-choice reasoning tasks: GSM8K for math problem solving and KodCode for code generation. These datasets are not part of the joint optimization objective, but serve as diagnostic stress tests. The key distinction is that they require *free-form generation* rather than choosing from a fixed set of options, which makes them substantially harder in a pure pretraining setup. In practice, strong performance on such tasks typically relies on task-specific finetuning, so evaluating them here highlights whether adaptive mixture optimization can nevertheless provide indirect benefits.

Benchmark	Chance	Human Acc.	Annotators
Scientific Knowledge			
ARC (E/C)	25%	~95%	Crowdworkers (science)
SciQ	25%	89.3%	Crowdworkers
Commonsense Reasoning			
HellaSwag	25%	95.6%	Crowdworkers
PIQA	50%	94.9%	Crowdworkers
Logical Deduction			
LogiQA	25%	96%	Civil service
Specific Knowledge			
MedQA	25%	73%	Doctors
MathQA	25%	92.2%	Crowdworkers

Table 4.1: 7 evaluation benchmarks with human performance baselines.

- GSM8K [20]: Grade-school math word problems that require multi-step symbolic reasoning. Evaluation is strict exact match on the final numeric answer. Because the answer space is open-ended, models usually need math-specific finetuning or prompting strategies to achieve strong results.
- KodCode [95]: Synthetic coding tasks paired with reference solutions and unit tests. Evaluation is based on code execution ($Pass@k$), with difficulty stratified into self-verification categories (easy/medium/hard/fail). Since models must generate valid, executable code rather than pick among choices, competitive performance is typically achieved only after code-specific finetuning.

Together, these ablations extend the evaluation beyond multiple-choice reasoning and test whether GRAPE can transfer benefits to tasks that are known to require more specialized adaptation.

Generalization Benchmarks To assess whether GRAPE’s mixture optimization yields skills that transfer beyond its training targets, we evaluate on six benchmarks excluded from pretraining. The suite spans reading comprehension, commonsense, causal, and open-book reasoning, plus broad factual knowledge.

Reading Comprehension.

- RACE [48]: Multi-passage, multi-choice reading comprehension from middle/high-school English exams. Long contexts and distractors require synthesis across sentences and careful elimination.

Commonsense Reasoning.

- WinoGrande [74]: Large-scale pronoun resolution with minimal lexical cues. Adversarially filtered items reduce annotation artifacts, stressing true commonsense.
- CommonsenseQA [80]: Multiple-choice questions built from ConceptNet relations. Requires relational commonsense and disambiguation among plausible distractors.

Causal and Pairwise Inference.

- COPA [71]: Binary-choice causal reasoning (choose the more plausible cause/effect). Demands sensitivity to everyday causality and pragmatic plausibility rather than surface cues.

Open-Book Scientific Reasoning.

- OpenBookQA [57]: Multiple-choice questions requiring combining a small science “open book” with outside commonsense. Challenges include multi-hop retrieval and composition beyond direct lookup.

Broad Knowledge and Multi-Domain Skills.

- MMLU [38]: Multi-task, multi-domain exam-style questions across 50+ subjects. Probes breadth (specialized and general knowledge) and reasoning under domain shift.

Dataset sizes. Table 4.2 reports the number of training and evaluation samples for each benchmark, along with the answer format and evaluation metric. Training split size determines the amount of supervision available, while evaluation sets are used only for performance measurement. Unless noted otherwise, we use the Hugging Face `train` split for training and the `test` split for evaluation (HellaSwag uses `validation`; KodCode is split 90/10 from `train`).

Benchmark	Train samples	Eval samples	Answer Format / Metric
ARC (E/C)	2.6k	3.2k	MCQ / Accuracy
SciQ	12.7k	3.0k	MCQ / Accuracy
HellaSwag	39.9k	10.0k	MCQ / Accuracy
PIQA	16.1k	3.0k	Binary choice / Accuracy
LogiQA	7.4k	0.7k	MCQ / Accuracy
MedQA	127k	12k	MCQ / Accuracy
MathQA	37.2k	3.7k	MCQ / Accuracy
GSM8K (ablation)	7.5k	1.0k	Free-form numeric / EM
KodCode (ablation)	402k	44.7k	Code generation / Pass@k

Table 4.2: 9 benchmark datasets with training and evaluation sample counts.

4.4 Datasets and Domain Partitions

ClimbLab

ClimbLab is a filtered English-text corpus of approximately 1.2 trillion tokens, constructed from the Nemotron-CC [78] and SmolLM-Corpus [3] datasets and further processed using automatic filtering and clustering techniques [24]. It is partitioned into $K=20$ semantic clusters, each representing a coherent topical domain while remaining large enough to support stable training. This structure provides a controlled yet realistic setting for domain reweighting experiments. Table 4.3 lists the 20 clusters along with their dominant topics.

Cluster ID	Topics (description)	Size (B tokens)	Share (%)
1	Mathematics, Algorithms, Programming, Software Development, Data Analysis	17.79	1.52
2	Books, Education, Writing, Literature, AI Ethics, History, Philosophy	109.73	9.38
3	Environmental Education, History, Architecture, Engineering, Classical Music	80.62	6.89
4	Education, Teaching, Science, Engineering, Psychology, Special Education	64.70	5.53
5	International Trade, Business, Economics, AI Consulting, Ethical Decision-Making	92.97	7.94
6	Genetics, Biotechnology, AI, Robotics, Aging, Healthcare, Industrial Automation	70.95	6.06
7	Chemistry, Insects, Taxonomy, Agriculture, Gardening, Veterinary Science	64.04	5.47
8	Gaming, Role-Playing, Board Games, Video Games, Strategy, Fantasy, VR	24.68	2.11
9	Astronomy, Cosmology, Astrophysics, Space Exploration, Urban Planning	12.75	1.09
10	Health, Sleep, Clinical Technology, Fitness, Addiction, Early Childhood Education	135.45	11.57
11	Software Development, Programming, Web Development, JavaScript, Databases	37.11	3.17
12	Technology, Mathematics, Legal Content, Human Rights, Energy Efficiency, Industrial Equipment	78.31	6.69
13	Sports, Cricket, Soccer, Tennis, Basketball, Cultural Heritage, Competition	10.95	0.94
14	Music, Instrumental Practice, Guitar, Jazz, Singing, Composition, Music Theory	15.64	1.33
15	Film, Cinema, Horror, Sci-Fi, Comics, Literature, Criticism, Philosophy	35.24	3.01
16	Sustainability, Climate Change, Renewable Energy, Environmental Conservation	52.24	4.46
17	Cardiovascular Health, Medical Research, Immunology, Cancer Prevention, Drug Therapy	82.23	7.03
18	Technology, Cybersecurity, Social Media, Privacy, AI, Cloud Computing	54.02	4.62
19	Social Media, Digital Communication, Internet Culture, Misinformation, Psychology	50.32	4.30
20	Public Safety, Law Enforcement, Political History, Social Justice, Government	79.47	6.79
Total	—	1170.30	100

Table 4.3: 20 ClimbLab semantic clusters with topics and token distributions.

Corpus structure. The clusters vary greatly in size, from 10.95 B tokens (ID 13) to 135.45 B (ID 10), i.e., over a $12\times$ difference. Large clusters such as 10, 2, 5, 17, 20, 3, and 12 each contribute $\gtrsim 6\%$ of the corpus, while the smallest clusters (13, 9, 14) contribute only $\sim 1\text{--}1.3\%$. This skewed distribution provides a realistic stress test for mixture optimization: uniform sampling naturally favors dominant domains, so effective reweighting must counteract this imbalance to maintain balanced performance on targets.

Beyond scale, the clusters also span diverse content areas, which we group into four broad categories:

- *Technical domains:* Clusters 1 and 11 focus on mathematics and programming.
- *Biomedical domains:* Clusters 6, 7, 10, and 17 cover genetics, chemistry, health, and medicine.
- *Knowledge-rich domains:* Clusters 2, 4, 5, 12, and 20 emphasize education, business, and law.
- *Cultural domains:* Clusters 8, 14, and 15 provide narratives from gaming, music, and film.

This diversity enables us to test whether GRAPE can prioritize domains that improve both general reasoning and specialized tasks, without collapsing onto the largest clusters.

SlimPajama

SlimPajama [77] is a 627B-token English corpus with high-quality preprocessing and deduplication. Consequently, SlimPajama serves as a rigorous testbed for assessing the effectiveness of data mixture optimization algorithms in the context of realistic and heterogeneous textual distributions. SlimPajama consists of $K=7$ source-defined domains that capture diverse textual styles and knowledge areas, as detailed in Table 4.4.

Domain	Description
ArXiv	Academic papers across STEM fields, formal scientific writing
Books	Literature, fiction, and educational texts with narrative structure
Common Crawl	Diverse web content with varying quality and topics
C4	High-quality filtered web text from Common Crawl
GitHub	Source code, documentation, and programming discussions
StackExchange	Technical Q&A across programming, science, and general topics
Wikipedia	Encyclopedic articles covering diverse knowledge domains

Table 4.4: 7 SlimPajama source domains and their characteristics.

Corpus structure. These domains exhibit significant heterogeneity in terms of:

- Writing style: From formal academic prose (ArXiv) to conversational Q&A (StackExchange) and narrative text (Books)
- Technical complexity: Ranging from accessible encyclopedic content (Wikipedia) to specialized programming documentation (GitHub)
- Domain expertise: Covering scientific knowledge, programming skills, general knowledge, and literary content

This source-based partitioning contrasts with ClimbLab’s semantic clustering approach (see 4.4), providing a more realistic but challenging setting for mixture optimization. Unlike semantically coherent clusters, source domains may contain mixed content types and varying quality levels, making it harder for GRAPE to identify optimal domain-task relationships. We use SlimPajama primarily for ablation experiments to test GRAPE’s robustness beyond semantically clustered settings and validate that our method generalizes to real-world heterogeneous corpora.

4.5 Multilingual Transfer Learning

Wiki-40B. Wiki-40B [37] is a cleaned Wikipedia corpus with consistent preprocessing across languages.

Role	Languages
Sources	English, French, German, Spanish, Italian, Russian
Targets	Turkish, Danish, Catalan, Polish, Romanian, Portuguese, Dutch, Ukrainian

Table 4.5: 6 source and 8 target language assignments for Wiki40B.

We define six high-resource source languages and eight low-resource target languages (Table 4.5). Here, the targets are the low-resource languages, on which we evaluate transfer performance.

Linguistic features. The source and target languages in Wiki40B span multiple families, writing systems, and levels of grammatical complexity. Most sources are Indo-European (Romance, Germanic, Slavic), while Turkish is included as a typologically distant non-Indo-European language. Scripts also vary: most languages use the Latin alphabet, but Russian and Ukrainian use Cyrillic. Languages further differ in how much grammatical information is expressed in word endings: English has relatively simple morphology, Romance languages show moderate inflection, and Slavic languages such as Russian, Polish, and Ukrainian are more complex.

Table 4.6 summarizes these properties. It shows which families each language belongs to, their writing system, and the most relevant relations between sources and targets. This context is essential for interpreting our multilingual experiments: some targets benefit from close relatives among the sources (e.g., Spanish–Catalan, Russian–Ukrainian), while others present harder challenges (e.g., Turkish has no close relatives, Cyrillic vs. Latin creates tokenization gaps).

Language	Family / Branch	Script	Relation to Sources
<i>Sources</i>			
English	Germanic	Latin	Simple grammar, bridge language
French	Romance	Latin	Related to Spanish, Italian, Catalan, Portuguese, Romanian
German	Germanic	Latin	Rich grammar, related to Danish, Dutch
Spanish	Romance	Latin	Close to Catalan, Portuguese
Italian	Romance	Latin	Related to Romanian, Catalan, French
Russian	Slavic	Cyrillic	Complex grammar, related to Ukrainian, Polish
<i>Targets</i>			
Turkish	Turkic	Latin	Distant from all sources
Danish	Germanic	Latin	Related to German, Dutch, English
Catalan	Romance	Latin	Very close to Spanish, related to French, Italian
Polish	Slavic	Latin	Related to Russian, Ukrainian
Romanian	Romance	Latin	Related to Italian, French, Spanish
Portuguese	Romance	Latin	Very close to Spanish, related to Catalan
Dutch	Germanic	Latin	Related to German, Danish, English
Ukrainian	Slavic	Cyrillic	Closely related to Russian, Polish

Table 4.6: 14 Wiki40B language families, scripts, and transfer relationships.

4.6 Models and Training Setup

4.6.1 Choice of Architecture

We use GPT-2 decoder-only Transformers trained from scratch at two scales, corresponding to the Small (125M) and Large (774M) variants described in Radford et al. [69].

This architecture is widely adopted in pretraining research because it is simple, well-documented, and reproducible. Training from scratch ensures that the effects we observe can be attributed to our reweighting method rather than to artifacts of prior pretraining.

Using two model sizes strikes a balance between computational feasibility and representativeness: the 125M model allows for faster experimentation and ablation studies, while the 774M model provides a more demanding setting that is closer to the scale of current large language models.

Model	Layers	Embedding dim	Hidden dim	Attention heads	Parameters
GPT-2 Small	12	768	3072	12	125M
GPT-2 Large	36	1280	5120	20	774M

Table 4.7: GPT-2 model architectures and parameter counts.

Training Configuration. Detailed training configuration, including hardware specifications, optimization settings, and hyperparameters, is provided in Appendix B (Section B.2).

Computational overhead. Reweighting introduces additional computational and memory costs compared to standard pretraining (see Chapter 3 for detailed efficiency analysis). With $\Delta T_z = \Delta T_\alpha = 100$ in all runs, Table 4.8 summarizes the approximate overhead for our experimental setups.

Setting	K (domains)	N (targets)	Formula value	Overhead
SlimPajama	7	6	$(6+1)/100 + (7+1)/100$	15%
ClimbLab	20	6	$(6+1)/100 + (20+1)/100$	28%
Multilingual	6	8	$(8+1)/100 + (6+1)/100$	15%

Table 4.8: Compute overhead from GRAPE reweighting across 3 experimental settings.

4.7 Baseline Methods

We compare GRAPE against both static and dynamic reweighting baselines, summarized in Table 4.9. The table outlines how each method handles task weights, domain weights, the update frequency, and the signal used for domain reweighting. Notably, GRAPE is the only approach that adapts both task and domain weights online, using gradient alignment at every update step.

Method	Task Weights (z)	Domain Weights (α)	Update Freq.	Signal for α
GRAPE	Adaptive	Adaptive	Every 100 steps	Gradient alignment
DoGE	Fixed (uniform)	Adaptive	Every 100 steps	Grad. align. (avg. target)
DoGE-PCGRAD	Fixed (PCGrad)	Adaptive	Every 100 steps	Grad. align. (PCGrad target)
ClimbMix	Fixed	Fixed	Offline (precomp.)	Proxy reg. + bootstrapping
REGMIX	Fixed	Fixed	Offline (precomp.)	Proxy model regression
CRISP	Fixed	Fixed	Offline (precomp.)	Embedding similarity

Table 4.9: Comparison of methods by task weights, domain weights, update frequency, and the signal used for α updates.

GRAPE-ClimbMix. We implement a variant of GRAPE that initializes the domain weights α_0 using the ClimbMix weights [24] instead of a uniform distribution. From this initialization, weights are updated adaptively during training using the GRAPE reweighting rule, with all hyperparameters kept identical to the standard GRAPE setup. This allows us to isolate the effect of initialization while preserving GRAPE’s adaptive update dynamics.

DoGE. We implement DoGE [29] following the out-of-domain generalization setting. Domain weights α are updated online during training based on the alignment between the gradient of each source domain and a reference gradient defined as the average across all target tasks. Domains with higher alignment are upweighted, while those with negative alignment are downweighted. In our implementation, the weights are updated every $\Delta T_\alpha = 100$ steps under uniform task weights (z), and the same regularization hyperparameters as GRAPE are applied. This procedure maintains α as a dynamic quantity that evolves throughout pretraining.

DoGE-PCGRAD. We implement a multi-task extension of DoGE [29] by incorporating PCGrad [97]. At each reweighting step, gradients from the target tasks are sequentially processed: when a pair of gradients conflict (negative dot product), one is projected onto the normal plane of the other. A random ordering of tasks is used to avoid bias, and the final conflict-free reference gradient g_{PCGrad} is computed as the average of the adjusted task gradients. Domain weights α are then updated every $\Delta T_\alpha = 100$ steps based on their alignment with g_{PCGrad} . The same regularization hyperparameters as GRAPE are applied. This setup allows DoGE to adaptively reweight domains in a multi-task setting while avoiding gradient conflicts, though it requires storing all target gradients in memory at each update.

DoGE-MGDA (Ablation Study Only). For ablation, we extend DoGE with MGDA [22] to explore multi-task reweighting based on Pareto-stationarity. At each reweighting step, MGDA computes a convex combination of target task gradients to find a minimum-norm point in their convex hull, approximating a direction where no task can be improved without increasing loss in another. This reference gradient g_{MGDA} is used to update domain weights α every $\Delta T_\alpha = 100$ steps via gradient alignment. While MGDA offers theoretical guarantees for multi-task trade-off, it incurs extra quadratic programming overhead and is evaluated only in ablation (not in main results). Regularization hyperparameters match those used by GRAPE.

DoGE-CAGrad (Ablation Study Only). For completeness, we also evaluate a multi-task ablation based on CAGrad [52]. At each reweighting step, CAGrad decomposes each target task gradient into shared and conflicting parts, rescales the gradients to reduce negative transfer, and then average-adjusts them to produce the reference direction g_{CAGrad} for updating α . As with other gradient surgery methods, all target gradients must be stored per update. This ablation is included for comparison only and is not a core baseline. Regularization matches GRAPE.

ClimbMix. We use the domain weights provided by ClimbMix [24]. These weights are derived in the original paper through a three-round regression-and-bootstrapping procedure guided by validation performance on PIQA, ARC-E, and HellaSwag. In each round, candidate weight vectors are trained with 62M- and 350M-parameter proxy models, their validation scores are recorded, and a regression model is fitted to predict performance from the weights. The predictor is then used to rank unseen candidates, from which new vectors are sampled, pruned, and re-evaluated in the next round. After three iterations, this procedure produces a final weight vector α^* . In our experiments, we take this final vector directly and keep it fixed throughout pretraining.

REGMIX. We implement REGMIX [54] to pre-compute offline static domain weights (α) used throughout the entire pretraining. We optimize for average performance across all target tasks. The method employs a two-stage approach: first training multiple small proxy models with different data mixtures, then learning a regression function to map mixture configurations to expected average performance across target tasks. Following Liu et al. [54], we train 768 TinyLlama-1M proxy models on 1B tokens each with varying data mixture configurations. Of these, 512 are used to fit the regression model, while 256 are reserved for evaluating regressor performance. For each proxy model, we record average validation loss across target tasks, then train a LightGBM regressor to predict domain weights that minimize this average loss. The predicted weights remain fixed throughout main model pretraining. Other hyperparameters follow the default values from Liu et al. [54].

CRISP. We implement CRISP [36] to pre-compute static embedding-based domain weights used throughout pretraining. The method calculates domain weights by measuring similarity between training and target domains in embedding space. For each training and target domain, we compute sequence embeddings using SBERT MiniLM-L6-v2 on 512-token chunks for training domains and on individual samples (padded or truncated to 512 tokens) for target domains. To avoid overfitting, we select 5 representative centroids for each training domain $D_k, k \in [K]$ and each target domain using k-means clustering. We then apply nearest-neighbor classification ($k=5$) to map target domain centroids to the closest centroids within each training domain. Finally, we compute domain weights by aggregating the distance-weighted k-NN assignments of all target centroids to training domains.

4.8 Evaluation Metrics.

We adopt two evaluation metrics:

- Language modeling: log-perplexity on held-out validation/test sets.
- Reasoning: log-perplexity and 5-shot exact match accuracy on benchmark splits.

For accuracy results, we report the highest performance achieved over three random seeds, as measured by the LLM-Eval framework [5].

Chapter 5

English Reasoning Experiments

5.1 Aggregate Performance: Data Efficiency and Convergence

We evaluate GRAPE against uniform sampling, RegMix, CRISP, and DoGE baselines on a 125M parameter model trained on ClimbLab data, measuring 5-shot accuracy across six reasoning tasks as a function of training tokens.

GRAPE achieves superior data efficiency and final performance compared to all baselines. At the 1.6B token mark, GRAPE delivers a +1.7 point improvement over the strongest competitor (DoGE) while reaching comparable performance $1.8\times$ earlier. The accuracy trajectories reveal distinct learning patterns: GRAPE improves smoothly and monotonically throughout training without early saturation, while DoGE rises rapidly initially but tapers off later, widening the performance gap. CRISP follows a similar but weaker pattern, and static strategies (uniform and RegMix) show consistently shallow improvements. These results demonstrate that GRAPE not only reaches strong performance more efficiently but also sustains learning where alternatives stall, establishing its central advantage of stronger generalization with fewer tokens.

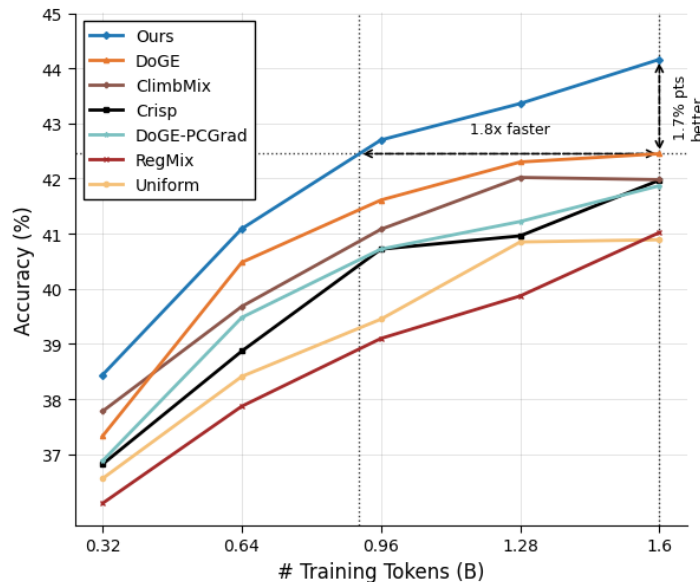
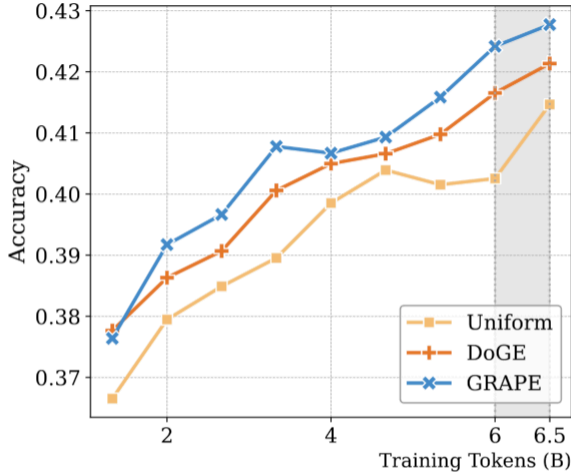


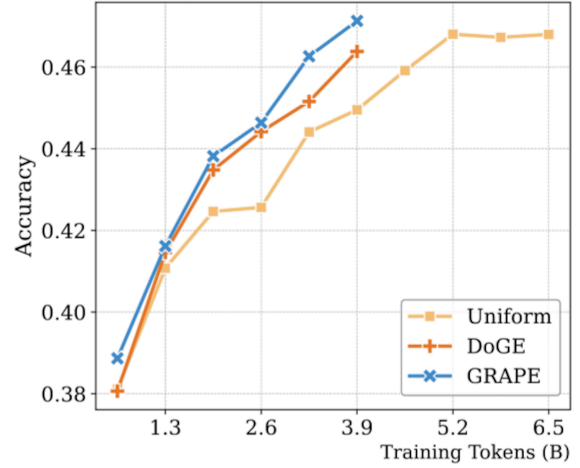
Figure 5.1: **125M, 1.6B tokens (ClimbLab), 6-task setting.** Average 5-shot accuracy vs. training tokens. *GRAPE demonstrates superior data efficiency and final performance.*

We extend our evaluation to 0.7B parameter models trained on both SlimPajama and ClimbLab datasets to assess scaling behavior. GRAPE’s advantages persist with increased model capacity across both datasets. On 0.7B models, both DoGE and GRAPE substantially outperform the uniform sampling baseline, confirming the

general efficacy of adaptive domain reweighting approaches. For ClimbLab, GRAPE achieves comparable scores to the uniform baseline with 40% fewer tokens while maintaining superior final performance. The SlimPajama results show similar patterns, with GRAPE consistently leading throughout training and achieving the highest final accuracy. These scaling results demonstrate the robustness of our approach across different datasets and model sizes.



(a) SlimPajama dataset



(b) ClimbLab dataset

Figure 5.2: **0.7B, full runs, 6-task setting.** Average 5-shot accuracy vs. training tokens across different datasets. *GRAPE maintains superior performance and data efficiency at larger scale on both corpora.*

5.2 Task-Level Performance Results

We evaluate GRAPE against baseline reweighting methods on six English reasoning benchmarks, reporting task-level log-perplexity (Table 5.1) and 5-shot accuracy (Table 5.2).

GRAPE consistently outperforms all baselines on both metrics with more balanced improvements across tasks. In terms of perplexity, GRAPE attains the lowest average log-perplexity (3.45), a $\sim 7\%$ reduction vs. Uniform (3.71) and an additional $\sim 3\%$ over DoGE, the strongest domain-only method. The largest absolute gains vs. Uniform appear on HellaSwag (-0.49), ARC-E (-0.29), and ARC-C (-0.28), spanning factual science QA and narrative commonsense. For accuracy, GRAPE yields the highest average 5-shot accuracy (44.16%), improving over Uniform by +3.3 points overall, with notable gains on ARC-E (+6.4), SciQ (+6.9), and PIQA (+2.1). While other adaptive methods show task-specific strengths—e.g., CRISP on LogiQA, DoGE on SciQ—GRAPE’s task-aware design produces robust, balanced improvements across factual, physical, and narrative reasoning.

Method	Task Log-Perplexity						Avg.
	ARC-C	ARC-E	LogiQA	PIQA	SciQ	HellaSwag	
Uniform	3.69	3.68	3.98	3.45	4.29	3.19	3.71
RegMix	3.59	3.60	4.03	3.54	4.23	3.20	3.70
ClimbMix	3.61	3.58	3.46	4.06	<u>2.98</u>	3.98	3.61
CRISP	3.58	3.59	3.95	3.50	4.28	3.12	3.67
DoGE-PCGrad	3.58	3.54	3.85	3.46	4.16	3.07	3.61
DoGE	3.48	<u>3.44</u>	3.90	<u>3.44</u>	4.12	<u>2.94</u>	3.55
GRAPE	3.41	3.39	<u>3.82</u>	3.41	3.97	2.70	3.45
GRAPE-ClimbMix	<u>3.45</u>	3.47	3.46	4.03	2.90	3.82	<u>3.52</u>

Table 5.1: **125M, 1.6B tokens (ClimbLab), 6-task setting.** Task log-perplexity (lower is better). *GRAPE achieves the best average across all reasoning tasks.**

Method	Task Accuracy (%)						Avg.
	ARC-C	ARC-E	LogiQA	PIQA	SciQ	HellaSwag	
Uniform	23.12	40.74	27.96	58.98	67.50	27.02	40.89
RegMix	24.66	42.85	26.11	59.30	65.90	27.30	41.02
ClimbMix	24.83	42.85	25.19	60.55	70.80	27.67	41.98
CRISP	22.70	44.32	29.65	57.34	70.40	27.40	41.97
DoGE-PCGrad	24.66	42.63	27.19	58.71	70.50	27.50	41.86
DoGE	24.57	45.33	25.35	59.52	72.20	27.77	42.45
GRAPE	26.11	47.14	27.65	<u>61.10</u>	74.40	28.56	44.16
GRAPE-ClimbMix	<u>26.02</u>	<u>46.72</u>	27.19	62.24	<u>73.80</u>	<u>28.01</u>	<u>43.99</u>

Table 5.2: **125M, 1.6B tokens (ClimbLab), 6-task setting.** Task 5-shot accuracy (higher is better). *GRAPE achieves balanced improvements across all reasoning tasks.**

5.3 Analysis of Baseline Domain Weight Profiles

Fig. 5.3 reports the end-of-training domain weights, i.e., the mixture over corpus clusters used to sample pretraining tokens at the last checkpoint after consuming the 1.6B-token budget. For adaptive baselines, these equal their domain mixtures at the last checkpoint. All baselines fix target-task weights at $1/N$ and differ only in how they allocate pretraining tokens across sources. Uniform is flat over targets and domains and serves as a reference.

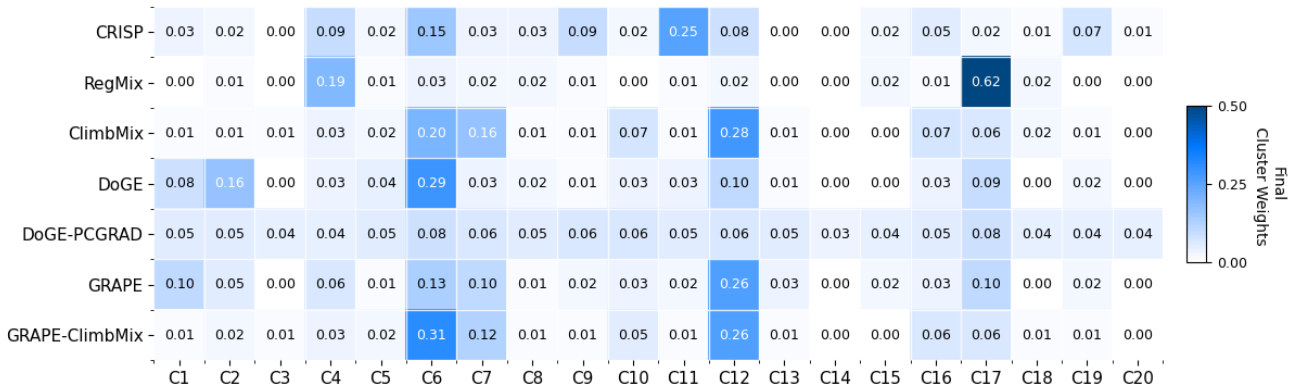


Figure 5.3: **125M, 1.6B tokens (ClimbLab), 6-task setting.** End-of-training domain weights heatmap. *Adaptive methods converge to STEM-oriented clusters.*

*Bold indicates best performance; underline indicates second-best.

We relate the 6-task results (Tables 5.1–5.2) to the domain-weight profiles in Fig. 5.3. Across stronger methods, four clusters recur with substantial mass: C1 (Math/Computing), C6 (Life Sciences/Tech), C12 (Technical/Legal), and C17 (Medical). These STEM-oriented sources typically transfer to factual science QA (ARC-E, ARC-C), science reading comprehension (SciQ), and—via structural regularities—LogiQA; narrative/cultural clusters align less directly with HellaSwag and parts of PIQA. Sustained mass on C1/C6/C12/C17 is a reliable indicator of strong outcomes on ARC-E/ARC-C and SciQ, with C1/C12 additionally supporting LogiQA. Methods that adapt only the *domain weights* (e.g., DoGE, DoGE-PCGrad) deliver substantial early gains on science-oriented targets but tend to stall on PIQA/HellaSwag. By contrast, pairing STEM-oriented domain-weight profiles with *task-aware* prioritization (GRAPE, GRAPE-ClimbMix) sustains improvements across all targets (Tables 5.1–5.2; Fig. 5.3).

RegMix (static). Within the 1.6 B-token budget, RegMix assigns ≈ 1.0 B to C17 (cardiovascular/biomedical) and ≈ 0.6 B across the remaining clusters. We observe small shifts in log-perplexity and no corresponding gains on multiple-choice evaluations (ARC-C/ARC-E, PIQA, SciQ, HellaSwag, LogiQA).

ClimbMix (static; tuned for ARC-E, PIQA, HellaSwag). Under the same 1.6 B cap, ClimbMix uses ≈ 0.45 B for C12 (technology/mathematics/legal/industrial), ≈ 0.32 B for C6 (genetics/AI/robotics/healthcare), and ≈ 0.26 B for C7 (chemistry/agriculture). We observe improvements on ARC-E and PIQA and competitive results on HellaSwag; aggregate accuracy plateaus on the remaining tasks.

CRISP (static). Within the 1.6 B budget, CRISP uses ≈ 0.40 B for C11 (software/programming/web) and ≈ 0.24 B for C6. We observe an increase on LogiQA and limited changes on PIQA, SciQ, and HellaSwag.

DoGE-PCGrad (adaptive). DoGE-PCGrad distributes the 1.6 B budget broadly without sustained peaks. We observe small positive deltas across tasks but no best-in-class results.

DoGE (adaptive). Within the 1.6 B cap, DoGE increases shares for C6 with a secondary increase for C12 while keeping C2 (books/education) present. We observe reliable gains on ARC-E, PIQA, and SciQ, with smaller changes on the remaining benchmarks.

GRAPE-ClimbMix (adaptive). Starting from the ClimbMix prior under the 1.6 B budget, we observe larger gains on science- and practice-oriented evaluations and a small reduction in overall balance relative to vanilla GRAPE.

5.4 Weight Evolution: Emergent Task Curriculum and Domain Selection

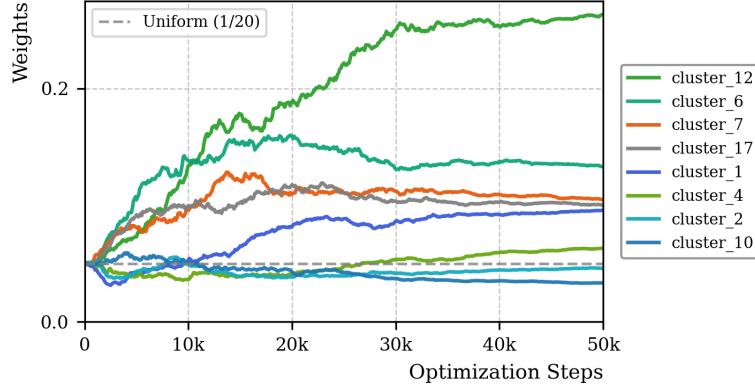
We analyze the evolution of task weights and domain weights during training to identify how GRAPE allocates training signal across heterogeneous reasoning tasks and their underlying text sources (Figures 5.4a–5.4c, Table 5.3). Task weights reveal which benchmarks are prioritized as training progresses, while domain weights quantify the emphasis placed on different clusters of the ClimbLab corpus.

GRAPE uncovers a staged curriculum that emerges from training dynamics, with task weights driving domain weight allocation towards STEM-oriented clusters. In the early phase (0–2.5B tokens), ARC-E and ARC-C dominate, grounding the model in factual science reasoning. Around 5B tokens, PIQA rises sharply as physical commonsense becomes the primary bottleneck. Later, allocation stabilizes around ARC-E, PIQA, and HellaSwag, covering factual, physical, and contextual reasoning. LogiQA and SciQ remain downweighted yet still improve: SciQ benefits from STEM-heavy pretraining clusters, while LogiQA gains indirectly from code- and math-related clusters. At the domain level, GRAPE prioritizes STEM-oriented clusters (math/computing,

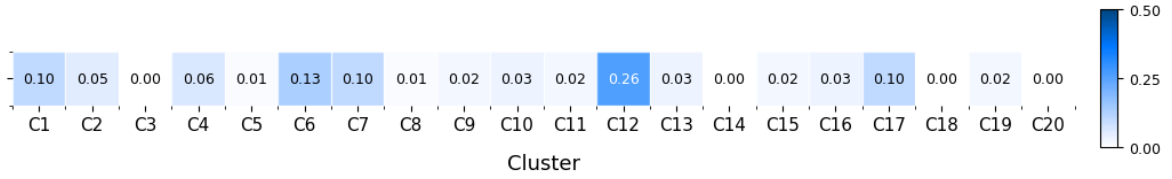
life sciences, technical/legal, medical texts) that supply the structured reasoning needed for tasks like ARC-E and ARC-C. In contrast, commonsense tasks like PIQA and HellaSwag require explicit task-level upweighting since their reasoning style is less represented in the STEM-heavy source distribution. This partitioning shows how GRAPE allocates signal hierarchically: task weights select which reasoning skills to prioritize, and domain weights react by emphasizing source clusters that support those target skills. Together, this hierarchical weighting yields the balanced improvements in Table 5.2.

Cluster	Domain Focus	Key Characteristics
1	STEM - Math/Computing	Multi-step procedural problems
6	STEM - Life Sciences/Tech	Domain-specific scientific reasoning
12	STEM/Legal - Mixed	Technical & legal reasoning
17	STEM - Medical Sciences	Medical research & analysis

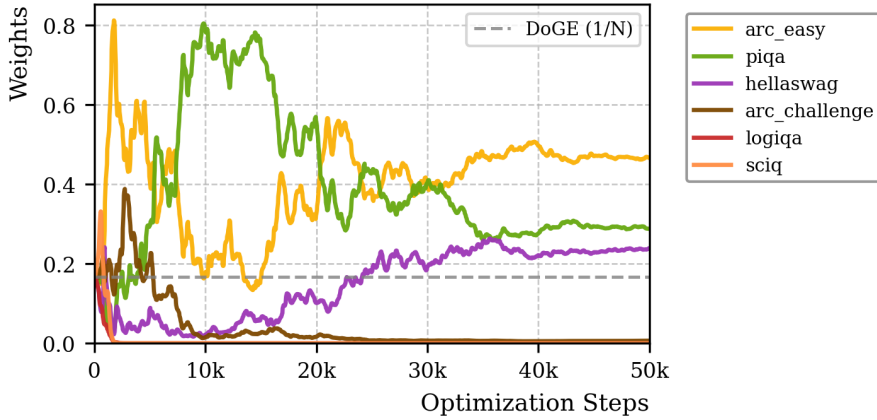
Table 5.3: High-weight domain clusters identified by GRAPE emphasize STEM and procedural reasoning.



(a) Trajectory of domain weights during training (top 8 clusters by final weight).



(b) Heatmap of final domain weights across all clusters.



(c) Evolution of target task weights (dynamic curriculum).

Figure 5.4: **125M, 1.6B tokens (ClimbLab), 6-task setting.** Domain and target weight trajectories: (a) domain weight evolution, (b) final domain weight heatmap, (c) target task weight evolution. *STEM clusters dominate while target weights reveal an emergent curriculum.*

5.5 Scaling Behavior at 0.7B Parameters

We evaluate GRAPE at 0.7B parameters on the same 6-task setting.

Method	Individual Task Accuracy (%)						Avg.	Tasks > Uniform
	ARC-C	ARC-E	LogiQA	PIQA	SciQ	HellaSwag		
Uniform	28.07	52.99	28.57	<u>62.73</u>	84.00	<u>32.33</u>	48.11	–
DoGE	29.61	57.58	25.96	61.70	86.40	31.05	<u>48.72</u>	3/6
GRAPE	<u>28.92</u>	<u>55.60</u>	28.58	65.56	<u>84.50</u>	34.19	49.56	6/6

Table 5.4: **0.7B, full run (ClimbLab), 6-task setting.** Task 5-shot accuracy at larger scale. *GRAPE improves on all tasks and achieves best average.**

The superior performance of GRAPE scales with model size but narrows as scale increases. At the end of their respective training runs, GRAPE outperforms both uniform and DoGE mixtures, achieving the best average accuracy at both scales (as seen in Table 5.4). However, the margin over the uniform mix and the best baseline is smaller (Table 5.5). The gains concentrate on skills emphasized later in the curriculum (commonsense/physical reasoning), while early reading-style tasks change little. A natural explanation is a mismatch between model size and optimization horizon: the 0.7B run was not trained for a proportionally longer number of steps/tokens, so its final checkpoint reflects an earlier slice of its curriculum where improvements accumulate more slowly. In addition, the reweighting dynamics (applied every 100 steps and scaled by the instantaneous learning rate) and hyperparameters were inherited from the 125M setting, which can dampen observable gains at larger scale.

Scale	Uniform	DoGE	GRAPE	Δ vs. Uniform	Δ vs. DoGE
125M	40.89	42.45	44.16	+3.3	+1.7
0.7B	48.11	48.72	49.56	+1.5	+0.8

Table 5.5: **125M and 0.7B, ClimbLab, 6-task setting.** Average 5-shot accuracy and relative gains. *GRAPE’s advantage narrows as model scale increases.*

Skill-based progression persists through scaling. Early learning emphasizes pattern extraction and reading-like abilities (ARC-E/C), then attention shifts to more demanding reasoning and world-knowledge-based abilities (PIQA, HellaSwag) at both 125M and 0.7B, as seen in Figure 5.5. This persistence suggests that *ordering the learning focus matters for effective training*. In practice, data schedules that *learn what to learn*—allocating emphasis to abilities that unlock the next tranche of gains rather than pushing uniformly—tend to make better use of compute and data. This points to adaptive scheduling as a first-class principle for pretraining, with approaches like GRAPE that optimize the learning focus over time being the right direction for future large-scale training.

Temporal concentration profile changes with scale. At 125M, concentration is sharp early and later broadens; at 0.7B, the mix starts more spread out and becomes more concentrated near the end. This may reflect a capacity effect; greater representational room supports multiple abilities in parallel before narrowing focus. It could also relate to schedule coupling; because reweighting tracks the learning rate, linear decay preserves larger effective updates deeper into training, while cosine decay damps them earlier, changing how strongly late reallocations appear. Another consideration is horizon, our 0.7B snapshot at 7.8B tokens ($4.875\times$ the 125M run) may capture an earlier stage of the learning trajectory; the weights don’t seem to have stabilized yet and any later widening of the mix may fall outside the observed window.

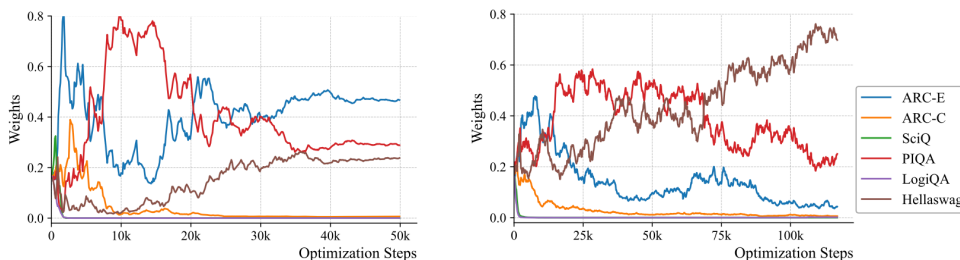


Figure 5.5: **125M vs. 0.7B, full runs (ClimbLab), 6-task setting.** Target-weight evolution comparison.

5.6 Impact of Task Diversity on Weight Dynamics

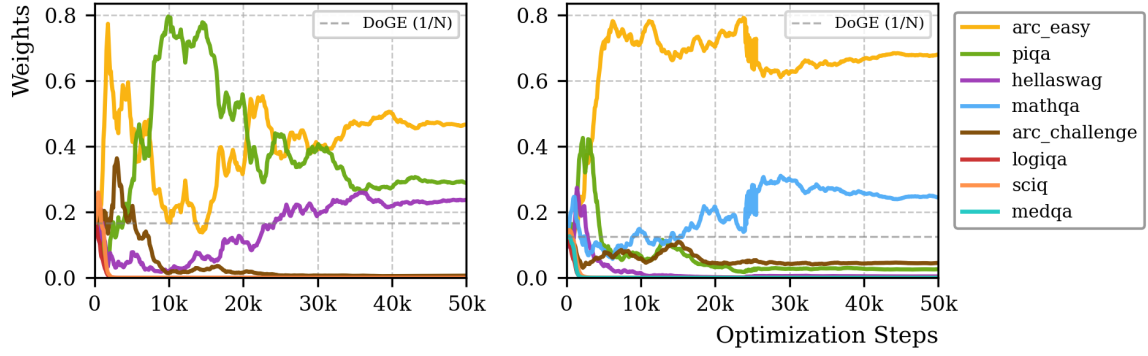
We extend our 6-task setup by adding MathQA (mathematical reasoning) and MedQA (clinical reasoning) to create an 8-task suite, training GRAPE on both configurations to examine how task diversity affects weight allocation patterns. As Table 5.6 shows, GRAPE achieves the lowest average log-perplexity with competitive accuracy across the full eight-task suite.

Adding reasoning tasks reshapes both the target curriculum and the domain mixture, enabling GRAPE to outperform the strongest baseline. In the 8-task setting, physical reasoning receives initial attention before the focus shifts to science and mathematical reasoning, with the weights ultimately converging to a less distributed mixture dominated by ARC-E and MathQA (Fig. 5.6, top). The persistence of MathQA despite its low accuracy score reflects its difficulty; GRAPE recognizes this bottleneck and allocates sustained attention throughout training to improve performance. The domain weights accommodate this less distributed target mixture: C1 (math/programming) receives substantially more weight in the 8-task setting to support the difficult mathematical reasoning requirement, while the most upweighted domains (C1, C6, C2) become more concentrated than in the 6-task case, which spreads mass across C12, C6, C1, C17, and C7 (Fig. 5.6, second/third). Notably, these fewer, highly upweighted domains also lie close together in the embedding space (Fig. 5.6, bottom). This concentration likely reflects the reduced diversity of the target distribution: as the task weights become less evenly spread, the domain allocation follows by focusing on the most relevant sources.

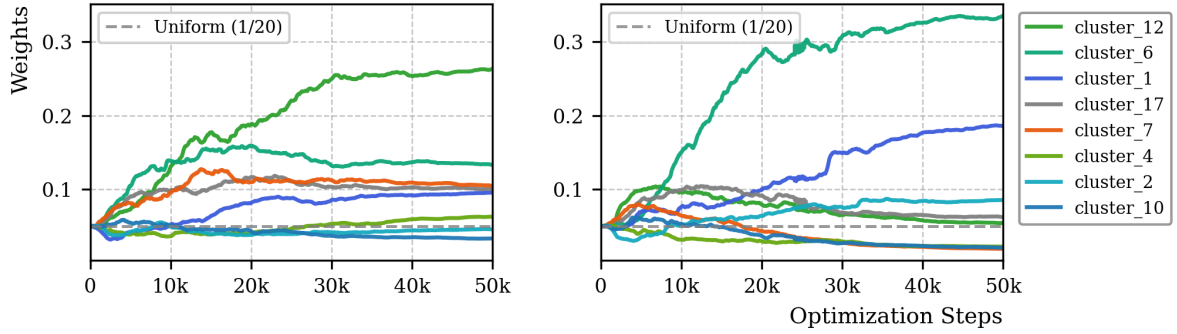
	ARC-C	ARC-E	LogiQA	PIQA	SciQ	HellaSwag	MedQA	MathQA	Avg.
Log-Perplexity (↓)									
Uniform	3.91	3.98	3.69	<u>4.44</u>	3.42	<u>4.15</u>	3.87	4.53	4.00
DoGE	<u>3.77</u>	<u>3.76</u>	<u>3.62</u>	<u>4.44</u>	<u>3.23</u>	4.16	<u>3.33</u>	<u>4.49</u>	<u>3.85</u>
GRAPE	3.67	3.60	3.57	4.32	3.07	4.11	2.56	4.34	3.66
Accuracy (%) (↑)									
Uniform	23.12	40.74	27.96	58.98	67.50	27.02	20.90	20.80	35.88
DoGE	19.97	<u>46.17</u>	22.58	<u>59.41</u>	<u>78.80</u>	27.21	<u>20.74</u>	<u>21.01</u>	<u>36.99</u>
GRAPE	<u>21.84</u>	48.82	<u>23.66</u>	59.96	80.20	<u>27.18</u>	19.72	22.04	37.93

Table 5.6: **125M, 1.6B tokens (ClimbLab), 8-task setting.** Perplexity and 5-shot accuracy. *GRAPE achieves the lowest average perplexity and accuracy.**

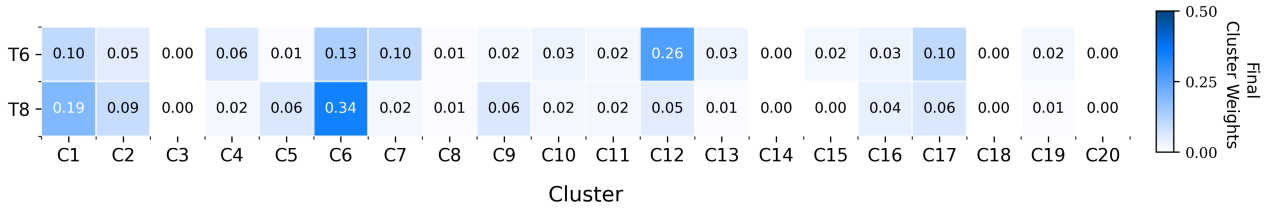
*Bold indicates best performance; underline indicates second-best.



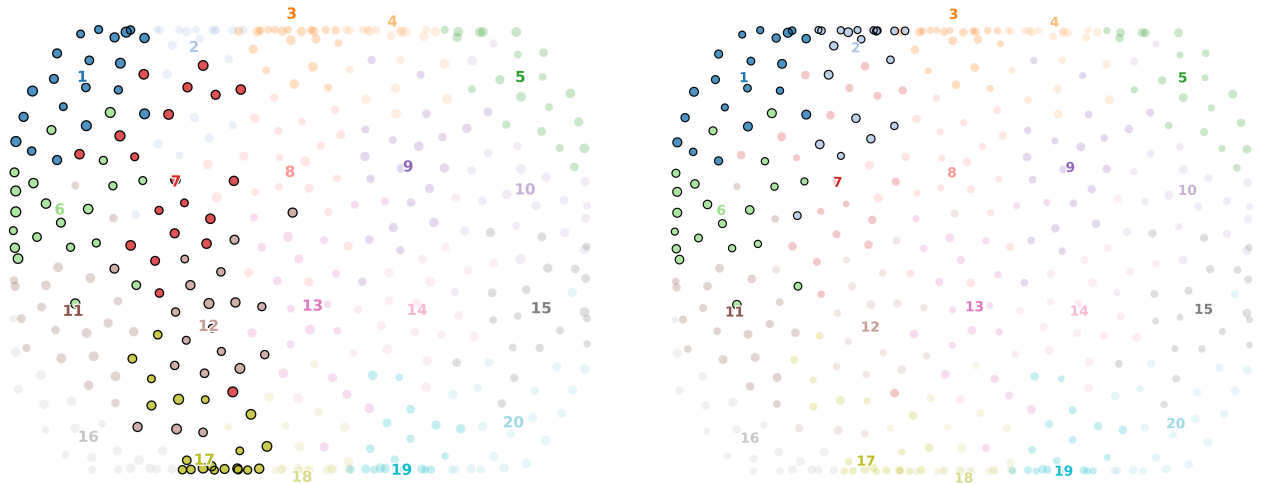
(a) Target-task weight trajectories over training.



(b) Domain (cluster) weight trajectories (top eight by final mass).



(c) Heatmap of final domain weights.



(d) Two-dimensional projection of document embeddings by cluster (left: 6-task, right: 8-task). Points are documents; numbers denote cluster IDs.

Figure 5.6: **125M, 1.6B tokens (ClimbLab), 6-task (left) and 8-task (right) settings.** (a) Target weight trajectories, (b) domain weight trajectories, (c) final domain heatmaps, (d) embedding projections with clusters having final weight above uniform emphasized. *Adding tasks concentrates domain allocation into a tighter STEM-centric mixture.*

5.7 Adaptive vs. Static Reweighting on Matched Targets

We evaluate GRAPE against ClimbMix—our strongest static baseline—using ClimbLab’s three-target setup (HellaSwag, ARC-E, and PIQA) to isolate the benefit of adaptive reweighting under identical coverage and targets.

GRAPE outperforms ClimbMix on all targets while generalizing better to non-targets. It raises both target and overall scores—gains on LogiQA and SciQ more than offset a small drop on ARC-C. Inspecting the target-weight dynamics (Figure 5.7a), GRAPE first upweights physical reasoning (PIQA) and then settles into a balanced, non-uniform allocation in which commonsense reasoning (HellaSwag) receives the most mass. This training-aware trajectory yields more transferable reasoning skills. In contrast, ClimbMix pays a substantial upfront search cost (typically 112 proxy trainings with a 350M proxy) to fix domain weights, whereas GRAPE discovers and refines the mixture online. The benefit is evident even at comparable scales; as models grow and training horizons lengthen—shifting domain sensitivities over time—the gap between a static mixture and GRAPE’s adaptive policy should widen further.

Method	ARC-C	ARC-E	LogiQA	PIQA	SciQ	HellaSwag	Avg. (targets)	Avg. (all)
ClimbMix	24.83	42.85	25.19	60.55	70.80	27.67	43.69	41.98
GRAPE	24.23	42.97	26.73	61.43	72.00	28.00	44.29	42.56

Table 5.7: **125M, 0.66B tokens (ClimbLab), 3-task setting.** Task 5-shot accuracy on matched targets. *GRAPE improves all targets and generalizes better to non-targets.**

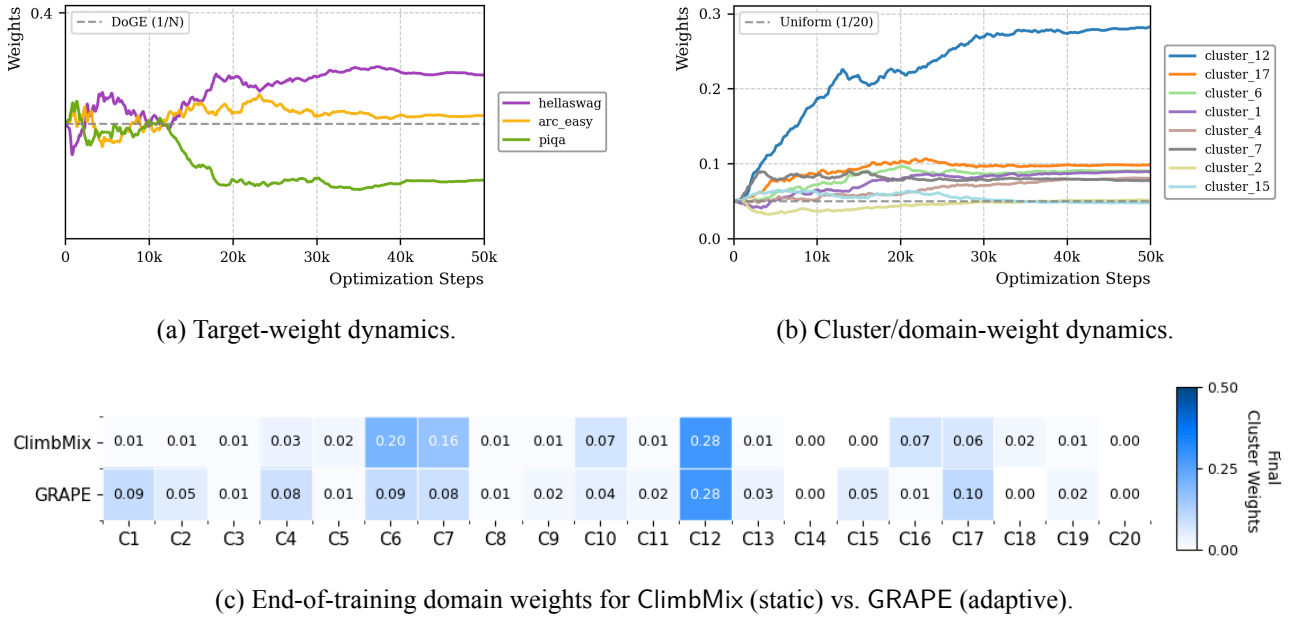


Figure 5.7: **125M, 0.66B tokens (ClimbLab), 3-task setting.** (a) Target weight dynamics, (b) domain weight dynamics, (c) final domain allocations. *GRAPE achieves similar final domains but superior outcomes through dynamic adaptation.*

5.8 Generalization Performance on Unseen Benchmarks

We test GRAPE on six reasoning benchmarks that were not used during pretraining, comparing against a Uniform pretraining mixture and DoGE. These benchmarks include reading comprehension (RACE), causal reasoning (COPA), and broad knowledge tasks (MMLU) that require different reasoning patterns than the scientific and commonsense tasks used in training. They also include commonsense reasoning (WinoGrande, CommonsenseQA)

*Bold indicates best performance; underline indicates second-best.

and scientific reasoning (OpenBookQA) tasks that share similar reasoning requirements with the training tasks but present them in different formats and contexts.

As shown in Table 5.8, GRAPE achieves the best performance on four benchmarks—RACE, WinoGrande, OpenBookQA, and COPA—while remaining competitive on MMLU and CommonsenseQA. The largest improvements occur on OpenBookQA and WinoGrande, with substantial gains also on RACE and COPA.

GRAPE has developed genuine reasoning capabilities rather than simply overfitting to its training tasks. By shifting probability mass toward training targets predicted to be *hurt* by the current step, GRAPE indirectly prioritizes data that support cross-sentence evidence synthesis, composition of dispersed facts, and causal reasoning—skills central to ARC-E, PIQA, and HellaSwag. The strong performance on unseen tasks—particularly the largest gains on OpenBookQA, WinoGrande, RACE, and COPA—indicates that these competencies transfer beyond the specific training contexts, suggesting that GRAPE has learned robust reasoning skills rather than task-specific patterns. As expected, tasks requiring primarily breadth of knowledge (MMLU, CommonsenseQA) show smaller improvements, consistent with GRAPE’s focus on developing reasoning-intensive capabilities rather than broad factual knowledge.

Method	RACE	WinoGrande	OpenBookQA	MMLU	CommonsenseQA	COPA	Avg	# > uniform
Uniform	25.65	49.80	15.40	25.07	20.15	57.00	32.18	–
DoGE	<u>26.12</u>	49.80	<u>26.40</u>	26.43	<u>19.82</u>	<u>60.00</u>	34.93	4
GRAPE	27.37	52.49	28.60	<u>25.53</u>	18.92	61.00	35.99	5

Table 5.8: **125M, 1.6B tokens (ClimbLab), 6-task setting.** 5-shot accuracy $\uparrow$ (%) on unseen tasks. *GRAPE demonstrates strong generalization beyond training targets.**

5.9 Robustness Across Target Combinations

We evaluate GRAPE’s performance across twelve distinct multi-task configurations (T1–T12) spanning diverse reasoning domains. Each configuration combines 2–4 targets drawn from our benchmark suite, yielding heterogeneous task mixtures that stress different aspects of reasoning.

GRAPE delivers consistent gains across all target task combinations. As shown in Table 5.9, GRAPE achieves the strongest results across all twelve multi-task configurations, outperforming both DoGE and Uniform baselines. Notably, GRAPE maintains its advantage even in the most challenging, high-cardinality mixtures, with worst-case performance remaining robust across all configurations. This indicates that the method does not merely excel on easier combinations, but sustains reliable results even when interference and imbalance are greatest, highlighting its resilience to mix difficulty.

*Bold indicates best performance; underline indicates second-best.

Method	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10	T11	T12	Avg	Worst
Log-Perplexity (↓)														
Uniform	4.30	4.01	4.11	3.75	3.85	3.62	3.77	3.71	4.02	4.29	4.34	4.26	3.99	3.62
DoGE	4.18	4.01	4.00	3.67	3.70	3.63	3.68	3.64	3.84	4.13	4.13	4.07	3.85	3.63
GRAPE	3.53	3.61	3.70	3.39	3.36	3.03	3.45	3.28	3.58	3.66	3.55	3.48	3.50	3.03
Accuracy (%) (↑)														
Uniform	17.4	27.9	12.7	13.8	27.0	14.1	10.1	27.1	36.6	31.1	23.5	22.2	22.9	10.1
DoGE	18.2	28.6	13.1	14.6	28.2	14.0	10.2	28.1	37.7	31.7	25.0	23.8	23.8	10.2
GRAPE	18.9	29.1	13.4	14.4	28.9	15.2	10.4	28.9	38.3	33.2	25.6	24.4	24.4	10.4

Table 5.9: **125M, 0.66B tokens (SlimPajama), T1–T12 settings.** Average log-perplexity and 5-shot exact-match accuracy across 12 target configurations. *GRAPE achieves the best performance across all settings.**

The method also generalizes beyond its targets, to unseen tasks. Table 5.10 demonstrates that GRAPE consistently outperforms DoGE on a range of held-out tasks—these include benchmarks in reasoning, mathematics, language understanding that were not used as training targets—across all twelve experimental configurations. Although differences between methods are somewhat reduced for transfer scenarios compared to in-target performance, GRAPE reliably maintains an advantage. This indicates that its allocation policy supports generalization and does not overfit to the specific mix of tasks used during training.

Method	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10	T11	T12	Avg	Worst
Uniform	32.5	32.5	32.5	32.5	32.5	32.5	32.5	32.5	32.5	32.5	32.5	32.5	32.5	32.5
DoGE	35.8	32.5	35.3	35.5	32.3	35.2	34.8	31.0	34.9	29.8	35.6	33.6	33.6	29.8
GRAPE	36.4	33.1	35.4	35.7	33.0	36.1	36.0	32.6	35.1	34.5	36.9	40.1	34.9	30.7

Table 5.10: **125M, 0.66B tokens (SlimPajama), T1–T12 settings.** 5-shot exact-match accuracy on unseen tasks (not used during training) across 12 target configurations. *GRAPE consistently outperforms DoGE on held-out tasks, demonstrating strong generalization capabilities.**

GRAPE adapts curriculum to mixture composition. The training schedule shows distinct allocation patterns across different task types. Math receives an early lift and then holds a substantive share; when surrounded by many other reasoning targets, its allocation rises well above $1/N$, suggesting the schedule compensates for scarcity to keep that skill active. Code behaves differently: from the start, the code target is pared back to (near) zero and GitHub is sampled sparingly—this slightly lowers code perplexity relative to DoGE but yields a more even profile as language/commonsense tasks improve and the overall mix strengthens. When HellaSwag is present, attention settles there for longer stretches, consistent with a broad reading/commonsense signal that supports gains across the suite. Taken together, the curriculum preserves math when it risks being diluted, trims code when it offers limited short-horizon benefit, and leans on widely reusable signals to keep progress steady. Across all mixture configurations, GRAPE consistently samples the book domain more frequently than DoGE, likely because its diverse linguistic content provides a strong foundation for reasoning tasks across multiple domains. These adaptive behaviors are visualized in the target-task weight evolution plots (Figures A.2–A.24 in Appendix A).

5.10 Evaluating Pretrained Initialization Quality via Supervised Fine-Tuning

We evaluate how well each method serves as a pretrained initialization for supervised fine-tuning (SFT). We fine-tune models trained with GRAPE, DoGE, and Uniform on six target tasks using identical supervised data

*Bold indicates best performance; underline indicates second-best.

and protocols, then compare both end-of-pretraining and post-SFT performance (see Section 4 for details).

GRAPE provides a superior pretrained initialization that enables more effective supervised fine-tuning across all evaluated tasks. Table 5.11 reveals three key patterns: (1) GRAPE achieves the best post-SFT performance even when its pretraining accuracy is not the highest (e.g., LogiQA), indicating that its parameters are better positioned for supervised optimization; (2) on limited-headroom tasks (PIQA, HellaSwag) where all models improve modestly, GRAPE preserves or widens its advantage, suggesting more efficient progress per labeled example; and (3) on knowledge-centric targets (SciQ, ARC), SFT lifts all baselines and compresses gaps, yet GRAPE maintains its lead, demonstrating that its benefits persist beyond pretraining artifacts. These results indicate that GRAPE reallocates pretraining toward domains whose gradients align with under-served targets, steering parameters into regions where supervised descent is more predictable and sample-efficient.

Task	Uniform		DoGE		GRAPE	
	Pretrained	+SFT	Pretrained	+SFT	Pretrained	+SFT
PIQA	58.98	61.26	59.52	62.24	61.10	63.93
SciQ	67.50	79.30	72.20	84.30	74.40	85.00
ARC-E	40.74	42.76	45.33	47.43	47.14	48.70
ARC-C	23.12	25.85	24.57	25.00	26.11	27.13
HellaSwag	27.02	27.20	27.77	27.96	28.56	29.02
LogiQA	27.96	29.19	25.35	29.34	27.65	30.72
Average	40.89	44.26	42.46	46.21	44.16	47.58

Table 5.11: **125M, 1.6B tokens (ClimbLab), 6-task setting.** Pretrained vs. post-SFT accuracy. *GRAPE provides a stronger initialization for supervised fine-tuning.**

5.11 Ablation: Progress Signals for Target Reweighting

We compare ROI (the default progress signal used in all main GRAPE experiments throughout this work) with two alternative progress signals for GRAPE’s target reweighting (Eq. (3.1)) on SlimPajama (0.66B token budget) across 12 multi-task configurations (T1–T12): a scale-dependent *Gap-of-Improvement* (GOI) and a smoothed relative improvement (ROI-EMA). The resulting weight update formulations for each variant are detailed in Appendix A.

Signals.

GOI (*Gap-of-Improvement*) measures realized absolute progress:

$$\text{GOI}_n^{(t)} = \ell_n(\theta_t) - \ell_n(\theta_{t+1}),$$

which is *scale-dependent* and can over-favor intrinsically low-loss tasks.

ROI-EMA (*Relative Rate-of-Improvement, EMA-normalized*) normalizes by an exponential moving average to reduce variance:

$$\text{ROI-ema}_n^{(t)} = \frac{\ell_n(\theta_t) - \ell_n(\theta_{t+1})}{\ell_{t,n}^{\text{ema}}}, \quad \ell_{t,n}^{\text{ema}} = \beta \ell_{t-1,n}^{\text{ema}} + (1 - \beta) \ell_n(\theta_t), \quad \beta = 0.7,$$

trading reactivity for stability.

*Bold indicates best performance; underline indicates second-best.

ROI (*Normalized Rate-of-Improvement*) uses the current loss for normalization:

$$\text{ROI}_n^{(t)} = \frac{\ell_n(\theta_t) - \ell_n(\theta_{t+1})}{\ell_n(\theta_t)},$$

yielding a responsive, unitless quantity that compares heterogeneous tasks fairly.

Scale-invariant progress signals achieve superior end-of-pretraining perplexity across diverse multi-task configurations. ROI attains the strongest average and worst-case test log-perplexity and posts the most per-configuration wins. GOI is occasionally competitive on later task configurations (T10-T12) that include medical reasoning tasks, where it achieves dramatic improvements on high-loss tasks like MedQA (reducing perplexity from 2.6 to 2.4), driving down the average perplexity through its bias toward tasks with naturally higher losses that show large absolute improvements, while ROI-EMA performs best on configurations with high task heterogeneity where the smoothing reduces noise in progress estimates. These outcomes reflect how signals couple to multiplicative reweighting: scale normalization is critical (ROI and ROI-EMA); excessive smoothing (ROI-EMA) can defer necessary reallocations; absolute drops (GOI) bias toward low-scale tasks unless counter-balanced.

Method	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10	T11	T12	Avg (↓)	Worst (↓)	# wins
Uniform	4.30	4.01	4.11	3.75	3.85	3.62	3.20	3.90	4.33	4.62	4.81	4.53	4.09	4.54	0
DOGE	4.18	4.94	4.00	3.66	3.70	3.63	3.12	3.83	4.13	4.44	4.57	4.33	4.04	4.43	0
GRAPE	3.53	3.61	3.70	<u>3.39</u>	3.36	3.03	2.93	3.45	3.85	3.94	3.93	<u>3.70</u>	3.54	3.95	8
GRAPE-EMA	3.68	3.69	3.73	3.37	<u>3.37</u>	3.09	2.96	3.53	<u>3.88</u>	<u>3.91</u>	3.90	3.73	3.57	4.00	1
GRAPE-GAP	<u>3.58</u>	<u>3.68</u>	<u>3.68</u>	3.42	3.36	<u>3.05</u>	3.11	<u>3.51</u>	<u>3.88</u>	3.86	3.85	3.59	<u>3.55</u>	<u>3.96</u>	<u>3</u>

Table 5.12: **125M, 0.66B tokens (SlimPajama), T1–T12 settings.** Average test log-perplexity across 12 task combinations with summary statistics. *ROI attains the most wins across diverse multi-task configurations and delivers best average and worst-case perplexity.**

Different progress signals create distinct trade-offs between target task specialization and unseen task generalization. GAP uses absolute drops and is therefore not scale-invariant: when tasks differ in loss magnitude due to label entropy, class imbalance, tokenization length, calibration, or noise, equal representational gains can yield unequal absolute decreases. GAP thus shifts probability toward numerically larger-loss regions, which can be beneficial when larger losses reflect genuine underfit (leading to better transfer and a higher floor), but detrimental when large losses are driven by noise or metric scale, in which case capacity is misallocated and in-domain accuracy can trail. ROI divides by the current loss, rendering updates scale-invariant and producing comparable progress across tasks; this accounts for its balanced seen/unseen accuracy and its superior average and worst-case perplexity. ROI-EMA further stabilizes this normalization by smoothing the denominator, which reduces ranking oscillations and improves target (seen) accuracy, at the cost of lag during regime shifts. Consequently, when high observed losses correspond to learnable, underfit structure, GAP is advantageous; when losses reflect scale artifacts or noise, ROI or ROI-EMA provide safer allocations. This aligns with our results as seen in Table 5.13: GAP attains more wins on unseen accuracy and strong floors, ROI-EMA attains more wins on seen accuracy, and ROI remains a robust default by combining scale-invariant progress with the best perplexity.

*Bold indicates best performance; underline indicates second-best.

Target (Seen) Accuracy (%)														
Method	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10	T11	T12	Avg	# Wins
DoGE	18.2	28.6	13.1	<u>14.6</u>	28.2	14.0	10.2	28.1	37.7	31.7	25.0	23.8	22.8	0
GRAPE	<u>18.9</u>	<u>29.1</u>	<u>13.4</u>	14.4	28.9	15.2	<u>10.4</u>	28.9	38.3	33.2	<u>25.6</u>	24.4	<u>23.7</u>	3
GRAPE-EMA	19.5	29.3	13.5	14.9	30.3	<u>15.5</u>	10.5	<u>28.7</u>	<u>37.7</u>	<u>33.3</u>	25.7	<u>24.1</u>	24.0	8
GRAPE-GAP	<u>18.9</u>	29.3	13.1	13.7	<u>28.9</u>	16.0	<u>10.4</u>	28.5	37.3	33.8	25.7	<u>24.1</u>	<u>23.7</u>	4
Unseen Accuracy (%)														
DoGE	<u>35.8</u>	32.5	35.3	<u>35.5</u>	32.3	35.2	34.8	31.0	34.9	29.8	35.6	33.6	33.8	0
GRAPE	36.4	33.1	<u>35.4</u>	35.7	<u>33.0</u>	<u>36.1</u>	36.0	32.6	<u>35.1</u>	<u>34.5</u>	36.9	<u>40.1</u>	<u>35.4</u>	6
GRAPE-EMA	35.5	<u>32.9</u>	35.3	35.1	32.8	35.8	35.7	31.9	34.9	34.3	<u>36.8</u>	39.8	<u>35.4</u>	0
GRAPE-GAP	36.4	32.2	36.1	35.3	33.1	36.3	<u>35.9</u>	<u>32.1</u>	36.2	35.5	36.9	40.2	35.6	9

Table 5.13: **125M, 0.66B tokens (SlimPajama), T1–T12 settings.** Target vs. unseen accuracy (%). *Progress signal choice induces different generalization profiles.**

We note that none of these end-of-pretraining summaries—perplexity, in-domain accuracy, or unseen accuracy—is a perfect predictor of post-SFT behavior. As decision heuristics: low perplexity often tracks broad linguistic competence and calibration; strong target accuracy indicates features already tuned to the source family (useful when downstream matches pretraining); strong unseen accuracy suggests representational headroom for shift and novel tasks. Additionally, for T1–T8, we include two generation-style tasks, for which accuracy is inherently very low without finetuning, further limiting the informativeness of raw pretraining accuracy on these tasks.

5.12 Ablation: Alignment Operators for Task Weight Updates

We investigate how different alignment operators in the adversarial task reweighting step affect downstream performance. The z -update shifts probability mass toward tasks least helped by the current parameter update through a multiplicative scheme:

$$z_n^{t+1} \propto z_n^t \exp\left(-\frac{\gamma_t}{\mu_z} \mathcal{A}(\nabla_{\theta} \log \ell_n(\theta_t), d_t)\right),$$

where ℓ_n is the loss on task T_n , d_t is the preconditioned update direction at step t , and $\sum_n z_n^{t+1} = 1$. We compare three alignment operators:

$$\mathcal{A}_{\cos} = \cos \angle(\nabla \log \ell_n, d_t) \in [-1, 1], \quad \mathcal{A}_{\|\cdot\|} = \|\nabla \log \ell_n\|, \quad \mathcal{A}_{\cdot} = \nabla \log \ell_n^{\top} d_t.$$

The dot product operator achieves superior performance across all benchmarks, attaining the best average log-perplexity and outperforming both cosine similarity and gradient norm only on every task. This advantage stems from the dot product’s ability to capture both directional alignment and gradient magnitude in a single statistic. While cosine similarity encodes directional agreement but ignores scale differences between tasks, and gradient norm captures magnitude but lacks directional information, the dot product preserves magnitude and directional alignment simultaneously. This comprehensive signal enables more effective task reweighting, with the largest performance margins appearing on SciQ and ARC-E. For fair comparison, we adjust μ_z per operator to ensure comparable scaling of $(\gamma_t/\mu_z)\mathcal{A}$ across all choices, yet the dot product remains optimal.

*Bold indicates best performance; underline indicates second-best.

Alignment in z -update	Task Log-Perplexity ↓						Avg.
	ARC-C	ARC-E	LogiQA	PIQA	SciQ	HellaSwag	
Cosine similarity	3.75	3.85	3.60	4.36	3.25	4.13	3.82
Gradient norm only	3.78	3.81	3.64	4.52	3.23	4.17	3.86
Dot product (ours)	3.60	3.58	3.55	4.24	2.96	3.99	3.65

Table 5.14: **125M, 573M tokens (ClimbLab), 6-task setting.** Task log-perplexity under different alignment operators. *Dot product captures sign, magnitude, and direction simultaneously.**

5.13 Ablation: Predictive vs. Realized Progress Signals

We compare two progress signals for GRAPE’s inner reweighting over targets (z): (i) *direct RoI*, which uses realized step-to-step loss differences $\ell_n(\theta_t) - \ell_n(\theta_{t+1})$ to update z ; and (ii) *directional alignment*, which uses the directional derivative of log-loss along the actual optimizer step, $a_n^t = \langle \nabla_{\theta} \log \ell_n(\theta_t), d_t \rangle$ (first-order link already established in Eq. (3.1)). This comparison tests whether a look-ahead, direction-aware signal is preferable to realized deltas when embedded in GRAPE’s min-max updates.

As seen in Table 5.15, directional alignment consistently outperforms direct RoI across all tasks, achieving substantial improvements in average accuracy on 5-shot accuracy across six benchmarks. The largest gains occur on ARC-E and PIQA, followed by ARC-C, SciQ, and HellaSwag, with minimal improvement on LogiQA. Notably, the direct-RoI variant performs worse than DoGE, which keeps target weights uniform and adapts only domain weights.

Directional alignment succeeds because it provides stable, predictive signals that enable effective task reweighting, while direct RoI fails due to noisy realized loss differences that lead to unstable updates. In our setting, the effective parameter change per step is small due to learning-rate scheduling, adaptive pre-conditioning, momentum/EMA, weight decay, and gradient clipping. Consequently, realized loss differences $\ell_n(\theta_t) - \ell_n(\theta_{t+1})$ are typically very small relative to their sampling variability, leading to erratic task rankings when fed into the adversarial z -update. This instability can move probability mass away from true bottlenecks, explaining why direct RoI underperforms even the simpler DoGE approach. Directional alignment, by contrast, evaluates for each task whether the current preconditioned step d_t is predicted to decrease or increase its loss (sign) and by how much (magnitude). This signed, magnitude-aware quantity provides stable progress estimates that enable focused yet stable reallocations, down-weighting tasks already helped by the current step while up-weighting those that would otherwise degrade, ultimately producing more effective domain shifts.

Method	ARC-C	ARC-E	LogiQA	PIQA	SciQ	HellaSwag	Avg.
Uniform	23.12	40.74	27.96	58.98	67.50	27.02	40.89
DoGE (uniform z)	<u>24.57</u>	<u>45.33</u>	25.35	<u>59.52</u>	72.20	<u>27.77</u>	<u>42.45</u>
GRAPE + <i>direct RoI</i>	23.29	42.76	27.50	56.86	<u>72.80</u>	27.16	41.73
GRAPE (alignment)	26.11	47.14	<u>27.65</u>	61.10	74.40	28.56	44.16

Table 5.15: **125M, 1.6B tokens (ClimbLab), 6-task setting.** Task 5-shot accuracy under different progress signals. *Directional alignment outperforms realized loss differences.**

5.14 Ablation: Impact of Update Frequency on Adaptation

We systematically vary the joint update cadence ΔT —the number of optimizer steps between simultaneous updates of target weights (z) and domain weights (α)—across $\{10, 50, 100, 200\}$ while holding data and training

*Bold indicates best performance; underline indicates second-best.

budget constant. The 5-shot accuracy results on six benchmarks and cross-task mean are shown in Table 5.16.

Mid-range update frequency ($\Delta T = 100$) achieves optimal performance, delivering the highest cross-task average and outperforming both very frequent updates ($\Delta T = 10$, $\Delta T = 50$) and sparse updates ($\Delta T = 200$). While frequent updates show localized spikes on SciQ and sparse updates peak on LogiQA, these task-specific gains are offset by consistent performance drops on ARC-C, ARC-E, and PIQA. Across all cadences, GRAPE consistently outperforms DoGE, with the largest margin at the optimal $\Delta T = 100$.

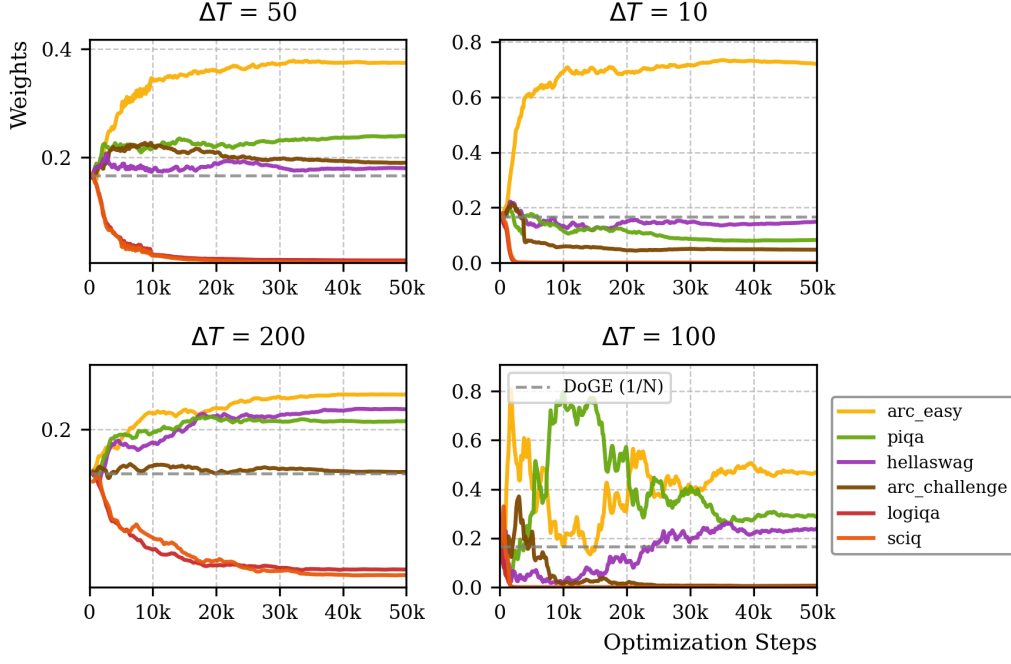
ΔT	Method	ARC-C	ARC-E	LogiQA	PIQA	SciQ	HellaSwag	Avg.
10	GRAPE	<u>25.34</u>	<u>46.84</u>	25.04	58.92	76.30	27.66	<u>43.35</u>
10	DoGE	24.23	44.82	26.27	58.27	74.20	<u>27.82</u>	42.60
50	GRAPE	25.09	45.83	25.50	60.12	73.30	27.86	42.95
50	DoGE	24.40	43.64	26.88	58.98	70.10	27.53	41.92
100	GRAPE	26.11	47.14	27.65	61.10	<u>74.40</u>	28.56	44.16
100	DoGE	24.57	45.33	25.35	59.52	72.20	27.77	42.46
200	GRAPE	23.81	44.40	29.03	59.58	72.70	27.77	42.88
200	DoGE	24.32	43.31	<u>28.42</u>	59.14	70.10	27.07	42.06

Table 5.16: **125M, 1.6B tokens (ClimbLab), 6-task setting.** Task 5-shot accuracy under different update cadences ΔT . *Mid-range cadence ($\Delta T=100$) balances adaptation speed with stability.**

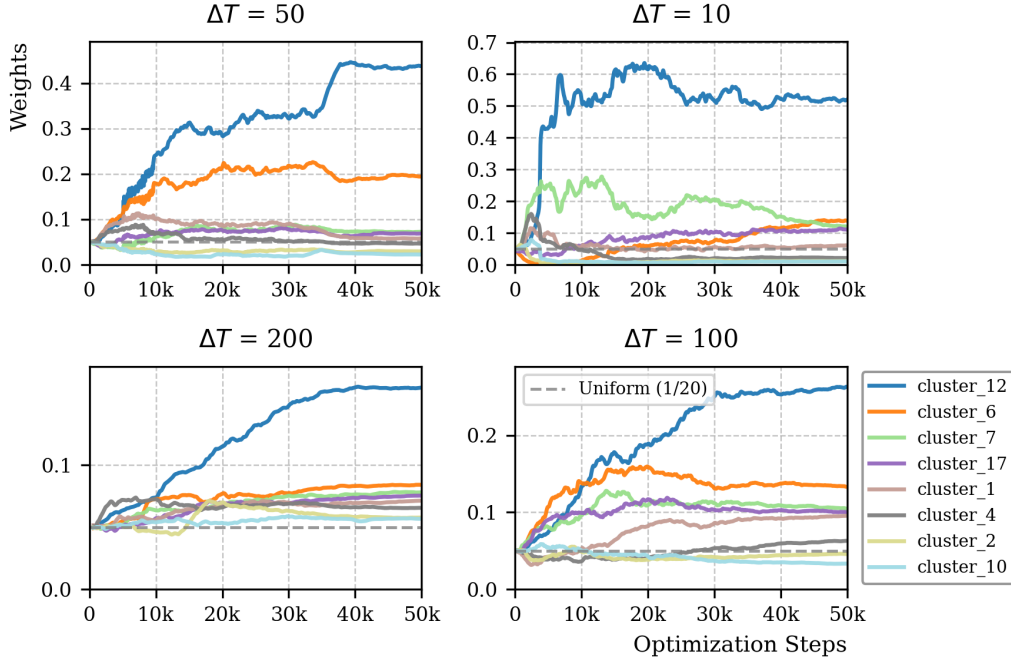
Update frequency critically affects curriculum formation, with the structured learning progression observed at $\Delta T = 100$ disappearing under other cadences. As seen in Figure 5.8, the target weight plots (top) show that at small update frequencies, the system concentrates heavily on a single task (ARC-E) while other tasks remain around uniform weights ($1/N$) or less, with higher update frequencies causing task weights to move increasingly toward zero as the distribution becomes more concentrated. At large update frequencies, $\Delta T = 200$, most tasks remain close to uniform weights, also eliminating the curriculum. The tasks that received zero weights in the $\Delta T = 100$ setting still get downweighted towards zero at other frequencies though that takes longer. In both cases, the curriculum disappears—either through extreme specialization or failure to develop meaningful task prioritization. The domain weight plots (bottom) show corresponding patterns in source allocation. Frequent updates at $\Delta T = 10$ cause high concentration on one-two domains (C12 dominates), while sparse updates at $\Delta T = 200$ result in domain weights closer to uniform that fail to identify beneficial sources. The mid-range cadence $\Delta T = 100$ enables the structured domain allocation patterns that support effective curriculum learning.

The observed performance pattern reflects a fundamental trade-off between adaptation speed and stability that is inherent in dynamic reweighting systems. The optimal cadence balances two competing failure modes: updating too frequently causes the system to overreact to short-term progress signals, leading to rapid convergence on a single task and domain that eliminates the curriculum. Conversely, updating too infrequently results in weights that remain close to uniform, also eliminating the curriculum as the system fails to learn meaningful task prioritization. Notably, GRAPE demonstrates greater sensitivity to update frequency compared to DoGE, with its performance advantages appearing less pronounced as updates become less frequent, already observable at $\Delta T = 200$. The effective adaptation lag—the number of steps required for training to meaningfully reflect a new sampling mixture (z_t, α_t)—depends on learning rate, momentum/EMA smoothing, preconditioning strength, and model scale. A mid-range cadence that approximately matches this lag produces the most reliable progress estimates and optimal average accuracy.

*Bold indicates best performance; underline indicates second-best.



(a) Target weight (z) evolution across ΔT values



(b) Domain weight (α) evolution across ΔT values

Figure 5.8: **125M, 1.6B tokens (ClimbLab), 6-task setting.** (a) Target weight evolution, (b) domain weight evolution across different ΔT values. *Optimal cadence maintains curriculum diversity while avoiding extreme bias and uniform distribution.*

Design implications of joint updates. The simultaneous updates of z and α tightly couple capability emphasis (what to optimize) with source composition (where to sample), which is advantageous when both evolve on similar timescales. When timescales diverge, decoupled cadences can be beneficial: faster target updates with slower domain updates to address capability bottlenecks on a stable data diet, or faster domain updates with slower target updates to rapidly pivot sources during data scarcity—though this risks chasing domain-specific artifacts if targets lag behind.

Computational considerations. While GRAPE adds a theoretical FLOP overhead of $\sim 28\%$ in this setting

(ClimbLab, 6 targets; see Table 4.8), practical runtime often hinges on memory pressure: additional per-task statistics and buffers can force smaller microbatches and more accumulation steps, reduce tensor-core occupancy, and increase memory traffic—effects that may dominate the nominal FLOP cost. Table 5.17 shows actual runtime measurements across different update frequencies, revealing that GRAPE’s computational overhead is modest in practice, with runtime differences primarily driven by update frequency rather than method choice in our experiments.

ΔT	Method	Runtime (2 x H100)
50	GRAPE	23.9h
50	DoGE	22.3h
100	GRAPE	15.8h
100	DoGE	15.0h
200	GRAPE	10.3h
200	DoGE	10.8h

Table 5.17: **125M, 1.6B tokens (ClimbLab), 6-task setting.** Runtime measurements across different update frequencies. *In practice, runtime differences are primarily driven by update frequency rather than method choice.*

5.15 Impact of Domain Granularity on Performance

We compare GRAPE under topic-based partitions (ClimbLab) versus source-based partitions (SlimPajama) at equal training tokens to control for total data exposure. Both use the same learning rate schedule but differ in training length (ClimbLab trains longer), placing them at different points in the optimization trajectory. This comparison provides a lower bound on the granularity advantage.

Finer-grained training domains yield larger improvements over uniform target weights. Table 5.18 shows that topic-based partitions (ClimbLab) outperform source-based partitions (SlimPajama) under GRAPE. The advantage concentrates on knowledge-intensive reasoning tasks. ClimbLab offers 20 semantically coherent clusters, while SlimPajama provides 7 broader source domains that conflate diverse topics and writing styles. As seen in Table 5.19, for knowledge-intensive reasoning tasks, the finer granularity enables GRAPE to identify and weight science-specific clusters that precisely match the knowledge required, whereas coarser domains mix relevant content with unrelated material. For continuation tasks, both settings perform similarly, as these benefit from diverse training regardless of granularity. For tasks like LogiQA where neither partition provides good alignment, finer granularity substantially reduces harm by allowing more selective downweighting of misleading subdomains while preserving useful logical content from educational clusters. Continuing training beyond the token-matched comparison point further strengthens the granularity advantage, as demonstrated by the final ClimbLab results achieving best average accuracy across all targets.

Finer domains induce a more adaptive curriculum, consistent with the observed performance gains. On SlimPajama, the schedule first upweights an easier target (SciQ) and then increasingly concentrates mass on a harder capability (ARC-C). The observed performance on ARC-C indicates it remains difficult to improve under coarse, source-based domains and the available budget, so it stays persistently upweighted as a bottleneck in the schedule. The early SciQ emphasis also aligns with the ClimbLab observation: in ClimbLab, SciQ receives little weight because the topic mixture already contains ample science-QA content, so no additional boost is needed. In contrast, on ClimbLab, ARC-C is again prioritized at a similar phase, but GRAPE can draw on STEM-specific clusters to make progress and then reallocate, producing a more agile curriculum. Other targets stay near-uniform under SlimPajama, reflecting a less agile schedule in which coarse domains both obscure progress signals and hinder finding the right data to advance capabilities.

Dataset	Domains	Accuracy (%)		
		DoGE	GRAPE	Gain (pts)
ClimbLab	topic-based	40.47	41.62	+1.15
SlimPajama	source-based	37.38	37.65	+0.27

Table 5.18: **125M, 0.64B tokens, 6-task setting.** Overall accuracy under different domain granularities. *Finer-grained domains enable larger improvements.*

Task	ClimbLab	SlimPajama	Type
ARC-C	+13.7	+0.4	Science QA (challenge; multi-step)
ARC-E	+4.9	+4.4	Science QA (grade-school)
SciQ	+1.9	+3.4	Science QA (factoid)
PIQA	+2.2	+2.5	Physical commonsense
HellaSwag	+0.3	+2.5	Commonsense continuation
LogiQA	-2.8	-12.8	Logical reasoning (reading comp.)

Table 5.19: **125M, 0.64B tokens, 6-task setting.** Task-level accuracy percentage improvements over DoGE. *Topic-based domains benefit knowledge-intensive tasks most.*

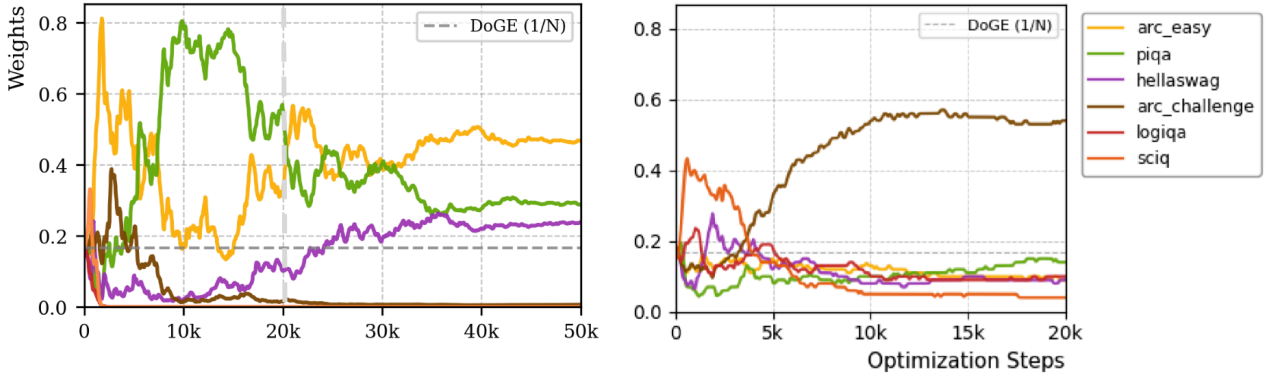


Figure 5.9: **125M, 0.64B tokens, 6-task setting.** Target weight trajectories under topic-based (left) vs. source-based (right) domains. *Finer domains enable more adaptive curriculum evolution.*

5.16 Validation on SlimPajama Across Multiple Task Suites

We evaluated GRAPE on SlimPajama with $K=7$ source domains (C4, CC, Wikipedia, arXiv, GitHub, StackExchange, Book) across 6-task and 8-task suites. Additional plots and ablations on SlimPajama are presented in Appendix A.

GRAPE achieves the lowest average log-perplexity and highest average 5-shot accuracy across both suites, outperforming all baseline methods (tables 5.20, 5.21, 5.22, 5.23). In the 6-task suite (ARC-E/ARC-C, HellaSwag, SciQ, PIQA, LogiQA), GRAPE yields the best average perplexity and accuracy at both 125M and 0.7B scales. In the 8-task suite (adding MathQA, MedQA), GRAPE leads on average metrics with particularly strong gains on MedQA, indicating effective adaptation to specialized clinical reasoning tasks (Table 5.22). GRAPE also achieves the best *worst-case* log-perplexity in both settings, demonstrating that it prevents targets from being starved under fixed token budgets (Fig. 5.10).

Baseline methods exhibit distinct allocation patterns that explain their performance gaps. CRISP over-allocates to code and Q&A (GitHub, StackExchange), which yields strong performance on certain reasoning

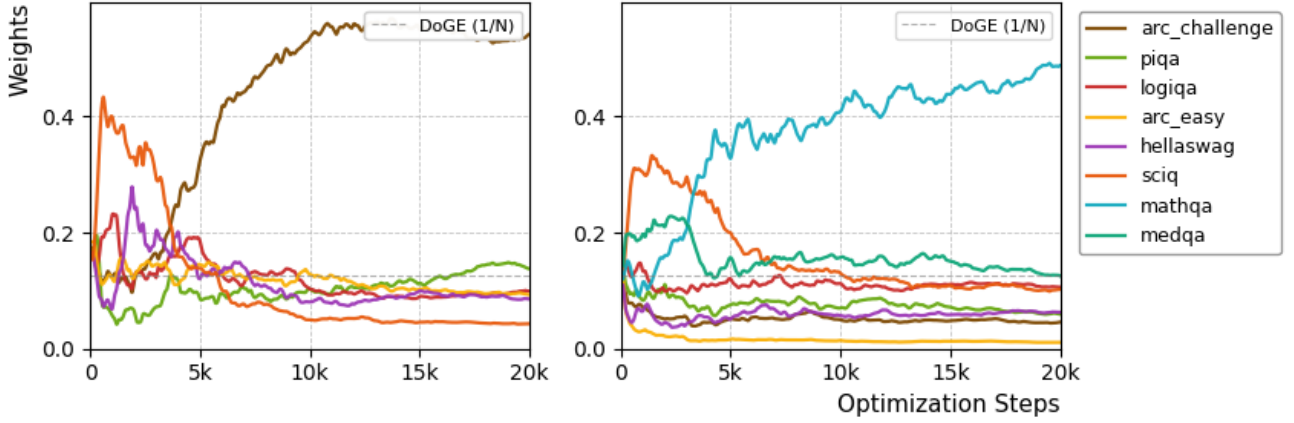


Figure 5.10: **125M, 0.66B tokens (SlimPajama), 6-task (left) and 8-task (right) settings.** Target weight trajectories vs. training tokens. *GRAPE reallocates toward still-improving hard targets while maintaining coverage.*

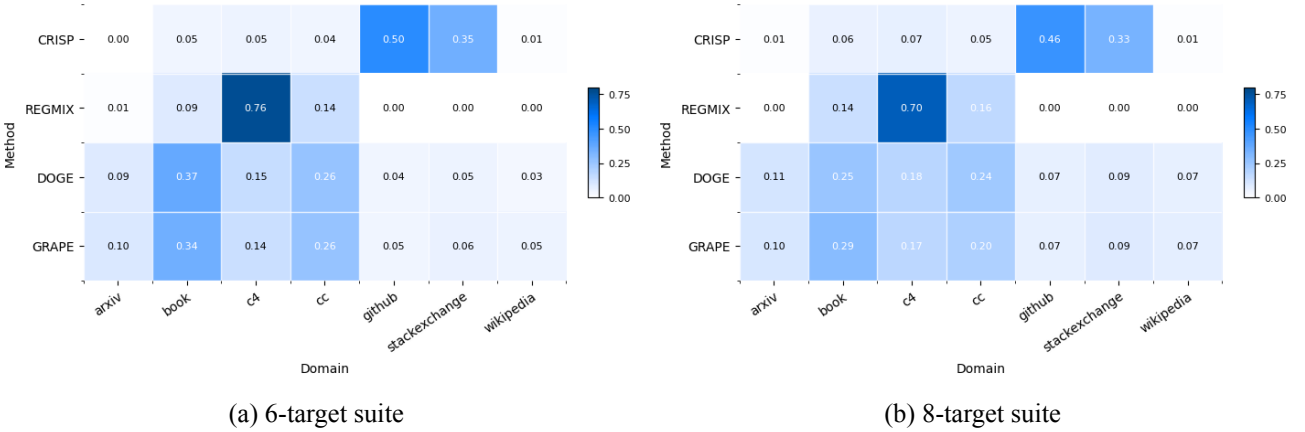


Figure 5.11: **125M, 0.66B tokens (SlimPajama), 6-task (a) and 8-task (b) settings.** Final domain weight heatmaps. *Different target sets lead to distinct allocation patterns.*

tasks like LogiQA and ARC-C but weakens averages on tasks requiring diverse narrative/expository content like ARC-E, SciQ, and MedQA (see heatmaps in Fig. 5.11a, 5.11b). RegMix collapses onto C4 with minor spillover to CC and book, achieving the best perplexity on web-like targets like LogiQA and SciQ but failing on specialized tasks requiring Q&A/code/math/biomedical content. DoGE spreads mass broadly over book/CC with modest arXiv/Wikipedia and minimal code/Q&A, producing solid but suboptimal averages.

The target weight trajectories (Fig. 5.10) evolve through two phases: an initial exploration phase, then a convergent phase. In the initial exploration phase (roughly the first 0.35B tokens), GRAPE allocates small spikes across multiple targets, with SciQ receiving sustained emphasis throughout this phase. Thereafter, weights converge on the hardest still-improving target while maintaining nonzero floors on all tasks. In the 6-task setting, ARC-Challenge dominates; in the 8-task setting, MathQA dominates.

Method	ARC-C	ARC-E	HellaSwag	LogiQA	PIQA	SciQ	Avg.	Worst
Uniform	4.83	4.92	4.30	4.77	4.08	4.56	4.58	4.92
CRISP	4.94	5.09	4.41	4.95	4.10	4.60	4.68	5.09
RegMix	<u>4.29</u>	<u>4.36</u>	4.05	3.68	4.16	3.81	<u>4.06</u>	<u>4.36</u>
DoGE	4.35	4.51	<u>3.83</u>	4.34	<u>3.66</u>	4.09	4.13	4.51
DoGE+PCGrad	4.38	4.54	3.88	4.35	3.71	4.18	4.17	4.54
GRAPE	4.05	4.08	3.23	<u>4.19</u>	3.54	<u>4.04</u>	3.86	4.19

Table 5.20: **125M, 0.66B tokens (SlimPajama), 6-task setting.** Task log-perplexity. *GRAPE achieves best average and worst-case across all tasks.**

Method	ARC-C	ARC-E	HellaSwag	LogiQA	PIQA	SciQ	Avg.
125M							
Uniform	21.50	32.45	26.44	26.42	55.50	52.40	35.78
CRISP	21.08	30.89	26.58	<u>27.80</u>	54.62	53.80	35.80
RegMix	20.56	32.79	<u>26.83</u>	24.42	58.92	52.50	36.00
DoGE	<u>21.67</u>	<u>32.66</u>	26.05	30.11	57.18	<u>56.60</u>	37.38
DoGE+PCGrad	21.33	32.20	25.96	27.19	<u>58.60</u>	53.50	36.46
GRAPE	21.76	34.09	26.69	26.27	<u>58.60</u>	58.50	37.65
0.7B							
Uniform	21.84	37.84	29.04	62.13	62.13	<u>69.60</u>	41.63
DoGE	<u>22.87</u>	<u>40.61</u>	31.16	<u>63.82</u>	<u>63.82</u>	69.80	42.19
GRAPE	23.46	42.17	31.90	64.42	64.42	68.70	42.92

Table 5.21: **125M and 0.7B, 0.66B tokens (SlimPajama), 6-task setting.** Task 5-shot accuracy at two scales. *GRAPE achieves best average at both scales.**

Method	ARC-C	ARC-E	HellaSwag	LogiQA	PIQA	SciQ	MathQA	MedQA	Avg.	Worst
Uniform	4.83	4.92	<u>4.56</u>	4.08	4.77	4.30	4.88	4.60	4.62	4.92
CRISP	4.89	4.95	4.55	<u>4.04</u>	4.79	4.36	4.79	4.69	4.63	4.95
RegMix	<u>4.30</u>	<u>4.40</u>	4.06	3.71	4.15	3.81	<u>4.55</u>	3.93	<u>4.11</u>	<u>4.55</u>
DoGE+PCGrad	4.35	4.47	4.17	3.71	4.35	3.84	4.65	4.07	4.20	4.65
DoGE	4.63	4.72	4.39	3.90	4.60	4.13	4.82	<u>4.36</u>	4.44	4.82
GRAPE	4.26	4.37	4.15	3.58	4.35	3.76	4.52	2.58	3.95	4.52

Table 5.22: **125M, 0.66B tokens (SlimPajama), 8-task setting.** Task log-perplexity with average and worst-case. *GRAPE achieves best average and worst-case perplexity.**

Method	ARC-C	ARC-E	HellaSwag	LogiQA	PIQA	SciQ	MathQA	MedQA	Avg.
Uniform	21.50	32.45	26.44	26.42	55.50	52.40	20.97	24.59	32.53
CRISP	<u>22.01</u>	30.56	26.24	<u>27.19</u>	55.88	51.10	19.97	21.21	29.27
RegMix	22.00	<u>32.40</u>	26.80	25.50	59.70	<u>56.00</u>	20.70	21.40	<u>33.10</u>
DoGE	21.59	31.57	26.32	25.96	56.31	48.60	19.50	27.34	32.14
DoGE+PCGrad	21.16	32.28	26.17	26.11	<u>57.62</u>	52.20	20.60	21.52	32.21
GRAPE	21.76	33.08	<u>26.69</u>	27.34	57.70	57.94	<u>20.74</u>	<u>27.18</u>	33.99

Table 5.23: **125M, 0.66B tokens (SlimPajama), 8-task setting.** Task 5-shot accuracy. *GRAPE achieves best average with strong specialized-task performance.**

*Bold indicates best performance; underline indicates second-best.

5.17 Gradient Aggregation Strategies for Multi-Task Pretraining

We apply PCGrad to the target-task gradients within DoGE (after aggregating domains via adaptive reweighting) and find that it does not improve—and often reduces—average performance relative to simply averaging the target gradients. We further evaluate MGDA (Multiple Gradient Descent Algorithm) and CAGrad (Conflict-Averse Gradient Descent) as described in Section 4.7. Table 5.24 reports normalized 5-shot accuracy at 0.96B tokens (ClimbLab). On average, plain DoGE remains strongest. MGDA, which by construction selects a lower-norm, conflict-safe combination of target gradients, attains the highest scores on the two most challenging benchmarks (ARC-Challenge, LogiQA), while CAGrad and PCGrad don’t have any advantage over simple averaging.

Taken together, these results—across MGDA/CAGrad/PCGrad and across budgets—indicate that step-level conflict resolution over targets is insufficient in this setting. The surgery rules are memoryless at the target level: they recompute a geometric adjustment each minibatch from noisy small-batch target gradients, which tends to shrink effective step norms and can prune synergistic cross-target components, while adding nontrivial per-step overhead. Without persistence on the target side, these adjustments do not accumulate into a stable target schedule and therefore fail to outperform simple averaging; moreover, their compute profile makes them unattractive for LLM pretraining at scale even if larger budgets might expose some benefits.

Method	ARC-C	ARC-E	LogiQA	PIQA	SciQ	HellaSwag	Avg.
DoGE	<u>24.57</u>	45.33	25.35	59.52	72.20	27.77	42.79
DoGE+MGDA	24.74	38.76	28.26	57.78	63.20	26.89	39.94
DoGE+CAGrad	22.44	38.80	25.65	57.67	63.80	26.79	39.19
DoGE+PCGrad	23.29	<u>40.61</u>	<u>27.96</u>	<u>57.83</u>	<u>66.60</u>	<u>26.87</u>	<u>40.36</u>

Table 5.24: **125M, 0.96B tokens (ClimbLab), 6-task setting.** Task 5-shot accuracy under different gradient aggregation methods. *Simple averaging with domain reweighting remains most effective.**

*Bold indicates best performance; underline indicates second-best.

Chapter 6

Multilingual Modeling Experiments

This chapter presents results from the multilingual transfer learning scenario described in Section 4.5. We evaluate GRAPE’s ability to balance performance across typologically diverse target languages while leveraging cross-lingual transfer from high-resource sources.

6.1 Aggregate Performance: Data Efficiency and Convergence

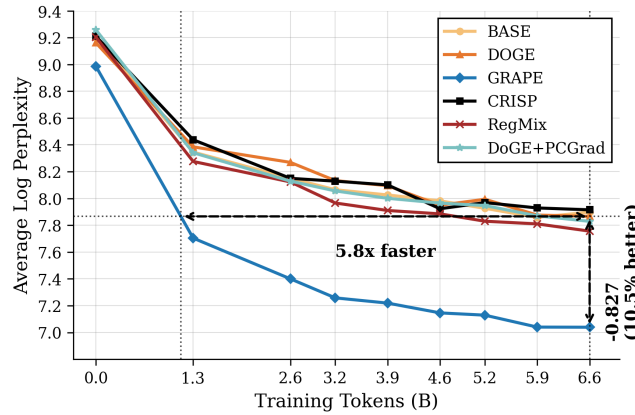


Figure 6.1: **125M, 1.6B tokens (Wiki40B), 8-task multilingual setting.** Average log-perplexity evolution during training across all target languages. *GRAPE achieves 5.8x faster convergence and 10.5% better final performance compared to all baselines.*

GRAPE delivers faster convergence and stronger final multilingual performance. At equal token budgets, GRAPE converges $5.8\times$ faster and achieves 10.5% lower final log-perplexity than baselines (Figure 6.1). These gains reflect an adaptive curriculum that prioritizes bottleneck languages early while preserving broad coverage, yielding consistent improvements across typologically diverse targets.

6.2 Task-Level Performance Results

We evaluate performance across two target configurations: a 4-task subset and the full 8-task suite. Results demonstrate GRAPE’s effectiveness in both settings, with particular attention to how curriculum dynamics adapt to different levels of target diversity.

Method	Catalan	Danish	Romanian	Ukrainian	Avg.
Uniform	6.61	8.15	8.16	7.75	7.67
RegMix	6.56	8.22	8.05	<u>7.44</u>	7.57
CRISP	6.46	8.12	8.01	7.89	7.62
DoGE-PCGrad	<u>6.36</u>	<u>8.05</u>	8.00	7.75	<u>7.54</u>
DoGE	6.43	8.10	<u>7.95</u>	7.69	<u>7.54</u>
GRAPE	6.27	7.92	4.31	7.33	6.46

Table 6.1: **125M, 1.6B tokens (Wiki40B), 4-task setting.** Final log-perplexity on 4-task setting targets only. *GRAPE achieves best average across all baselines.**

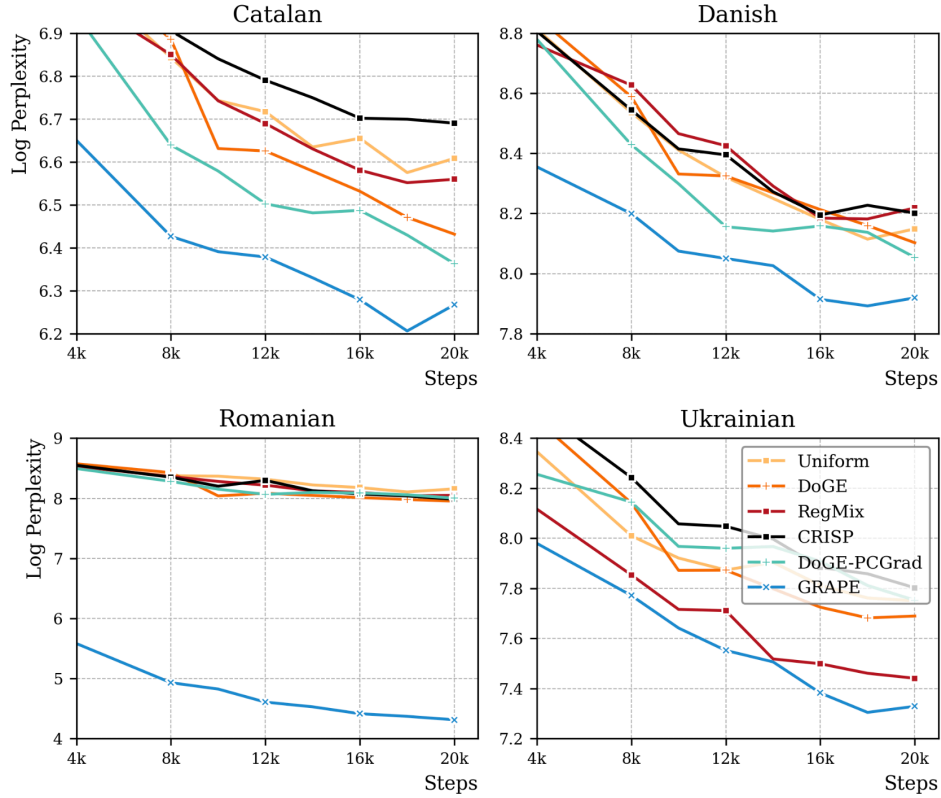


Figure 6.2: **125M, 1.6B tokens (Wiki40B), 4-task setting.** Log-perplexity evolution during training for 4-task setting targets. *GRAPE consistently achieves lowest perplexity across all languages, with particularly strong gains on Romanian.*

Method	Turkish	Danish	Catalan	Polish	Romanian	Portuguese	Dutch	Ukrainian	Avg.
Uniform	9.31	8.15	6.61	8.80	8.16	6.78	7.62	7.75	7.90
RegMix	9.14	8.15	6.47	<u>8.54</u>	<u>8.06</u>	<u>6.66</u>	7.73	<u>7.30</u>	<u>7.63</u>
CRISP	<u>9.02</u>	<u>8.08</u>	6.50	8.65	7.93	6.73	<u>7.39</u>	7.96	7.65
DoGE-PCGrad	9.11	8.11	6.48	8.57	<u>8.05</u>	6.67	7.59	8.05	7.70
DoGE	9.28	8.23	<u>6.45</u>	8.76	<u>8.05</u>	6.70	7.64	7.82	7.87
GRAPE	8.76	7.85	6.28	8.17	4.36	6.44	7.19	7.26	6.79

Table 6.2: **125M, 1.6B tokens (Wiki40B), 8-task setting.** Final log-perplexity on 8-task setting targets only. *GRAPE achieves best average and leads on all individual languages.**

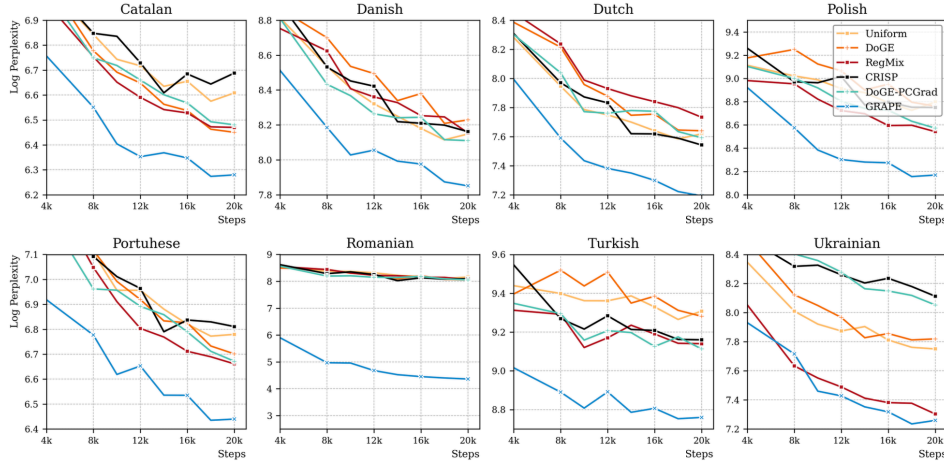


Figure 6.3: **125M, 1.6B tokens (Wiki40B), 8-task setting.** Log-perplexity evolution during training for 8-task setting targets. *GRAPE maintains consistent advantage across all eight languages, with dramatic improvements on hardest targets like Romanian and Turkish.*

GRAPE consistently achieves the best average performance while demonstrating particularly strong gains on the most challenging target languages. The method’s advantage becomes more pronounced in the larger 8-task setting, where it maintains leadership across all individual languages. Notably, GRAPE shows exceptional improvements on languages that typically pose the greatest learning challenges, suggesting effective identification and prioritization of bottleneck targets. Among the most dramatic improvements, Romanian and Turkish demonstrate the largest gains over uniform target weights (DoGE), indicating that GRAPE’s curriculum is particularly effective at addressing the learning bottlenecks in these linguistically challenging languages. The training evolution curves demonstrate GRAPE’s superior token efficiency, achieving lower perplexity levels with fewer training tokens across all target languages. This efficiency advantage is particularly pronounced for the hardest languages, where GRAPE reaches competitive performance levels much earlier in training compared to baseline methods.

Baseline methods exhibit distinct performance patterns and biases across language families. Uniform yields the worst average performance across both settings. DoGE improves over uniform sampling on most languages but leaves harder languages like Romanian and Turkish as persistent bottlenecks. Static mixture methods (CRISP, RegMix) show competitive performance on subsets of targets but induce trade-offs elsewhere. RegMix achieves competitive performance on Ukrainian, suggesting a bias towards Slavic languages, while CRISP shows relative improvements over DoGE on Romance languages (Catalan) and Germanic languages (Dutch), though never approaching GRAPE’s performance levels. DoGE-PCGrad reduces gradient interference and often edges other baselines on mid-tier targets, but without curriculum guidance cannot close performance gaps on the hardest languages. These patterns reveal that static approaches tend to over-concentrate on high-resource or linguistically similar training languages, while uniform methods fail to exploit cross-lingual transfer opportunities effectively.

6.3 Weight Evolution: Cross-Lingual Transfer and Linguistic Discovery

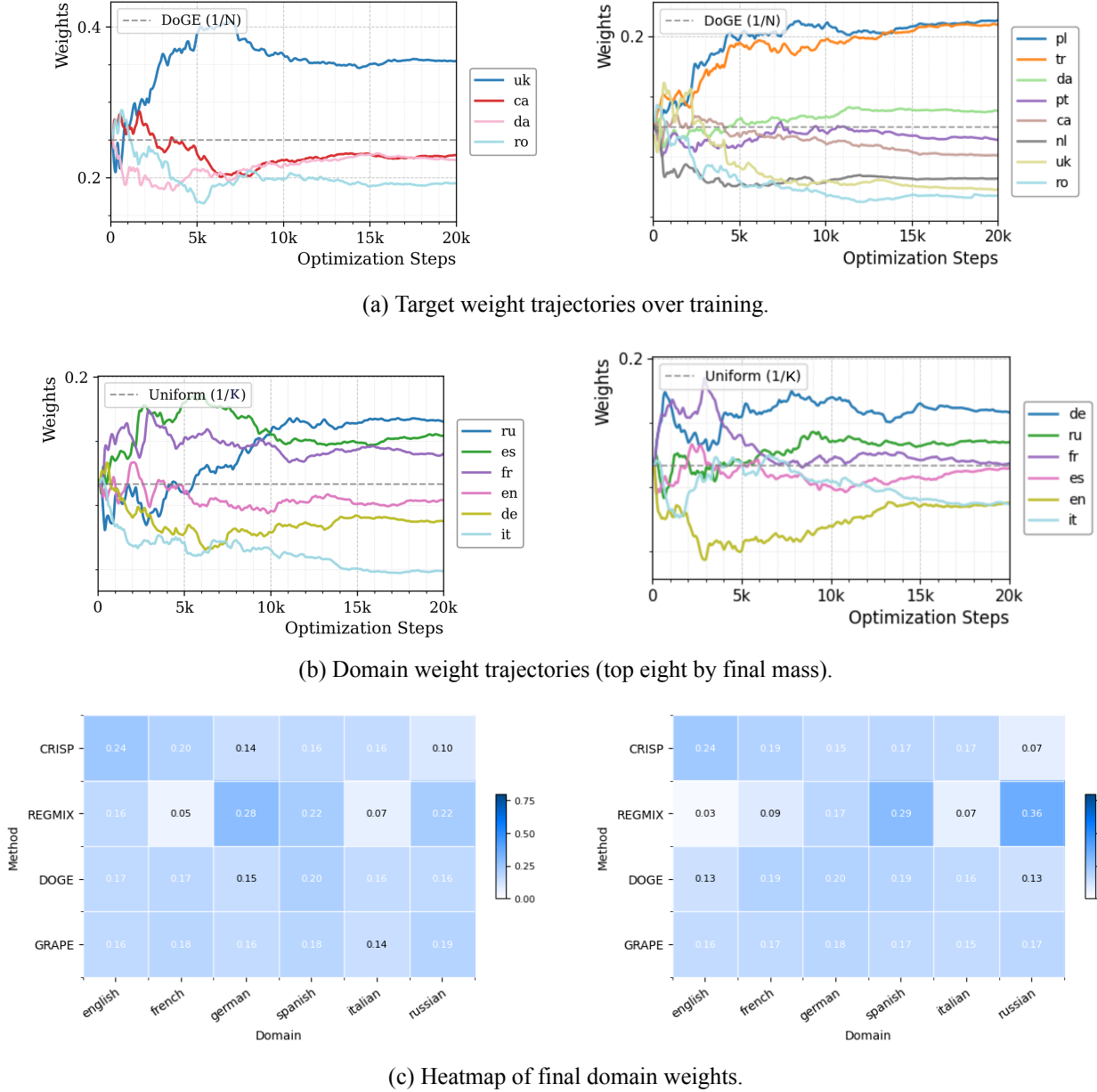


Figure 6.4: **125M, 1.6B tokens (Wiki40B), 4-task (left) and 8-task (right) settings.** (a) Target weight trajectories, (b) domain weight trajectories, (c) final domain heatmaps. *Weight evolution and transfer structure show GRAPE prioritizing laggards and their supporting sources across both task configurations.*

The weight evolution patterns in Figure 6.4 reveal sophisticated curriculum dynamics and cross-lingual relationship discovery. Target weight trajectories (panel a) show GRAPE developing distinct prioritization strategies: in the 4-task setting, Ukrainian receives sustained high weights while Romanian is initially downweighted before gradual recovery, suggesting Ukrainian serves as a transfer hub for Slavic languages. In the 8-task setting, Polish and Turkish emerge as the most heavily weighted targets, indicating their role as primary bottlenecks requiring intensive focus. Domain weight evolution (panel b) demonstrates systematic linguistic family exploitation: Russian maintains consistently high weights supporting Slavic targets, Spanish and French provide sustained support for Romance languages, while English and German are progressively downweighted as their transfer utility diminishes. The final allocation heatmaps (panel c) reveal GRAPE’s adaptive vs. static baseline patterns, showing how the method concentrates on linguistically supportive source languages.

GRAPE discovers and exploits linguistic family structures through systematic cross-lingual transfer.

This family-aware allocation produces quantifiable improvements across linguistic typologies (Table 6.3): Romance languages achieve the strongest gains, Slavic languages show consistent improvements, Germanic languages benefit moderately, and even Turkish, which lacks close Indo-European relatives, receives higher weight through transfer from structurally rich languages like German and French.

Language Family	Target Languages	Improvement vs. Uniform
Romance	Romanian, Portuguese, Catalan	+46.5% , +5.0%, +5.0%
Slavic	Ukrainian, Polish	+6.3%, <u>+7.2%</u>
Germanic	Danish, Dutch	+3.7%, +5.6%
Turkic	Turkish	+5.9%

Table 6.3: **125M, 1.6B tokens (Wiki40B), 8-task setting.** GRAPE discovers and exploits linguistic family relationships for improved transfer. *All language families show consistent gains, with Romance languages achieving the strongest improvements.**

6.4 Ablation: Progress Signal Variants for Multilingual Learning

We compare three variants of GRAPE progress signals against DoGE and Uniform baselines across both multilingual settings: GRAPE uses normalized rate-of-improvement, GRAPE-EMA applies exponential smoothing to the ROI denominator, and GRAPE-GAP employs absolute loss decrease as the curriculum signal.

Method	Catalan	Danish	Romanian	Ukrainian	Avg.
Uniform	6.61	8.15	8.16	7.75	7.67
DoGE	6.43	8.10	7.95	7.69	7.54
GRAPE	6.27	7.92	4.31	<u>7.33</u>	<u>6.46</u>
GRAPE-EMA	6.40	<u>8.00</u>	3.99	7.48	6.47
GRAPE-GAP	<u>6.55</u>	8.26	<u>4.02</u>	6.91	6.44

Table 6.4: **125M, 1.6B tokens (Wiki40B), 4-task setting.** GRAPE variants vs. DoGE and Uniform on 4-task setting. *All GRAPE variants substantially outperform domain-only and uniform baselines.**

Method	Turkish	Danish	Catalan	Polish	Romanian	Portuguese	Dutch	Ukrainian	Avg.
Uniform	9.31	8.15	6.61	8.80	8.16	6.78	7.62	7.75	7.90
DoGE	9.28	8.23	6.45	8.76	8.05	6.70	7.64	7.82	7.87
GRAPE	8.76	7.85	6.28	8.17	4.36	6.44	7.19	7.26	6.79
GRAPE-EMA	9.10	8.12	<u>6.36</u>	8.59	<u>4.01</u>	6.62	7.48	7.46	<u>6.97</u>
GRAPE-GAP	<u>8.86</u>	<u>7.88</u>	6.38	<u>8.27</u>	4.02	<u>6.65</u>	<u>7.33</u>	<u>7.40</u>	6.59

Table 6.5: **125M, 1.6B tokens (Wiki40B), 8-task setting.** GRAPE variants vs. DoGE and Uniform on 8-task setting. *GRAPE-GAP delivers 16% improvement over uniform baseline.**

Absolute loss decrease (GRAPE-GAP) consistently outperforms normalized rate-of-improvement signals, achieving the best average performance in both settings. This superiority stems from the gap-based signal’s natural prioritization of targets with numerically larger losses, which in multilingual settings often correspond to genuinely harder languages rather than scale artifacts. The normalized GRAPE signal, while effective, can be misled by loss scale differences between languages, whereas GRAPE-GAP directly captures the absolute learning potential. GRAPE-EMA provides intermediate performance through exponential smoothing of the rate-of-improvement denominator, reducing variance when high observed losses reflect noise rather than learnable structure, though the practical difference remains minimal in this setting.

*Bold indicates best performance; underline indicates second-best.

Chapter 7

Discussion and Future Directions

Skill-based curriculum and quality of GRAPE-pretrained model. Our experiments show that GRAPE uncovers a skill-based curriculum, organizing tasks to follow a natural progression where each new challenge builds on skills learned before. This sequencing helps the model learn more effectively fundamental skills that transfer to tasks not seen during pretraining. Beyond end-of-pretraining metrics, the trajectories themselves matter: improvements arrive earlier and distribute more evenly across tasks, and models initialized under these trajectories subsequently fine-tune more effectively, suggesting reduced interference and more favorable geometry for supervision. Notably, in our experiments, models initialized under GRAPE fine-tune more effectively than strong static mixtures, even for tasks where end-of-pretraining improvements are modest. This pattern is consistent with reduced interference among tasks and with GRAPE placing parameters in regions of the landscape that are easier to specialize under supervision.

Scaling to larger models. The success of this approach depends on using progress metrics and domain groupings that reflect the real structure in the data. As the backbone scales from 125M to 0.7B parameters, GRAPE continues to outperform baselines, yet its margin over a uniform mixture narrows. Further training and better adaptation of the hyperparameters (temperature, regularization, progress smoothing) are expected to improve the performance of GRAPE at larger scale and we leave this for future work. Interestingly, the nature of the curriculum appears to depend on both model size and how training domains are defined. Understanding how the “ideal” curriculum varies with model size remains important for developing effective adaptive pretraining strategies and the way GRAPE schedules tasks can also provide useful insights.

Progress metrics. Different progress metrics embody different trade-offs and can be appropriate for different scenarios. It is reasonable to view the choice among progress metrics we test—and their variants—as selecting points along a Pareto front: EMA-smoothed metrics are natural in noisier regimes where stability is prioritized; absolute gap (GOI) can be suitable when task scales are comparable and emphasizing large residuals is desirable; normalized RoI balances responsiveness with scale-awareness; and variance-aware or uncertainty-adjusted variants (e.g., RoI with adaptive baselines, control-variates, or noise penalties) may be preferable when responsiveness should be tempered by measured uncertainty. Which point is preferable depends on what we prioritize (e.g., efficiency under a short horizon, dispersion across targets, or robustness to measurement noise). Since post-training can reveal benefits that are not fully visible at pre-training time, it would be valuable to evaluate these metrics with a standardized SFT probe in future work, so that claims reflect where each choice lands on the Pareto front for the objectives we care about.

Scaling to larger target suites. Our experiments indicate that coupling a target-centric priority with domain-level allocation yields a curriculum that improves both responsiveness and stability when paced to the dynamics of learning. GRAPE increases the weight of targets whose progress slows and shifts domain sampling toward sources whose gradients are most aligned with helping those targets.

These strengths have practical limits. Updating task weights with full information requires computing alignment or progress signals for every target at each reweighting step. Periodic updates mitigate cost, yet as the number of targets grows the task-side overhead can dominate, both computationally and in memory footprint.

For large target suites, one scalable option is to cluster benchmarks into ability-coherent groups and optimize over the resulting composite targets. This compresses the task distribution while preserving coverage of the underlying skills the clusters represent. Similarity signals for clustering can come from dataset metadata (topic, genre, labeling protocol) or embedding-space affinities (representation or gradient features). The objective remains structured and tractable, and clusters can be refreshed periodically so that the grouping adapts as the model’s representations evolve.

Another option is to select a small set of representative targets that act as proxies for broader ability families. In our experiments, emphasizing core reasoning benchmarks led to gains that transferred to other reasoning tasks, indicating that training on few can improve many within the same ability cluster. This reduces inner-loop complexity while retaining downstream benefits. Representatives can be re-evaluated and updated over time (e.g., via periodic transfer checks or gradient-alignment diagnostics) to ensure they continue to span the abilities of interest.

A third practical remedy is to replace full-information task updates with partial feedback while preserving the target-centric objective. The idea is to observe a small, explicitly explored subset of targets at each reweighting step and to compensate for the missing observations by importance weighting, so that the resulting update remains faithful—on average—to the full-information signal but at substantially lower cost.

Let $\mathbf{z}^{(t)} \in \Delta_N$ denote the current task-weight vector. At reweighting step t , draw a subset S_t of size $m \ll N$ by sampling each task with probability

$$p_n^{(t)} = (1 - \varepsilon) z_n^{(t)} + \frac{\varepsilon}{N}, \quad \varepsilon \in (0, 1].$$

The mixture with the uniform distribution ensures persistent exploration so that coverage does not collapse. For each sampled task $n \in S_t$, compute the usual alignment or progress signal $a_n^{(t)}$ as in the full-information method and map it to a bounded reward

$$r_n^{(t)} = \frac{1}{2} \left(1 - \frac{a_n^{(t)}}{C} \right) \in [0, 1], \quad C > 0 \text{ chosen so that } |a_n^{(t)}| \leq C.$$

Larger rewards correspond to more misaligned, slower-improving tasks, maintaining GRAPE’s emphasis on laggards. Because only tasks in S_t are observed, form an importance-weighted estimator

$$\hat{r}_n^{(t)} = \frac{r_n^{(t)}}{p_n^{(t)}} \mathbb{1}_{\{n \in S_t\}},$$

which is unbiased for the full-information reward. Update the task weights with a multiplicative (EXP3-style [84]) rule,

$$\tilde{z}_n^{(t+1)} = z_n^{(t)} \exp(\eta_z \hat{r}_n^{(t)}), \quad z_n^{(t+1)} = \frac{\tilde{z}_n^{(t+1)}}{\sum_{j=1}^N \tilde{z}_j^{(t+1)}},$$

so that probability mass shifts toward targets whose signals indicate insufficient progress. In practice, estimator variance can be stabilized with baseline subtraction or clipping of $\hat{r}_n^{(t)}$; these controls do not alter the expectation of the update.

To couple the task update to the domain step without evaluating all targets, approximate the target mixture used for validation with a self-normalized plug-in estimator. Define

$$\bar{z}_n^{(t)} = \frac{z_n^{(t)} \mathbb{1}_{\{n \in S_t\}} / p_n^{(t)}}{\sum_{j \in S_t} z_j^{(t)} / p_j^{(t)}} \quad (n \in S_t), \quad \bar{z}_n^{(t)} = 0 \quad (n \notin S_t),$$

so that $\sum_n \bar{z}_n^{(t)} = 1$. Draw validation under $\text{mix}(\bar{\mathbf{z}}^{(t)})$, compute the alignment terms exactly as in GRAPE, and apply the standard multiplicative update for domain weights. This leaves the outer objective and rule unchanged; only the construction of the target mixture is approximate but consistent in expectation.

The computational effect is direct: the task-side work per reweighting step scales with m rather than N , reducing the dominant term from $\mathcal{O}(N)$ to $\mathcal{O}(m)$ while preserving explicit exploration and, in expectation, the same emphasis on slowly improving targets. The principal design choices are the exploration level ϵ , the subset size m , and simple variance controls; these can be tuned with the same cadence used in the full-information method. No experiments are reported here; this construction is provided as a concrete pathway to extend GRAPE when the target set grows into the hundreds or thousands.

Computational efficiency of adaptive reweighting. Beyond reducing the number of targets evaluated, computational efficiency can also be improved at the gradient level. Adaptive reweighting in GRAPE and DoGE requires repeated gradient–alignment estimates, incurring non-trivial compute and memory overhead regardless of whether all tasks are evaluated or a subset is sampled. A promising complementary direction is ghost gradient (ghost-product) computation [88], which obtains the required inner products via lightweight Jacobian–vector operations without materializing full per-task or per-domain gradients. This preserves the alignment signal while reducing peak memory and auxiliary compute, enabling more frequent updates and scaling to larger models, more domains, and more targets—orthogonal to the strategies above. Open questions include the variance of ghost-based estimates and their interaction with GRAPE’s minimax dynamics; stabilizing strategies (e.g., simple EMA smoothing) could be explored empirically. Investigating ghost gradients offers a practical path to improving wall-clock efficiency without altering the core objectives of these methods.

Chapter 8

Conclusion

This thesis addressed a fundamental challenge in large language model pretraining: how to optimize data mixtures across multiple, potentially competing objectives. We introduced **GRAPE**, a principled framework that jointly optimizes data mixtures through a minimax formulation, dynamically identifying lagging tasks and adapting domain weights to achieve balanced progress across all targets.

8.1 Key Findings

Our experiments yielded several important insights:

Emergent curricula reflect skill hierarchies. GRAPE autonomously discovered task orderings that follow natural skill progressions, where fundamental abilities develop before more complex ones. This behavior emerged without explicit curriculum design, suggesting that the optimization dynamics naturally align with effective learning trajectories.

Balanced progress enables better generalization. By preventing any single task from dominating the training signal, GRAPE reduced interference among objectives and created more favorable conditions for transfer learning. Models initialized under GRAPE trajectories fine-tune more effectively than those from static mixtures, even for tasks where end-of-pretraining improvements are modest.

Adaptive methods scale favorably. GRAPE maintained its advantages when scaling from 125M to 0.7B parameters, though the margin over uniform sampling narrowed at larger scales. This suggests that adaptive reweighting provides greater benefits in data-limited regimes, where careful selection matters most.

8.2 Limitations and Future Work

Several limitations suggest directions for future research:

Scaling to many targets. Computing alignment signals for all tasks at each reweighting step becomes computationally expensive as the number of targets grows. Future work could explore clustering benchmarks into ability-coherent groups, selecting representative targets as proxies, or using partial feedback with importance weighting to reduce computational overhead.

Progress metric selection. Although we compared several progress metrics, the optimal choice likely depends on specific training scenarios. Developing principled methods for selecting or combining metrics based on dataset characteristics remains an open challenge.

Computational efficiency. The gradient alignment computations in GRAPE incur non-trivial overhead. Techniques such as ghost gradients could substantially reduce memory and compute requirements, enabling more frequent updates and scaling to larger models.

8.3 Closing Remarks

The question of what data to train on—and in what proportions—is fundamental to machine learning. This thesis demonstrates that adaptive, multi-objective optimization of training mixtures can substantially improve model quality, efficiency, and robustness. By grounding our approach in principled optimization theory while remaining practical and scalable, GRAPE offers a path toward more reliable and balanced language models that serve diverse objectives reliably.

Appendix A

Ablation Studies

We conduct a comprehensive ablation study across twelve distinct task combinations (T1–T12) spanning reasoning, mathematics, language understanding, logical reasoning, scientific knowledge, medical reasoning, and code generation. We compare five methods: GRAPE, GRAPE-EMA, GRAPE-GAP, DoGE, and uniform sampling, measuring performance using both log-perplexity on target tasks and 5-shot exact-match accuracy.

Our experiments demonstrate that GRAPE variants consistently outperform baseline methods across diverse task compositions, with GRAPE achieving the best average perplexity in 6 out of 12 configurations. The experiments reveal distinct performance patterns: GRAPE-EMA excels on target task accuracy (6 wins), while GRAPE-GAP dominates on unseen task transfer (6 wins), suggesting that different variants optimize for specialization versus generalization. This systematic evaluation across two-task combinations (T2–T5), three-task combinations (T1, T7), and multi-task combinations (T6, T8–T12) validates the robustness of adaptive data weighting strategies across varying domain mixtures and task structures.

Progress Assessment Metrics for Group DRO

We evaluate the impact of different step-wise progress metrics on the performance of Group DRO within the GRAPE framework. In addition to the primary *Rate-of-Improvement* (ROI) metric, we investigate two alternative progress measures that give rise to the GRAPE-GAP and GRAPE-EMA variants evaluated throughout this chapter:

1. **Gap-of-Improvement (GOI):** measures the absolute reduction in loss per step.
2. **EMA-Rate-of-Improvement (ROI-ema):** normalizes loss reduction by an exponential moving average of the task loss to account for scale differences across tasks.

Metric definitions. The step-wise improvement of task $\mathcal{T}_n$ at step t is assessed as:

$$\begin{aligned} \text{GOI}_n^{(t)} &:= l_n(\boldsymbol{\theta}_t) - l_n(\boldsymbol{\theta}_{t+1}), \\ \text{ROI-ema}_n^{(t)} &:= \frac{l_n(\boldsymbol{\theta}_t) - l_n(\boldsymbol{\theta}_{t+1})}{l_{\text{ema},n}^t}, \quad \forall n \in [N], \end{aligned} \tag{A.1}$$

where the exponential moving average loss $l_{\text{ema},n}^t$ is updated as

$$l_{\text{ema},n}^t = \beta \cdot l_{\text{ema},n}^{t-1} + (1 - \beta) \cdot l_n(\boldsymbol{\theta}_t), \tag{A.2}$$

with hyperparameter $\beta = 0.7$ in our experiments.

Resulting update rules. Substituting these alternative progress metrics for r_n in the minimax objective yields modified update rules for task weights $\mathbf{z}$ and domain weights $\boldsymbol{\alpha}$. The core optimization principle remains unchanged, but the specific gradient terms within the exponential updates differ:

GRAPE–GAP (using GOI):

$$\begin{aligned}
 \mathbf{z}^t &= \frac{\hat{\mathbf{z}}^t}{\sum_{n \in [N]} \hat{z}_n^t} \quad \text{where} \quad \hat{z}_n^t \leftarrow z_n^{t-1} \cdot \exp \left(- \frac{\gamma_t}{\mu_{\mathbf{z}}} \mathbb{E}_{\mathbf{x} \sim \text{mix}(\boldsymbol{\alpha}^{t-1})} [\langle \nabla_{\boldsymbol{\theta}} l_n(\boldsymbol{\theta}_t), \nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}_t, \mathbf{x}) \rangle] \right) \\
 \boldsymbol{\alpha}^t &= \frac{\hat{\boldsymbol{\alpha}}^t}{\sum_{k \in [K]} \hat{\alpha}_k^t} \quad \text{where} \quad \hat{\alpha}_k^t \leftarrow \alpha_k^{t-1} \cdot \exp \left(\frac{\gamma_t}{\mu_{\boldsymbol{\alpha}}} \mathbb{E}_{\mathbf{y} \sim \text{mix}(\mathbf{z}^{t-1})} [\langle \mathbf{g}_k(\boldsymbol{\theta}_t), \nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}_t, \mathbf{y}) \rangle] \right)
 \end{aligned} \tag{A.3}$$

GRAPE–EMA (using ROI-ema):

$$\begin{aligned}
 \mathbf{z}^t &= \frac{\hat{\mathbf{z}}^t}{\sum_{n \in [N]} \hat{z}_n^t} \quad \text{where} \quad \hat{z}_n^t \leftarrow z_n^{t-1} \cdot \exp \left(- \frac{\gamma_t}{\mu_{\mathbf{z}}} \mathbb{E}_{\mathbf{x} \sim \text{mix}(\boldsymbol{\alpha}^{t-1})} \left[\left\langle \frac{\nabla_{\boldsymbol{\theta}} l_n(\boldsymbol{\theta}_t)}{l_{\text{ema},n}^t}, \nabla_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}_t, \mathbf{x}) \right\rangle \right] \right) \\
 \boldsymbol{\alpha}^t &= \frac{\hat{\boldsymbol{\alpha}}^t}{\sum_{k \in [K]} \hat{\alpha}_k^t} \quad \text{where} \quad \hat{\alpha}_k^t \leftarrow \alpha_k^{t-1} \cdot \exp \left(\frac{\gamma_t}{\mu_{\boldsymbol{\alpha}}} \mathbb{E}_{\mathbf{y} \sim \text{mix}(\mathbf{z}^{t-1})} [\langle \mathbf{g}_k(\boldsymbol{\theta}_t), \nabla_{\boldsymbol{\theta}} \log \ell(\boldsymbol{\theta}_t, \mathbf{y}) \rangle] \right)
 \end{aligned} \tag{A.4}$$

Task Combinations

Table A.1 organizes the twelve experimental configurations by domain focus, specifying the target tasks selected for each combination. This systematic organization isolates the effects of specific domain interactions and reveals how task composition influences the relative performance of adaptive weighting strategies.

Task	Domain Focus	Target Tasks
Two-Task Combinations		
T2	Math + Commonsense	GSM8K, PIQA
T3	Math + Commonsense	GSM8K, HellaSwag
T4	Math + Logic	GSM8K, LogiQA
T5	Math + Science	GSM8K, SciQ
Three-Task Combinations		
T1	Science + Math	ARC-C, ARC-E, GSM8K
T7	Math + Commonsense + Code	GSM8K, HellaSwag, KodCode
Multi-Task Combinations		
T6	Science + Math + Code	ARC-C, ARC-E, GSM8K, KodCode
T8	Science + Math + Commonsense + Code + Logic	ARC-C, ARC-E, GSM8K, HellaSwag, KodCode, LogiQA, PIQA, SciQ
T9	Core Reasoning	ARC-C, ARC-E, HellaSwag, LogiQA, PIQA, SciQ
T10	Medical + Science + Math + Commonsense + Logic	ARC-C, ARC-E, HellaSwag, LogiQA, MathQA, MedQA, PIQA, SciQ
T11	Medical + Math + Science + Commonsense + Logic	ARC-C, ARC-E, HellaSwag, LogiQA, MathQA, MedQA
T12	Medical + Math + Commonsense + Logic	HellaSwag, LogiQA, MathQA, MedQA

Table A.1: All Target Tasks Sets (T1–T12) : mapping organized by domain focus. Each combination tests different aspects of multi-target adaptive pretraining across reasoning, mathematical, linguistic, logical, scientific, medical, and coding domains.

Aggregate Performance Summary

GRAPE variants achieve superior performance across all twelve experimental configurations, with GRAPE winning 6 out of 12 configurations on average perplexity and GRAPE–GAP winning 3 configurations. Table A.2 shows that GRAPE variants consistently achieve the best perplexity scores across the majority of experiments, demonstrating robust performance across diverse task compositions. The perplexity improvements are substantial, with GRAPE achieving average perplexities as low as 2.93 (T7) compared to baseline methods that typically exceed 3.5.

Tasks	Uniform	DoGE	GRAPE	GRAPE-GAP	GRAPE-EMA	Best (avg)
T1 (Science + Math)	4.30 3.67 4.66	4.18 3.60 4.53	3.53 3.49 3.55	<u>3.58</u> <u>3.55</u> <u>3.59</u>	<u>3.68</u> <u>3.60</u> <u>3.73</u>	GRAPE
T2 (Math + Commonsense)	4.11 3.67 4.55	4.00 3.62 4.38	<u>3.70</u> 3.55 <u>3.85</u>	3.69 <u>3.57</u> 3.80	<u>3.73</u> <u>3.57</u> <u>3.89</u>	GRAPE-GAP
T3 (Math + Commonsense)	4.01 3.67 4.34	3.94 3.63 4.24	3.61 3.52 3.70	<u>3.68</u> <u>3.65</u> <u>3.70</u>	<u>3.70</u> <u>3.58</u> <u>3.81</u>	GRAPE
T4 (Math + Logic)	3.75 3.67 3.83	3.67 3.64 3.69	<u>3.40</u> <u>3.33</u> 3.47	<u>3.41</u> 3.28 <u>3.54</u>	3.38 <u>3.29</u> <u>3.46</u>	GRAPE-EMA
T5 (Math + Science)	3.85 3.67 4.02	3.70 3.54 3.85	<u>3.37</u> <u>3.30</u> 3.43	3.35 3.22 <u>3.48</u>	<u>3.37</u> <u>3.27</u> <u>3.47</u>	GRAPE-GAP
T6 (Science + Math + Code)	3.63 1.60 4.66	3.63 1.51 4.77	3.04 <u>1.55</u> <u>3.58</u>	<u>3.05</u> 1.52 3.52	<u>3.10</u> <u>1.54</u> <u>3.65</u>	GRAPE
T7 (Math + Comm. + Code)	3.20 1.60 4.34	3.12 1.49 4.26	2.93 1.55 <u>3.80</u>	<u>3.11</u> <u>2.01</u> 3.72	<u>2.96</u> <u>1.58</u> <u>3.81</u>	GRAPE
T8 (Sci. + Math + Comm. + Code + Logic)	3.91 1.60 4.66	3.83 1.60 4.58	3.46 1.68 4.30	<u>3.51</u> <u>1.93</u> <u>4.32</u>	<u>3.53</u> <u>1.70</u> <u>4.42</u>	GRAPE
T9 (Core Reasoning)	4.33 3.83 4.66	4.13 3.66 4.51	3.86 3.23 4.19	<u>3.88</u> <u>3.21</u> <u>4.21</u>	<u>3.88</u> <u>3.23</u> <u>4.21</u>	GRAPE
T10 (Med. + Sci. + Math + Comm. + Logic)	4.63 4.08 4.92	4.44 3.90 4.82	<u>3.95</u> <u>2.58</u> <u>4.52</u>	3.86 2.42 4.55	<u>3.91</u> <u>2.45</u> <u>4.51</u>	GRAPE-GAP
T11 (Med. + Math + Sci. + Comm. + Logic)	4.65 4.08 4.92	4.42 3.88 4.74	<u>3.91</u> <u>2.61</u> 4.52	3.81 2.45 <u>4.53</u>	<u>3.90</u> <u>2.51</u> <u>4.53</u>	GRAPE-GAP
T12 (Med. + Math + Comm. + Logic)	4.53 4.60 4.88	4.33 4.32 4.76	<u>3.70</u> <u>2.61</u> <u>4.51</u>	3.59 2.40 4.50	<u>3.73</u> <u>2.54</u> <u>4.56</u>	GRAPE-GAP
Wins (avg)	0	0	6	3	3	

Table A.2: Log perplexity summary for all ablation settings (lower is better). Each cell shows **avg** | **min** | **max** (across target tasks within the combination) for each method. “Best (avg)” indicates which method achieved the lowest average perplexity per task set. “Wins (avg)” gives the total out of 12 ablations. Bold values denote best performance; underlined values denote second-best performance.

The accuracy analysis reveals a clear specialization-generalization trade-off across GRAPE variants. GRAPE-EMA dominates target task performance with 6 wins out of 12 configurations, while GRAPE-GAP excels on unseen task transfer with 6 wins. This pattern suggests that exponential moving average smoothing optimizes for target task specialization, whereas the GAP-based gradient projection strategy prioritizes broad generalization. Table A.3 shows that GRAPE-EMA achieves the highest number of wins (7 out of 12) on average target task accuracy, while GRAPE-GAP maintains superior transfer performance across diverse held-out evaluation tasks. The consistent outperformance of GRAPE variants over DoGE and uniform sampling validates the effectiveness of adaptive data weighting strategies across varying task compositions.

Method	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10	T11	T12	Wins (/12)
Target tasks (i)													
DoGE	18.2	28.6	13.1	<u>14.6</u>	28.2	14.0	10.2	28.1	<u>37.7</u>	31.7	25.0	23.8	1 / 12
GRAPE	<u>18.9</u>	<u>29.1</u>	<u>13.4</u>	14.4	<u>28.9</u>	15.2	<u>10.4</u>	28.9	38.3	33.2	<u>25.6</u>	24.4	3 / 12
GRAPE-GAP	<u>18.9</u>	29.3	13.1	13.7	<u>28.9</u>	16.0	<u>10.4</u>	28.5	37.3	33.8	25.7	<u>24.1</u>	2 / 12
GRAPE-EMA	19.5	29.3	13.5	14.9	30.3	<u>15.5</u>	10.5	<u>28.7</u>	<u>37.7</u>	<u>33.3</u>	25.7	<u>24.1</u>	6 / 12
Unseen tasks (ii)													
DoGE	<u>35.8</u>	32.5	35.3	<u>35.5</u>	32.3	35.2	34.8	31.0	34.9	29.8	35.6	33.6	0 / 12
GRAPE	36.4	33.1	<u>35.4</u>	35.7	<u>33.0</u>	<u>36.1</u>	36.0	32.6	<u>35.1</u>	<u>34.5</u>	36.9	<u>40.1</u>	5 / 12
GRAPE-GAP	36.4	32.2	36.1	35.3	33.1	36.3	<u>35.9</u>	<u>32.1</u>	36.2	35.5	36.9	40.2	6 / 12
GRAPE-EMA	35.5	<u>32.9</u>	35.3	35.1	32.8	35.8	35.7	31.9	34.9	34.3	<u>36.8</u>	39.8	1 / 12

Table A.3: All Target Tasks Sets (T1–T12) : average 5-shot exact-match accuracy across (i) tasks set as targets in the method and (ii) unseen tasks (never used during training). “Wins (/12)” counts the number of times (out of 12) each method achieves the highest score per row. Bold values denote best performance; underlined values denote second-best performance.

Detailed Per-Task Analysis

The detailed per-task analysis reveals consistent patterns across all twelve experimental configurations. GRAPE variants achieve substantial perplexity reductions compared to baseline methods, with improvements ranging

from 0.3 to 1.0 log points across different task combinations. The weight dynamics visualizations show that GRAPE and its variants dynamically adjust attention across target tasks and training domains throughout training, with GRAPE-EMA showing more stable weight evolution and GRAPE-GAP exhibiting more aggressive rebalancing. Transfer performance tables demonstrate that models trained with GRAPE variants maintain strong generalization to held-out evaluation tasks, with GRAPE-GAP particularly excelling on unseen task transfer across diverse domain combinations.

Task combination T1: ARC-C, ARC-E, GSM8K

Method	ARC-C	ARC-E	GSM8K	Avg
Log Perplexity				
Uniform	4.57	4.66	3.67	4.30
DoGE	4.41	4.53	3.60	4.18
GRAPE	3.55	3.55	3.49	3.53
GRAPE-GAP	<u>3.59</u>	<u>3.59</u>	<u>3.55</u>	<u>3.58</u>
GRAPE-EMA	3.71	3.73	<u>3.60</u>	<u>3.68</u>
5-shot exact-match accuracy scores (%)				
Uniform	21.50	32.45	0.00	17.98
DoGE	21.84	32.91	0.00	18.25
GRAPE	<u>21.76</u>	34.81	<u>0.08</u>	<u>18.88</u>
GRAPE-GAP	<u>23.04</u>	<u>33.80</u>	0.00	<u>18.95</u>
GRAPE-EMA	23.98	<u>34.34</u>	0.15	19.49

Table A.4: Target Tasks Set {ARC-C, ARC-E, GSM8K} : log-perplexity ↓ and 5-shot exact-match accuracy ↑ at 0.66B training tokens.

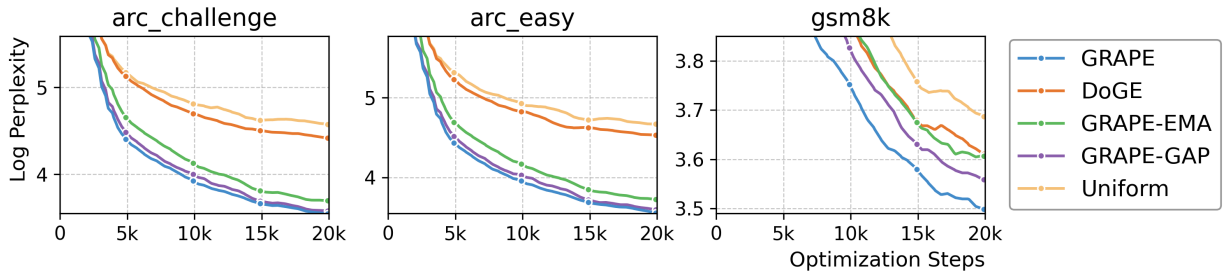


Figure A.1: Target Tasks Set {ARC-C, ARC-E, GSM8K} : log-perplexity on target tasks across methods (20k steps, 0.66B tokens).

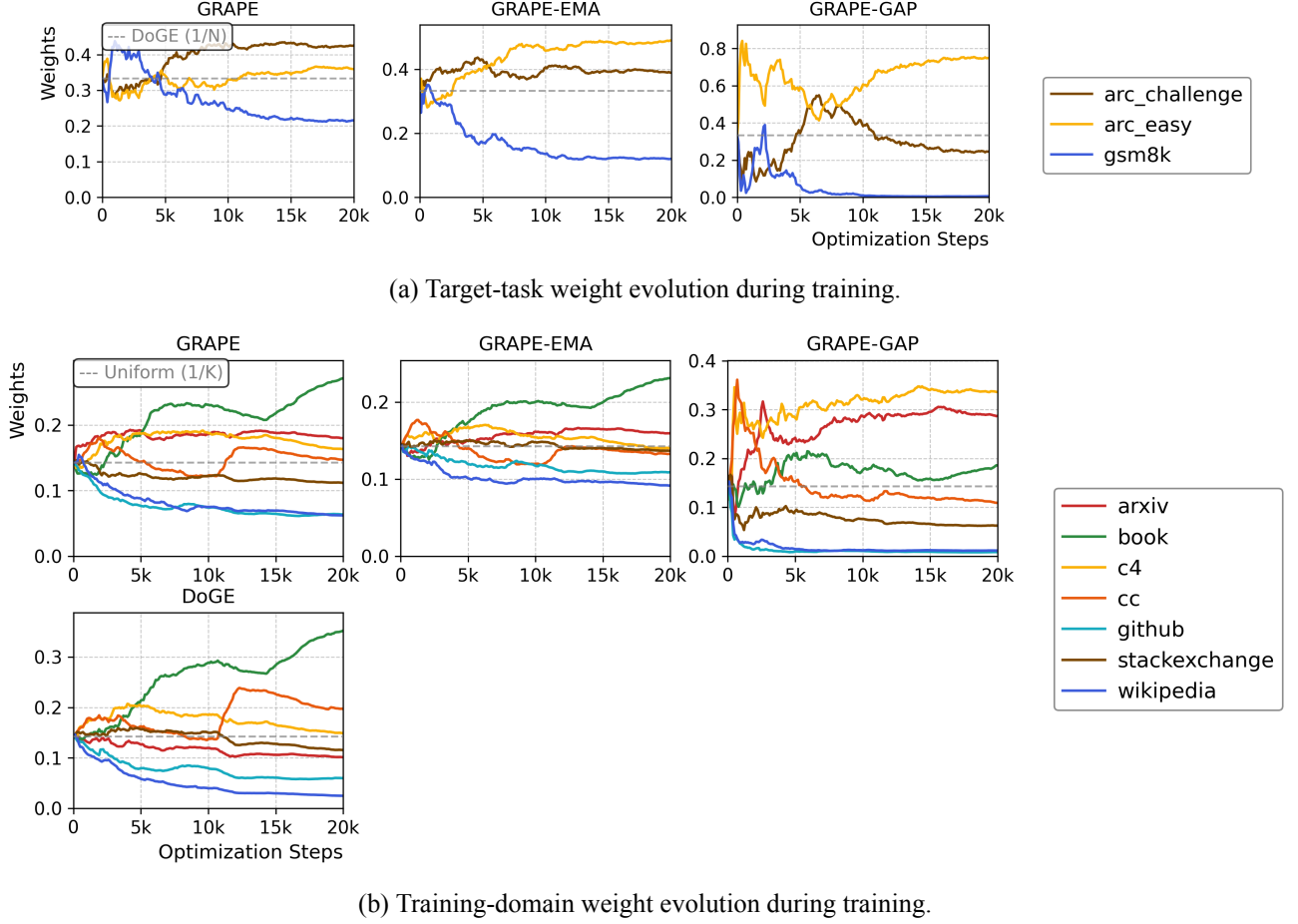


Figure A.2: Target Tasks Set {ARC-C, ARC-E, GSM8K} : weight dynamics. Top—target-task weights z ; bottom—training-domain weights α . Larger weights indicate greater emphasis during training.

Method	HellaSwag	LogiQA	SciQ	PIQA	MathQA	COPA	MMLU	OpenBookQA	RACE	WinoGrande	Avg
Uniform	26.44	26.42	52.40	55.50	20.97	56.00	25.28	13.20	23.73	51.30	32.53
DoGE	26.33	28.11	53.30	<u>57.13</u>	<u>21.00</u>	57.00	<u>25.14</u>	14.00	24.59	<u>51.46</u>	35.81
GRAPE	<u>26.58</u>	27.19	56.60	58.16	20.80	<u>59.00</u>	24.73	<u>14.80</u>	25.07	50.99	<u>36.39</u>
GRAPE-GAP	26.81	<u>28.42</u>	<u>54.80</u>	56.42	20.20	60.00	24.51	15.80	<u>24.88</u>	52.25	36.41
GRAPE-EMA	26.40	29.49	52.10	56.58	20.60	56.00	24.53	14.40	24.59	49.88	35.46

Table A.5: Target Tasks Set {ARC-C, ARC-E, GSM8K} : 5-shot accuracy $\uparrow$ on held-out tasks (%).

Task combination T2: GSM8K, PIQA

Method	GSM8K	PIQA	Avg
Log Perplexity			
Uniform	3.67	4.55	4.11
DoGE	3.62	4.38	4.00
GRAPE	3.55	<u>3.85</u>	<u>3.70</u>
GRAPE-GAP	<u>3.57</u>	3.80	3.69
GRAPE-EMA	<u>3.57</u>	3.89	3.73
5-shot exact-match accuracy scores (%)			
Uniform	0.00	55.50	27.75
DoGE	0.00	57.29	28.65
GRAPE	0.00	<u>58.11</u>	29.06
GRAPE-GAP	<u>0.15</u>	58.43	<u>29.29</u>
GRAPE-EMA	0.23	58.43	29.33

Table A.6: Target Tasks Set {GSM8K, PIQA} : log-perplexity ↓ and 5-shot exact-match accuracy ↑ at 0.66B training tokens.

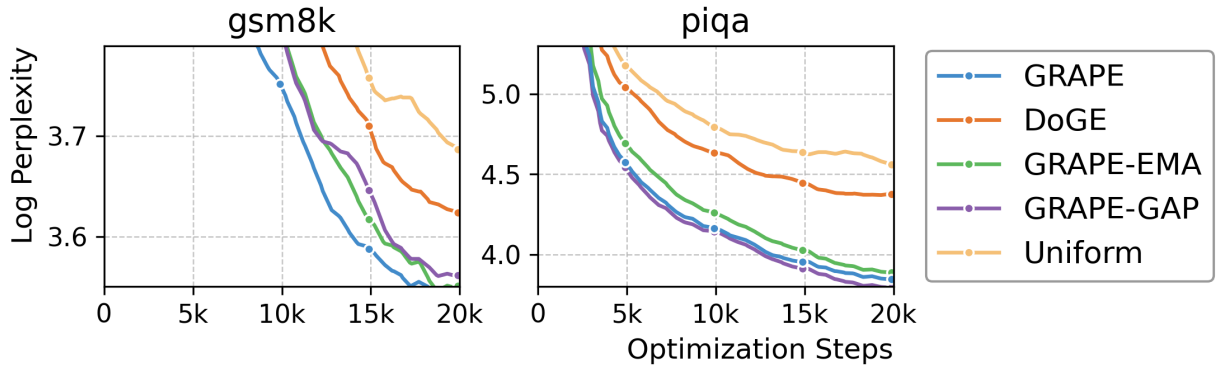
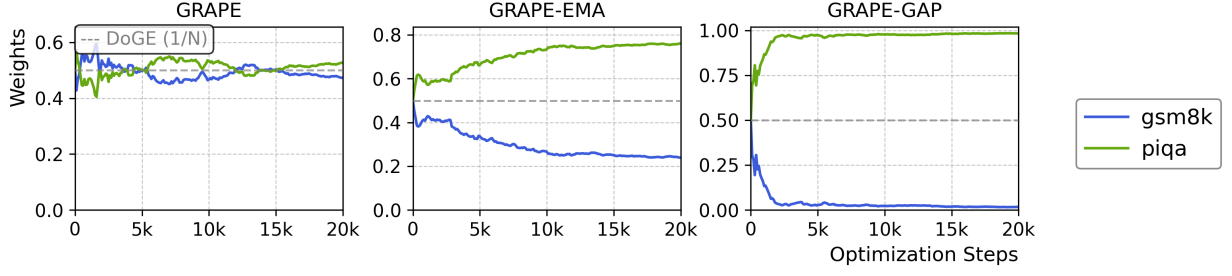
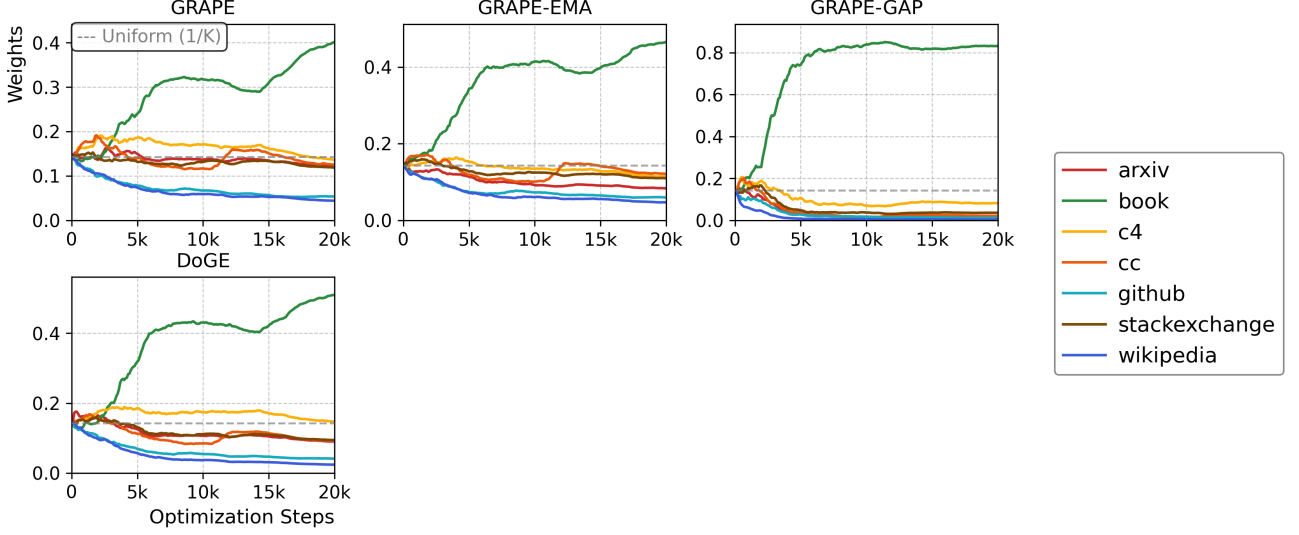


Figure A.3: Target Tasks Set {GSM8K, PIQA} : log-perplexity on target tasks across methods (20k steps, 0.66B tokens).



(a) Target-task weight evolution during training.



(b) Training-domain weight evolution during training.

Figure A.4: Target Tasks Set {GSM8K, PIQA} : weight dynamics. Top—target-task weights z ; bottom—training-domain weights α . Larger weights indicate greater emphasis during training.

T2	HellaSwag	LogiQA	ARC-C	ARC-E	SciQ	MathQA	COPA	MMLU	OpenBookQA	RACE	WinoGrande	Avg
Uniform	26.44	26.42	21.50	32.45	55.50	20.97	<u>56.00</u>	25.28	13.20	23.73	51.30	32.53
DoGE	26.63	29.80	<u>21.67</u>	33.04	55.20	21.40	54.00	<u>25.32</u>	<u>13.80</u>	<u>25.36</u>	51.30	32.50
GRAPE	26.38	<u>29.34</u>	21.42	33.16	58.30	<u>21.10</u>	58.00	24.18	12.40	26.03	53.51	33.07
GRAPE-GAP	<u>26.89</u>	23.81	21.93	34.76	56.00	20.00	55.00	25.45	<u>13.80</u>	<u>25.36</u>	<u>51.62</u>	32.24
GRAPE-EMA	27.02	27.80	21.59	<u>33.63</u>	<u>58.10</u>	20.10	58.00	25.29	14.00	25.17	51.38	<u>32.92</u>

Table A.7: Target Tasks Set {GSM8K, PIQA} : 5-shot accuracy $\uparrow$ on held-out tasks (%).

Task combination T3: GSM8K, HellaSwag

Method	GSM8K	HellaSwag	Avg
Log Perplexity			
Uniform	3.67	4.34	4.01
DoGE	3.63	4.24	3.94
GRAPE	3.52	3.70	3.61
GRAPE-GAP	3.65	3.70	3.68
GRAPE-EMA	<u>3.58</u>	<u>3.81</u>	<u>3.70</u>
5-shot exact-match accuracy scores (%)			
Uniform	<u>0.00</u>	26.44	13.22
DoGE	0.08	26.09	13.09
GRAPE	0.08	<u>26.78</u>	<u>13.43</u>
GRAPE-GAP	0.08	26.13	13.11
GRAPE-EMA	<u>0.00</u>	26.92	13.46

Table A.8: Target Tasks Set {GSM8K, HellaSwag} : log-perplexity ↓ and 5-shot exact-match accuracy ↑ at 0.66B training tokens.

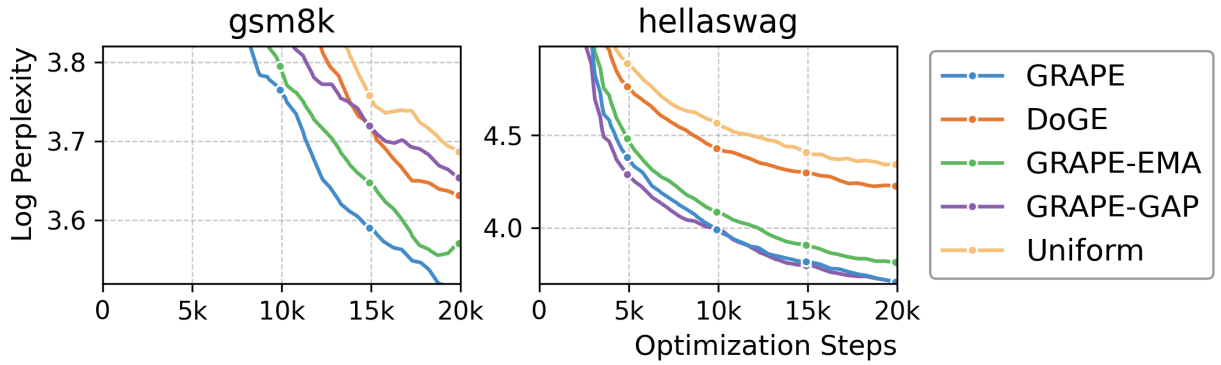
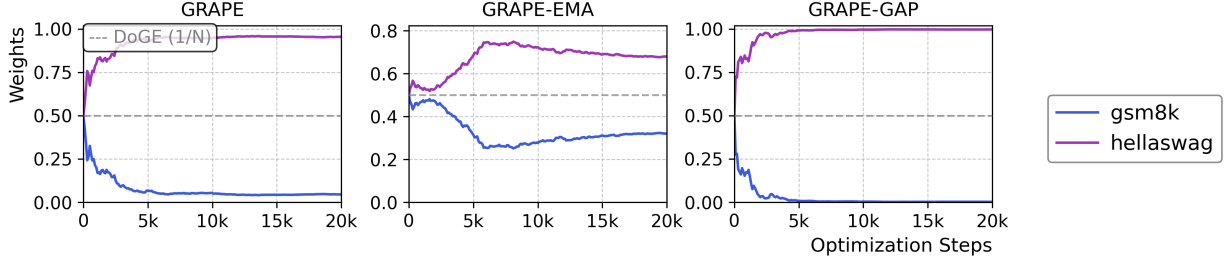
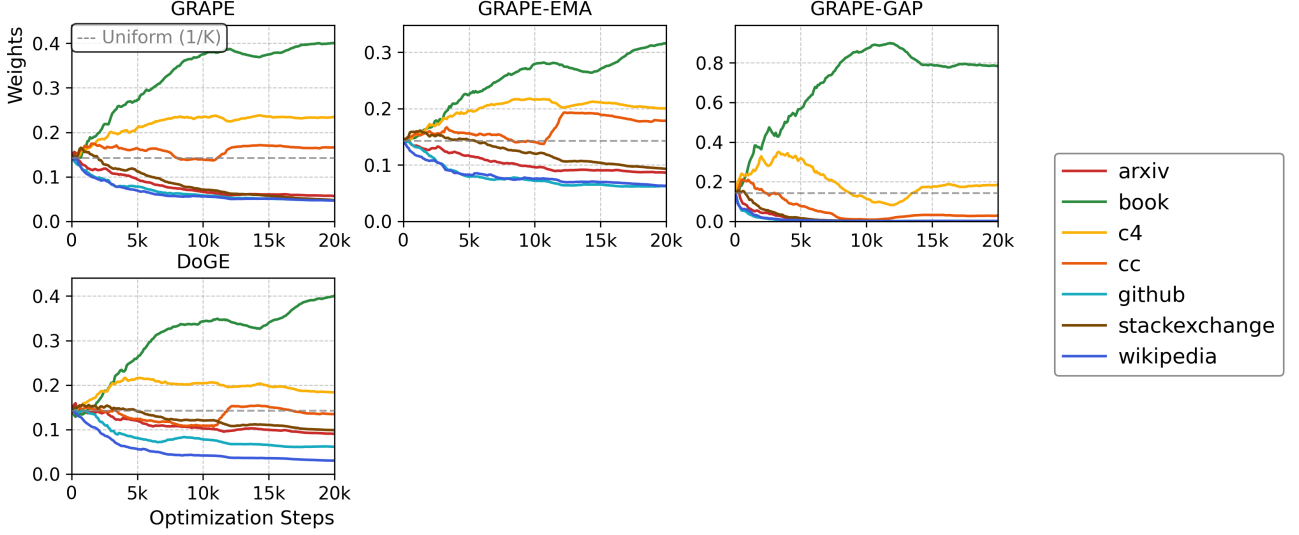


Figure A.5: Target Tasks Set {GSM8K, HellaSwag} : log-perplexity on target tasks across methods (20k steps, 0.66B tokens).



(a) Target-task weight evolution during training.



(b) Training-domain weight evolution during training.

Figure A.6: Target Tasks Set {GSM8K, HellaSwag} : weight dynamics. Top—target-task weights z ; bottom—training-domain weights α . Larger weights indicate greater emphasis during training.

T3	LogiQA	ARC-C	ARC-E	SciQ	PIQA	MathQA	COPA	MMLU	OpenBookQA	RACE	WinoGrande	Avg
Uniform	26.42	21.50	32.45	52.40	55.50	20.97	<u>56.00</u>	25.28	13.20	23.73	<u>51.30</u>	32.53
DoGE	28.26	21.50	33.50	54.50	57.45	20.60	55.00	25.44	13.60	24.98	52.96	35.25
GRAPE	<u>27.96</u>	21.08	33.38	56.70	59.25	<u>20.90</u>	<u>56.00</u>	24.59	13.00	25.36	51.22	<u>35.40</u>
GRAPE-GAP	26.88	23.04	34.34	<u>55.10</u>	58.81	20.80	62.00	25.29	<u>14.00</u>	25.55	50.75	36.05
GRAPE-EMA	27.50	<u>22.10</u>	<u>33.75</u>	54.90	<u>58.98</u>	<u>20.90</u>	55.00	<u>25.31</u>	14.20	<u>25.45</u>	50.59	35.33

Table A.9: Target Tasks Set {GSM8K, HellaSwag} : 5-shot accuracy $\uparrow$ on held-out tasks (%).

Task combination T4: GSM8K, LogiQA

Method	GSM8K	LogiQA	Avg
Log Perplexity			
Uniform	3.67	3.83	3.75
DoGE	3.64	3.69	3.67
GRAPE	<u>3.47</u>	<u>3.33</u>	<u>3.40</u>
GRAPE-GAP	3.54	3.28	3.41
GRAPE-EMA	3.46	<u>3.29</u>	3.38
5-shot exact-match accuracy scores (%)			
Uniform	<u>0.00</u>	26.42	13.21
DoGE	<u>0.00</u>	<u>29.19</u>	<u>14.60</u>
GRAPE	<u>0.00</u>	28.73	14.37
GRAPE-GAP	<u>0.00</u>	27.34	13.67
GRAPE-EMA	0.15	29.65	14.90

Table A.10: Target Tasks Set {GSM8K, LogiQA} : log-perplexity ↓ and 5-shot exact-match accuracy ↑ at 0.66B training tokens.

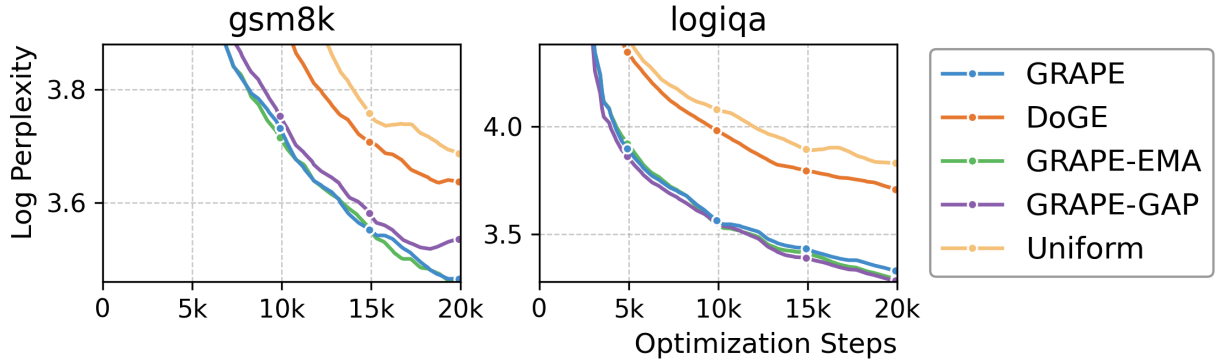
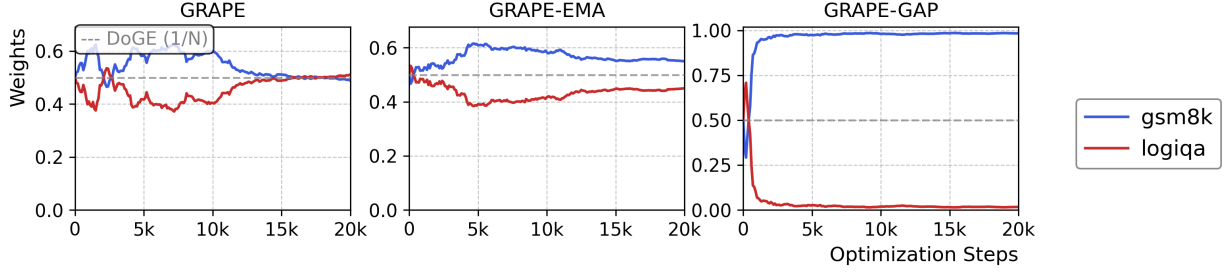
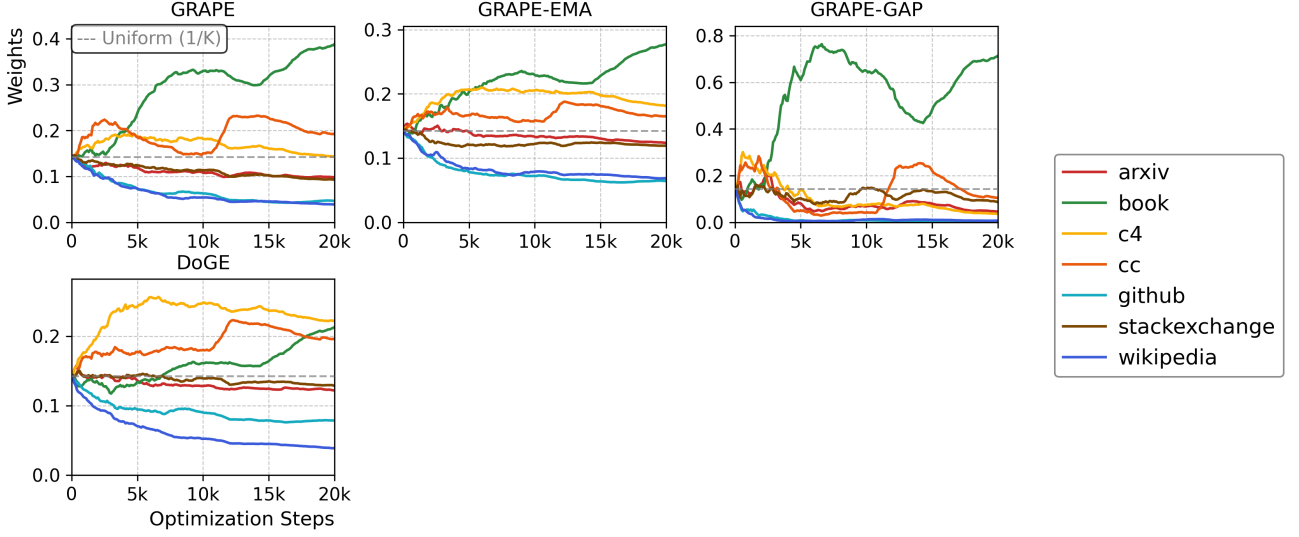


Figure A.7: Target Tasks Set {GSM8K, LogiQA} : log-perplexity on target tasks across methods (20k steps, 0.66B tokens).



(a) Target-task weight evolution during training.



(b) Training-domain weight evolution during training.

Figure A.8: Target Tasks Set {GSM8K, LogiQA} : weight dynamics. Top—target-task weights z ; bottom—training-domain weights α . Larger weights indicate greater emphasis during training.

T4	HellaSwag	ARC-C	ARC-E	SciQ	PIQA	MathQA	COPA	MMLU	OpenBookQA	RACE	WinoGrande	Avg
Uniform	26.44	21.50	32.45	52.40	55.50	<u>20.97</u>	56.00	<u>25.28</u>	13.20	23.73	<u>51.30</u>	32.53
DoGE	26.88	<u>22.10</u>	33.80	56.00	57.40	21.30	60.00	25.05	13.00	24.40	50.99	<u>35.54</u>
GRAPE	26.63	21.84	33.80	57.70	<u>58.05</u>	20.20	<u>58.00</u>	25.19	<u>13.60</u>	<u>25.55</u>	52.33	35.72
GRAPE-GAP	26.30	22.27	<u>33.71</u>	56.70	58.27	20.70	56.00	25.23	14.00	25.93	49.25	35.31
GRAPE-EMA	<u>26.87</u>	21.16	33.80	<u>57.40</u>	57.62	20.50	54.00	25.34	13.40	25.07	49.64	35.07

Table A.11: Target Tasks Set {GSM8K, LogiQA} : 5-shot accuracy $\uparrow$ on held-out tasks (%).

Task combination T5: GSM8K, SciQ

Method	GSM8K	SciQ	Avg
Log Perplexity			
Uniform	3.67	4.02	3.85
DoGE	3.54	3.85	3.70
GRAPE	3.43	<u>3.30</u>	<u>3.37</u>
GRAPE-GAP	<u>3.48</u>	3.22	3.35
GRAPE-EMA	<u>3.47</u>	<u>3.27</u>	<u>3.37</u>
5-shot exact-match accuracy scores (%)			
Uniform	0.00	52.40	26.20
DoGE	0.15	56.20	28.18
GRAPE	0.00	<u>57.80</u>	<u>28.90</u>
GRAPE-GAP	<u>0.08</u>	57.70	28.89
GRAPE-EMA	<u>0.08</u>	60.60	30.34

Table A.12: Target Tasks Set {GSM8K, SciQ} : log-perplexity ↓ and 5-shot exact-match accuracy ↑ at 0.66B training tokens.

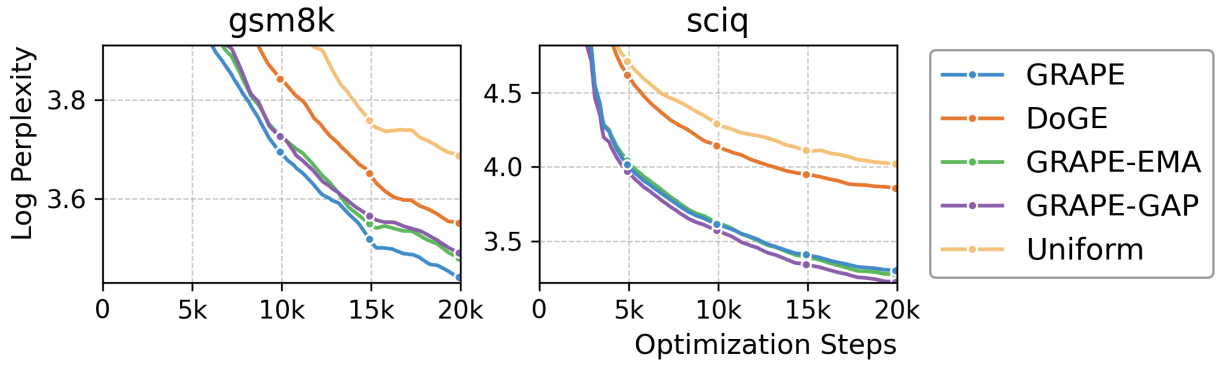
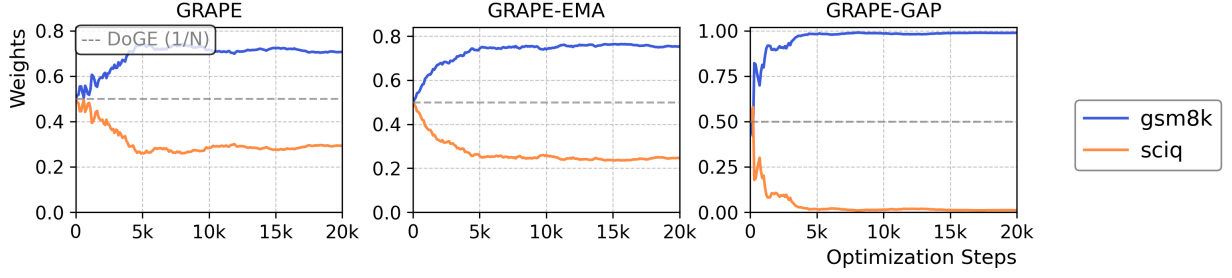
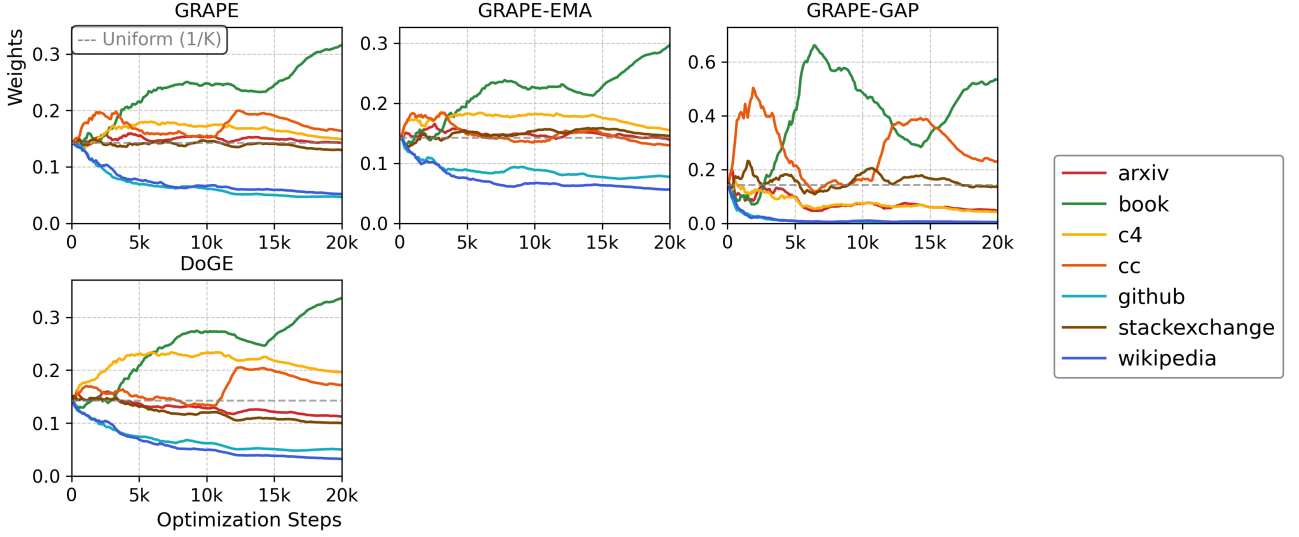


Figure A.9: Target Tasks Set {GSM8K, SciQ} : log-perplexity on target tasks across methods (20k steps, 0.66B tokens).



(a) Target-task weight evolution during training.



(b) Training-domain weight evolution during training.

Figure A.10: Target Tasks Set {GSM8K, SciQ} : weight dynamics. Top—target-task weights z ; bottom—training-domain weights α . Larger weights indicate greater emphasis during training.

T5	HellaSwag	LogiQA	ARC-C	ARC-E	PIQA	MathQA	COPA	MMLU	OpenBookQA	RACE	WinoGrande	Avg
Uniform	26.44	26.42	21.50	32.45	55.50	20.97	<u>56.00</u>	25.28	<u>13.20</u>	23.73	<u>51.30</u>	32.53
DoGE	26.57	26.73	21.93	32.62	57.40	20.40	55.00	<u>25.07</u>	12.60	<u>25.65</u>	51.14	32.28
GRAPE	26.77	31.49	22.70	<u>34.68</u>	<u>58.11</u>	20.80	55.00	24.31	13.80	24.98	50.75	<u>33.04</u>
GRAPE-GAP	<u>26.68</u>	<u>29.80</u>	<u>22.10</u>	34.18	58.22	20.80	58.00	24.95	13.00	25.84	50.04	33.05
GRAPE-EMA	26.66	27.65	21.42	35.31	57.83	<u>20.90</u>	<u>56.00</u>	24.94	12.60	24.31	53.35	32.81

Table A.13: Target Tasks Set {GSM8K, SciQ} : 5-shot accuracy $\uparrow$ on held-out tasks (%).

Task combination T6: ARC-C, ARC-E, GSM8K, KodCode

Method	ARC-C	ARC-E	GSM8K	KodCode	Avg
Log Perplexity					
Uniform	4.57	4.66	3.67	1.60	3.63
DoGE	4.64	4.77	3.60	1.51	3.63
GRAPE	<u>3.57</u>	<u>3.58</u>	3.44	<u>1.55</u>	3.04
GRAPE-GAP	3.52	3.52	<u>3.48</u>	1.69	<u>3.05</u>
GRAPE-EMA	3.64	3.65	3.55	<u>1.54</u>	3.10
5-shot exact-match accuracy scores (%)					
Uniform	21.50	32.45	<u>0.00</u>	3.88	14.46
DoGE	20.56	31.61	0.08	3.88	14.03
GRAPE	22.70	34.13	0.08	3.88	15.20
GRAPE-GAP	23.38	35.14	<u>0.00</u>	5.49	16.00
GRAPE-EMA	<u>23.04</u>	<u>34.89</u>	<u>0.00</u>	3.88	<u>15.45</u>

Table A.14: Target Tasks Set {ARC-C, ARC-E, GSM8K, KodCode} : log-perplexity ↓ and 5-shot exact-match accuracy ↑ at 0.66B training tokens.

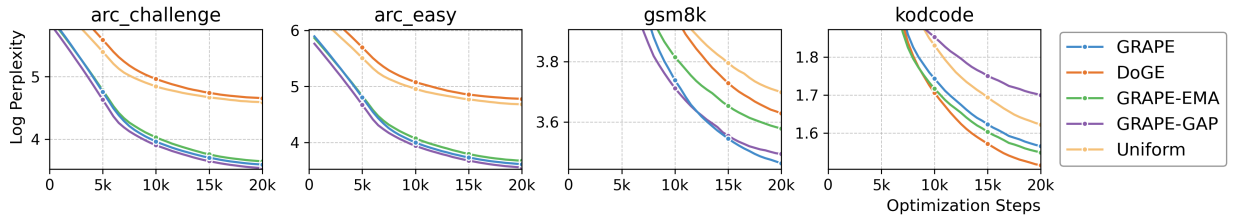


Figure A.11: Target Tasks Set {ARC-C, ARC-E, GSM8K, KodCode} : log-perplexity on target tasks across methods (20k steps, 0.66B tokens).

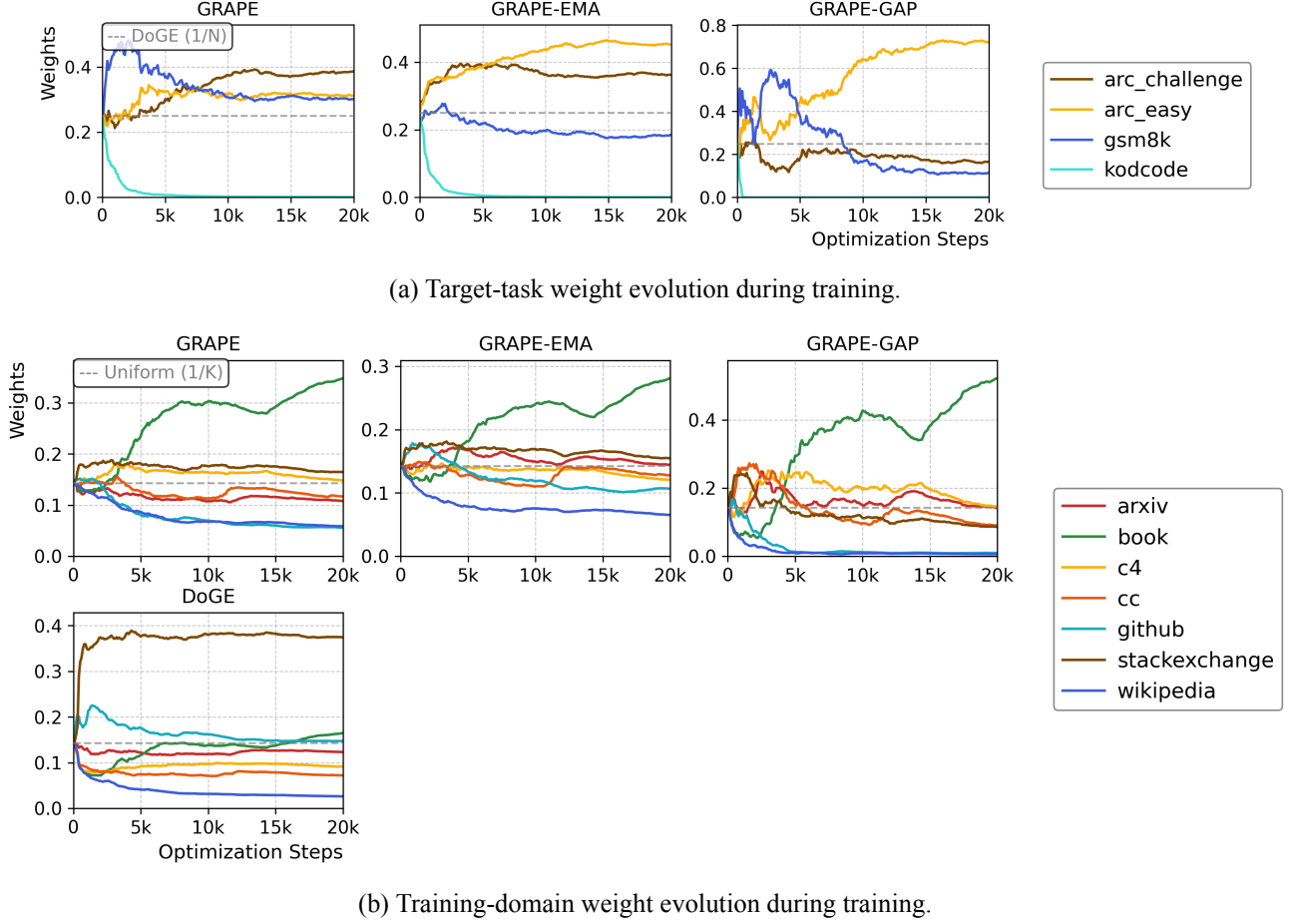


Figure A.12: Target Tasks Set {ARC-C, ARC-E, GSM8K, KodCode} : weight dynamics. Top—target-task weights z ; bottom—training-domain weights α . Larger weights indicate greater emphasis during training.

T6	HellaSwag	LogiQA	SciQ	PIQA	MathQA	COPA	MMLU	OpenBookQA	RACE	WinoGrande	Avg
Uniform	26.44	26.42	52.40	55.50	20.97	56.00	25.28	13.20	23.73	51.30	32.53
DoGE	26.64	26.27	53.00	55.88	<u>21.30</u>	57.00	24.33	12.80	24.78	50.43	35.24
GRAPE	26.13	26.57	56.90	58.49	21.40	<u>56.00</u>	24.03	<u>15.40</u>	25.07	52.17	<u>36.12</u>
GRAPE-GAP	<u>26.55</u>	27.65	<u>56.80</u>	<u>57.78</u>	21.10	55.00	<u>25.15</u>	15.80	<u>26.03</u>	51.46	36.33
GRAPE-EMA	26.45	<u>27.50</u>	55.00	<u>57.78</u>	20.40	54.00	24.94	14.40	26.12	<u>51.85</u>	35.84

Table A.15: Target Tasks Set {ARC-C, ARC-E, GSM8K, KodCode} : 5-shot accuracy $\uparrow$ on held-out tasks (%).

Task combination T7: GSM8K, HellaSwag, KodCode

Method	GSM8K	HellaSwag	KodCode	Avg
Log Perplexity				
Uniform	3.67	4.34	1.60	3.20
DoGE	3.61	4.26	1.49	3.12
GRAPE	3.45	<u>3.80</u>	1.55	2.93
GRAPE-GAP	<u>3.60</u>	3.72	2.01	<u>3.11</u>
GRAPE-EMA	<u>3.49</u>	3.81	<u>1.58</u>	<u>2.96</u>
5-shot exact-match accuracy scores (%)				
Uniform	0.00	26.44	3.88	10.11
DoGE	0.23	26.40	3.88	10.17
GRAPE	0.00	27.22	3.88	10.37
GRAPE-GAP	0.00	<u>27.01</u>	<u>4.29</u>	10.43
GRAPE-EMA	<u>0.08</u>	26.85	4.64	10.52

Table A.16: Target Tasks Set {GSM8K, HellaSwag, KodCode} : log-perplexity ↓ and 5-shot exact-match accuracy ↑ at 0.66B training tokens.

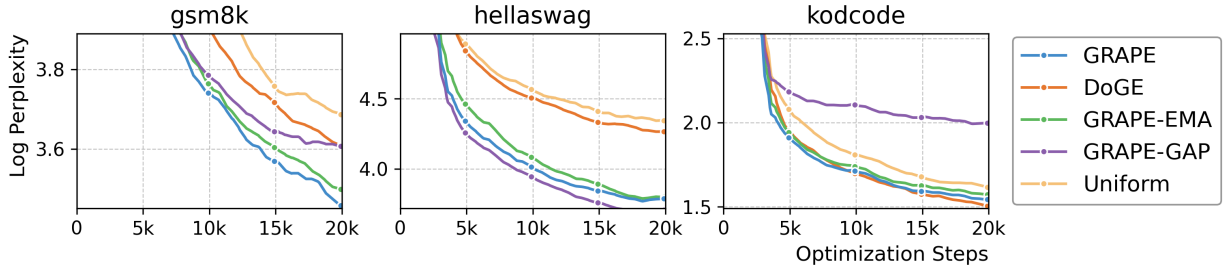


Figure A.13: Target Tasks Set {GSM8K, HellaSwag, KodCode} : log-perplexity on target tasks across methods (20k steps, 0.66B tokens).

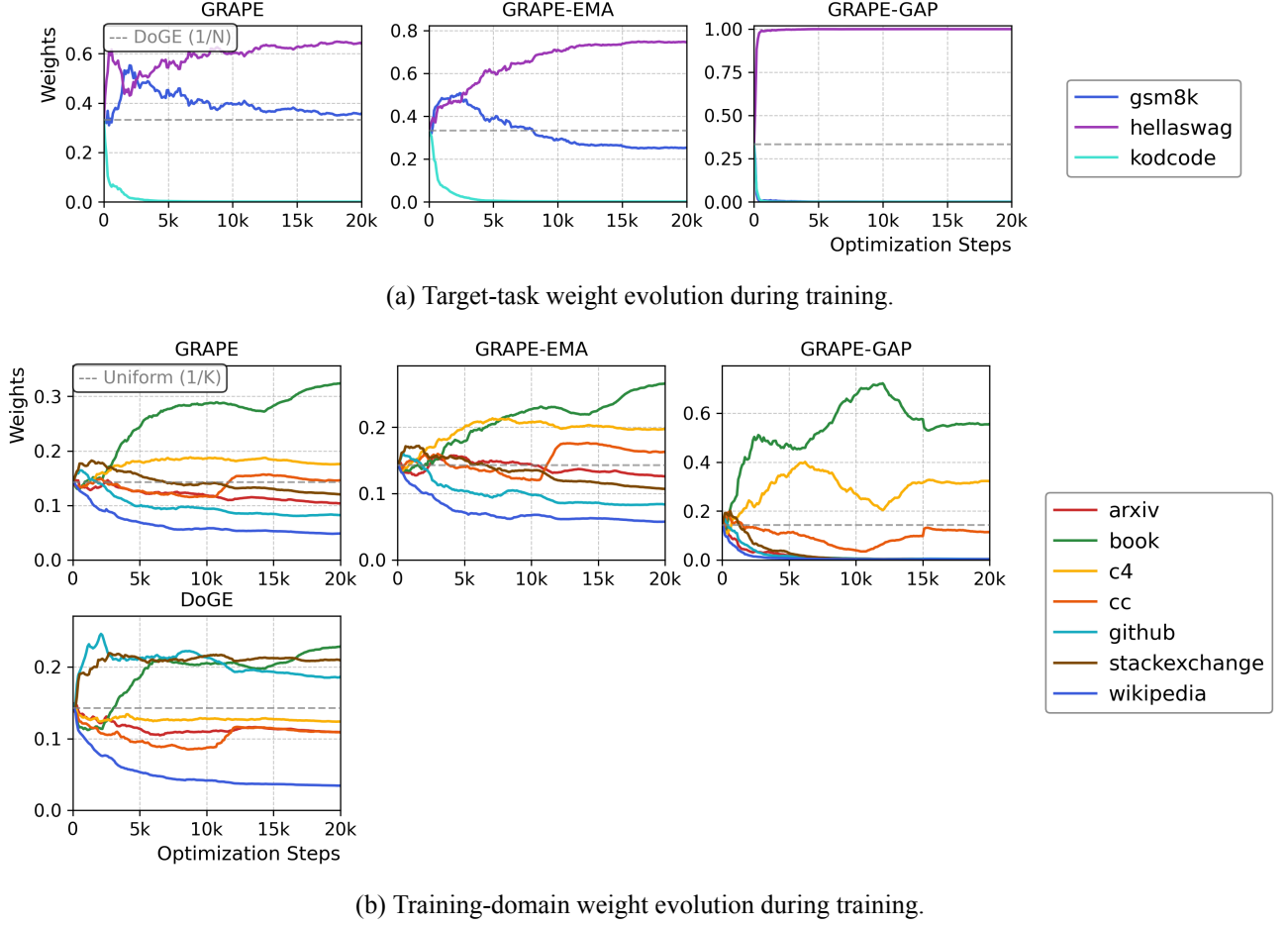


Figure A.14: Target Tasks Set {GSM8K, HellaSwag, KodCode} : weight dynamics. Top—target-task weights z ; bottom—training-domain weights α . Larger weights indicate greater emphasis during training.

T7	LogiQA	ARC-C	ARC-E	SciQ	PIQA	MathQA	COPA	MMLU	OpenBookQA	RACE	WinoGrande	Avg
Uniform	26.42	21.50	32.45	52.40	55.50	20.97	56.00	25.28	13.20	23.73	<u>51.30</u>	32.53
DoGE	31.03	22.18	32.20	53.00	57.34	20.50	53.00	<u>25.23</u>	13.00	24.78	50.12	34.76
GRAPE	<u>30.57</u>	<u>22.53</u>	<u>33.88</u>	56.50	59.63	21.10	57.00	25.22	<u>13.80</u>	25.74	50.43	36.04
GRAPE-GAP	25.50	22.10	33.96	<u>56.70</u>	<u>58.60</u>	<u>20.70</u>	60.00	24.94	15.40	<u>25.45</u>	51.85	<u>35.93</u>
GRAPE-EMA	28.88	22.95	33.63	56.90	57.67	<u>20.70</u>	<u>58.00</u>	24.81	13.40	25.17	50.75	35.72

Table A.17: Target Tasks Set {GSM8K, HellaSwag, KodCode} : 5-shot accuracy $\uparrow$ on held-out tasks (%).

Task combination T8: ARC-C, ARC-E, GSM8K, HellaSwag, KodCode, LogiQA, PIQA, SciQ

Method	ARC-C	ARC-E	GSM8K	HellaSwag	KodCode	LogiQA	PIQA	SciQ	Avg
Log Perplexity									
Uniform	4.57	4.66	3.67	4.34	1.60	3.83	4.55	4.02	3.91
DoGE	4.49	4.58	3.58	4.24	1.60	3.79	4.41	3.93	3.83
GRAPE	4.18	<u>4.30</u>	<u>2.09</u>	4.00	<u>1.68</u>	3.57	4.12	3.71	3.46
GRAPE-GAP	<u>4.20</u>	4.32	2.06	<u>4.05</u>	1.93	<u>3.62</u>	<u>4.15</u>	<u>3.76</u>	<u>3.51</u>
GRAPE-EMA	4.30	4.10	2.11	<u>4.42</u>	1.70	3.64	4.20	3.79	3.53
5-shot exact-match accuracy scores (%)									
Uniform	21.50	32.45	0.00	26.44	3.88	26.42	<u>55.50</u>	52.40	27.32
DoGE	21.50	33.04	0.08	<u>26.63</u>	3.88	27.80	55.00	57.13	28.13
GRAPE	21.76	<u>33.71</u>	0.76	26.10	4.29	29.19	55.40	59.74	28.89
GRAPE-GAP	<u>21.84</u>	32.49	1.06	26.10	4.29	26.73	56.20	<u>59.30</u>	28.50
GRAPE-EMA	22.01	34.13	<u>0.91</u>	26.87	4.96	<u>28.26</u>	54.90	57.45	<u>28.69</u>

Table A.18: Target Tasks Set {ARC-C, ARC-E, GSM8K, HellaSwag, KodCode, LogiQA, PIQA, SciQ} : log-perplexity ↓ and 5-shot exact-match accuracy ↑ at 0.66B training tokens.

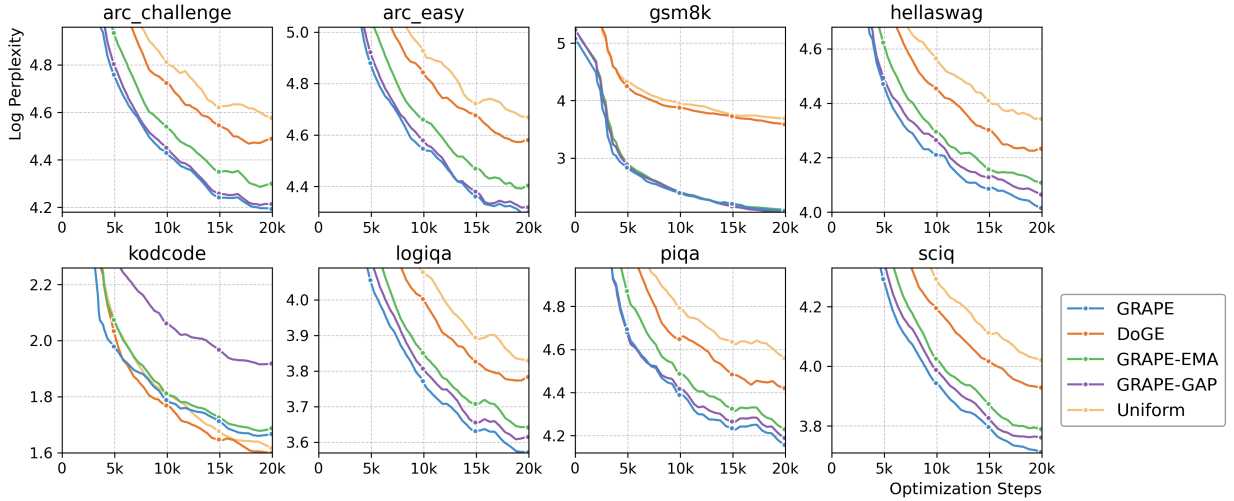


Figure A.15: Target Tasks Set {ARC-C, ARC-E, GSM8K, HellaSwag, KodCode, LogiQA, PIQA, SciQ} : log-perplexity on target tasks across methods (20k steps, 0.66B tokens).

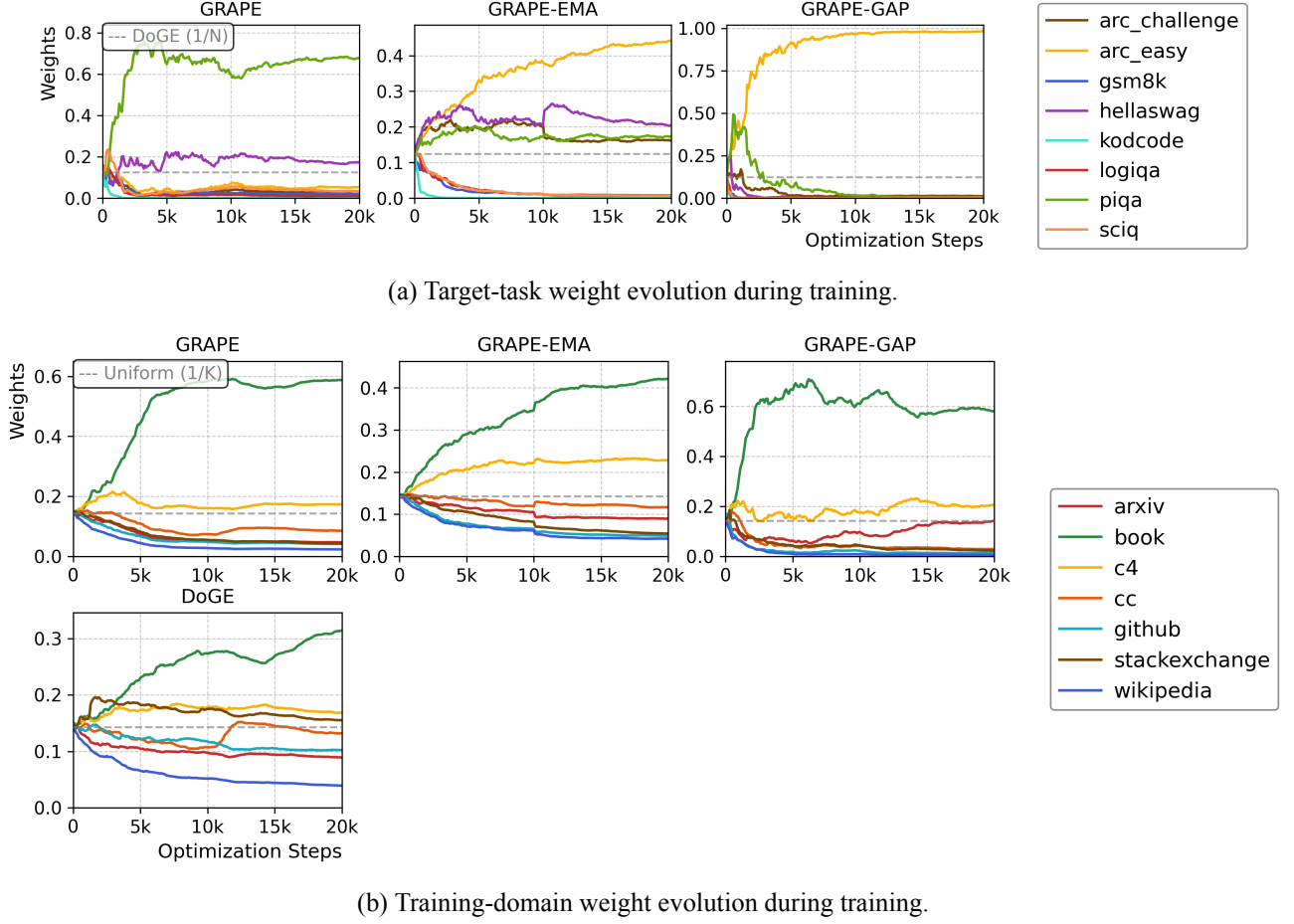


Figure A.16: Target Tasks Set {ARC-C, ARC-E, GSM8K, HellaSwag, KodCode, LogiQA, PIQA, SciQ} : weight dynamics. Top—target-task weights z ; bottom—training-domain weights α . Larger weights indicate greater emphasis during training.

T8	MathQA	COPA	MMLU	OpenBookQA	RACE	WinoGrande	Avg
Uniform	<u>20.97</u>	<u>56.00</u>	25.28	<u>13.20</u>	23.73	51.30	<u>32.53</u>
DoGE	20.60	54.00	24.66	12.80	24.78	49.25	31.02
GRAPE	20.10	61.00	<u>25.36</u>	12.40	25.45	<u>51.54</u>	32.64
GRAPE-GAP	20.70	55.00	25.45	14.00	<u>25.36</u>	51.85	32.06
GRAPE-EMA	21.20	<u>56.00</u>	24.41	12.80	25.17	<u>51.54</u>	31.85

Table A.19: Target Tasks Set {ARC-C, ARC-E, GSM8K, HellaSwag, KodCode, LogiQA, PIQA, SciQ} : 5-shot accuracy $\uparrow$ on held-out tasks (%).

Task combination T9: ARC-C, ARC-E, HellaSwag, LogiQA, PIQA, SciQ

Method	ARC-C	ARC-E	HellaSwag	LogiQA	PIQA	SciQ	Avg
Log Perplexity							
Uniform	4.57	4.66	4.34	3.83	4.55	4.02	4.33
DoGE	4.35	4.51	<u>4.09</u>	<u>3.66</u>	4.34	3.83	4.13
GRAPE	<u>4.05</u>	<u>4.08</u>	4.04	3.54	4.19	<u>3.23</u>	3.86
GRAPE-GAP	4.03	4.05	<u>4.11</u>	3.65	4.21	3.21	<u>3.88</u>
GRAPE-EMA	4.06	4.09	4.10	<u>3.60</u>	<u>4.21</u>	3.23	<u>3.88</u>
5-shot exact-match accuracy scores (%)							
Uniform	21.50	32.45	26.44	26.42	55.50	52.40	35.79
DoGE	21.76	33.59	26.28	30.11	57.73	56.80	<u>37.73</u>
GRAPE	<u>22.87</u>	34.39	26.71	<u>27.96</u>	<u>58.16</u>	59.50	38.27
GRAPE-GAP	23.29	33.50	26.19	26.73	57.56	56.80	37.35
GRAPE-EMA	22.44	<u>34.68</u>	<u>26.69</u>	26.11	58.87	<u>57.30</u>	37.68

Table A.20: Target Tasks Set {ARC-C, ARC-E, HellaSwag, LogiQA, PIQA, SciQ} : log-perplexity ↓ and 5-shot exact-match accuracy ↑ at 0.66B training tokens.

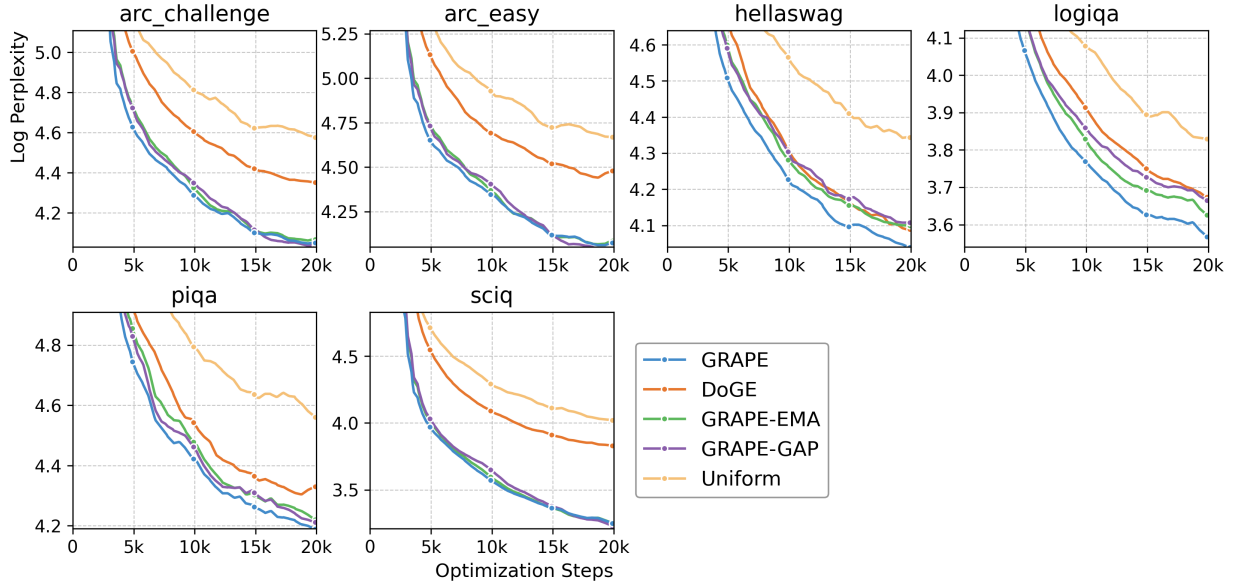


Figure A.17: Target Tasks Set {ARC-C, ARC-E, HellaSwag, LogiQA, PIQA, SciQ} : log-perplexity on target tasks across methods (20k steps, 0.66B tokens).

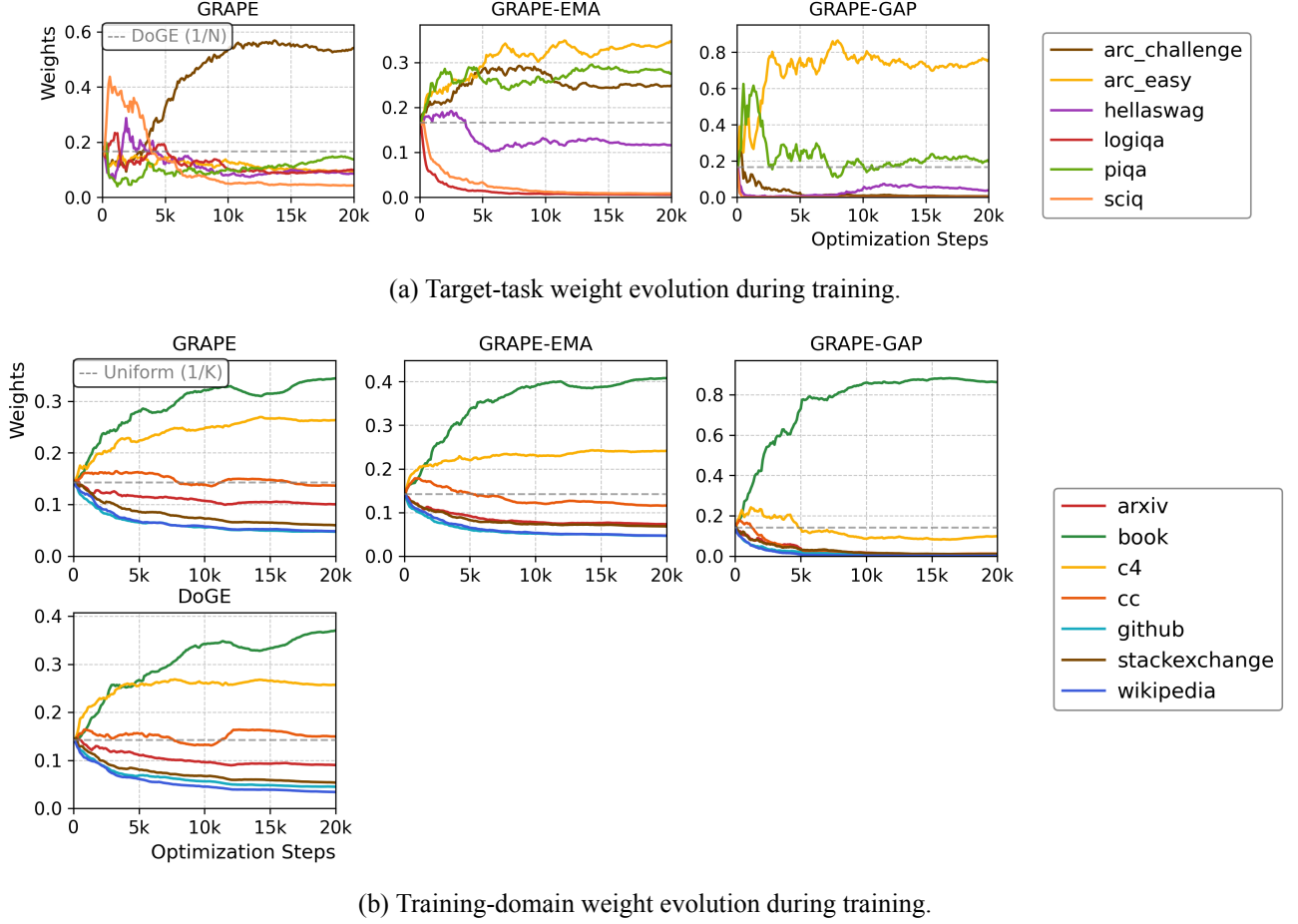


Figure A.18: Target Tasks Set $\{\text{ARC-C, ARC-E, HellaSwag, LogiQA, PIQA, SciQ}\}$: weight dynamics. Top—target-task weights z ; bottom—training-domain weights α . Larger weights indicate greater emphasis during training.

T9	COPA	MMLU	OpenBookQA	RACE	WinoGrande	Avg
Uniform	56.00	<u>25.28</u>	13.20	23.73	51.30	32.53
DoGE	57.00	24.68	13.60	<u>25.07</u>	<u>52.33</u>	34.94
GRAPE	<u>59.00</u>	24.38	<u>14.40</u>	25.93	51.93	<u>35.13</u>
GRAPE-GAP	63.00	25.02	16.00	24.88	53.04	36.19
GRAPE-EMA	56.00	25.94	13.60	24.69	<u>52.33</u>	34.91

Table A.21: Target Tasks Set $\{\text{ARC-C, ARC-E, HellaSwag, LogiQA, PIQA, SciQ}\}$: 5-shot accuracy $\uparrow$ on held-out tasks (%).

Task combination T10: ARC-C, ARC-E, HellaSwag, LogiQA, MathQA, MedQA, PIQA, SciQ

Method	ARC-C	ARC-E	HellaSwag	LogiQA	MathQA	MedQA	PIQA	SciQ	Avg
Log Perplexity									
Uniform	4.83	4.92	4.56	4.08	4.88	4.60	4.77	4.30	4.63
DoGE	4.63	4.72	4.39	3.90	4.82	4.36	4.60	4.13	4.44
GRAPE	<u>4.26</u>	<u>4.37</u>	<u>4.15</u>	<u>3.58</u>	<u>4.52</u>	<u>2.58</u>	<u>4.35</u>	<u>3.76</u>	<u>3.95</u>
GRAPE-GAP	4.14	4.27	4.04	3.54	4.55	2.42	4.21	3.69	3.86
GRAPE-EMA	<u>4.21</u>	<u>4.34</u>	<u>4.10</u>	3.59	4.51	<u>2.45</u>	<u>4.33</u>	<u>3.75</u>	<u>3.91</u>
5-shot exact-match accuracy scores (%)									
Uniform	21.50	32.45	26.44	26.42	20.97	24.59	55.50	52.40	32.41
DoGE	22.87	31.57	<u>26.50</u>	26.88	19.60	21.60	48.60	56.31	31.74
GRAPE	21.76	33.80	26.56	27.34	20.20	21.90	55.60	<u>58.16</u>	33.16
GRAPE-GAP	<u>22.53</u>	34.30	25.79	30.11	20.40	<u>22.50</u>	<u>55.50</u>	59.19	33.79
GRAPE-EMA	21.42	<u>33.92</u>	26.35	<u>28.88</u>	<u>20.80</u>	22.20	55.00	57.56	<u>33.26</u>

Table A.22: Target Tasks Set {ARC-C, ARC-E, HellaSwag, LogiQA, MathQA, MedQA, PIQA, SciQ} : log-perplexity ↓ and 5-shot exact-match accuracy ↑ at 0.66B training tokens.

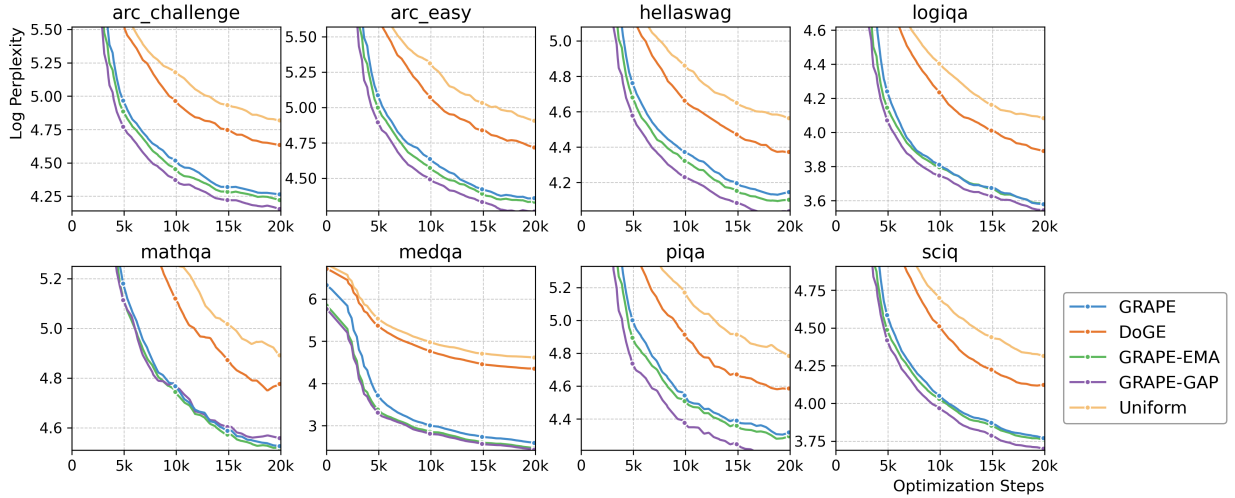


Figure A.19: Target Tasks Set {ARC-C, ARC-E, HellaSwag, LogiQA, MathQA, MedQA, PIQA, SciQ} : log-perplexity on target tasks across methods (20k steps, 0.66B tokens).

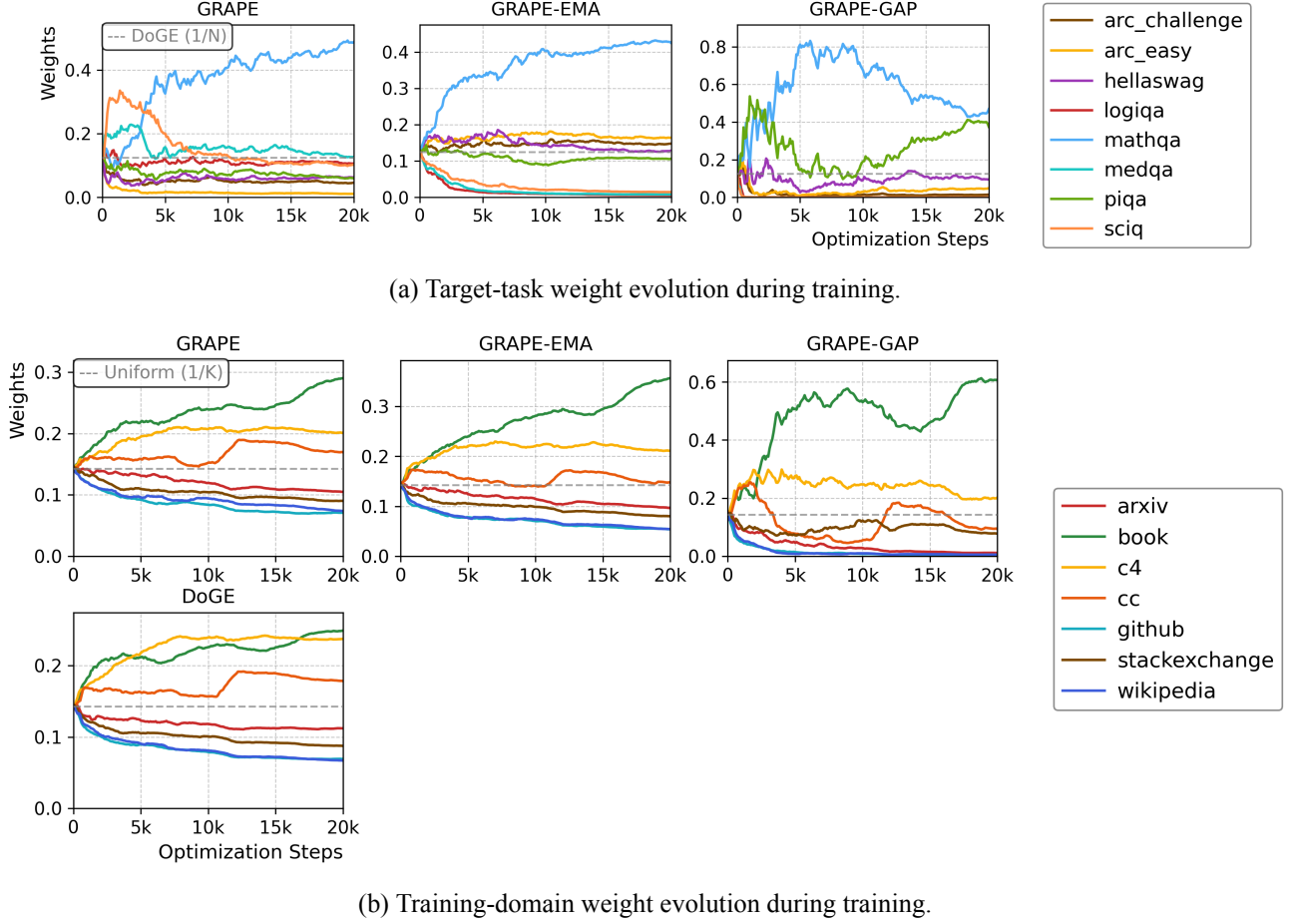


Figure A.20: Target Tasks Set {ARC-C, ARC-E, HellaSwag, LogiQA, MathQA, MedQA, PIQA, SciQ} : weight dynamics. Top—target-task weights z ; bottom—training-domain weights α . Larger weights indicate greater emphasis during training.

T10	COPA	MMLU	OpenBookQA	RACE	WinoGrande	Avg
Uniform	56.00	<u>25.28</u>	13.20	23.73	<u>51.30</u>	33.92
DoGE	56.00	24.93	12.80	23.92	49.64	33.46
GRAPE	57.00	25.22	14.80	<u>24.40</u>	51.14	<u>34.51</u>
GRAPE-GAP	63.00	24.60	13.40	26.03	50.36	35.48
GRAPE-EMA	<u>59.00</u>	25.40	<u>13.80</u>	23.64	49.64	34.30

Table A.23: Target Tasks Set {ARC-C, ARC-E, HellaSwag, LogiQA, MathQA, MedQA, PIQA, SciQ} : 5-shot accuracy $\uparrow$ on held-out tasks (%).

Task combination T11: ARC–C, ARC–E, HellaSwag, LogiQA, MathQA, MedQA

Method	ARC–C	ARC–E	HellaSwag	LogiQA	MathQA	MedQA	Avg
Log Perplexity							
Uniform	4.83	4.92	4.56	4.08	4.88	4.60	4.65
DoGE	4.57	4.68	4.37	3.88	4.74	4.28	4.42
GRAPE	<u>4.25</u>	<u>4.37</u>	<u>4.11</u>	<u>3.57</u>	4.52	<u>2.61</u>	<u>3.91</u>
GRAPE–GAP	4.15	4.27	3.97	3.50	<u>4.53</u>	2.45	3.81
GRAPE–EMA	<u>4.22</u>	<u>4.33</u>	4.20	3.63	4.52	<u>2.51</u>	<u>3.90</u>
5-shot exact-match accuracy scores (%)							
Uniform	21.50	32.45	26.44	26.42	20.97	24.59	25.40
DoGE	21.16	32.15	<u>26.58</u>	<u>27.96</u>	19.40	<u>22.80</u>	25.01
GRAPE	<u>21.84</u>	34.89	27.01	27.80	<u>20.70</u>	21.60	25.47
GRAPE–GAP	21.67	<u>34.39</u>	26.57	28.26	21.30	22.00	<u>25.70</u>
GRAPE–EMA	22.70	34.13	26.39	27.04	20.50	23.70	25.74

Table A.24: Target Tasks Set {ARC–C, ARC–E, HellaSwag, LogiQA, MathQA, MedQA} : log-perplexity ↓ and 5-shot exact-match accuracy ↑ at 0.66B training tokens.

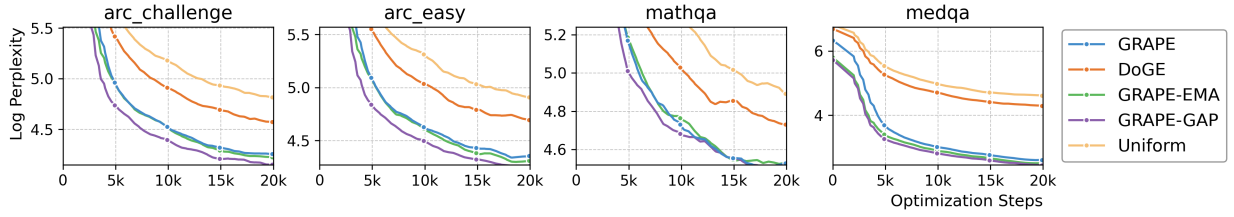


Figure A.21: Target Tasks Set {ARC–C, ARC–E, HellaSwag, LogiQA, MathQA, MedQA} : log-perplexity on target tasks across methods (20k steps, 0.66B tokens).

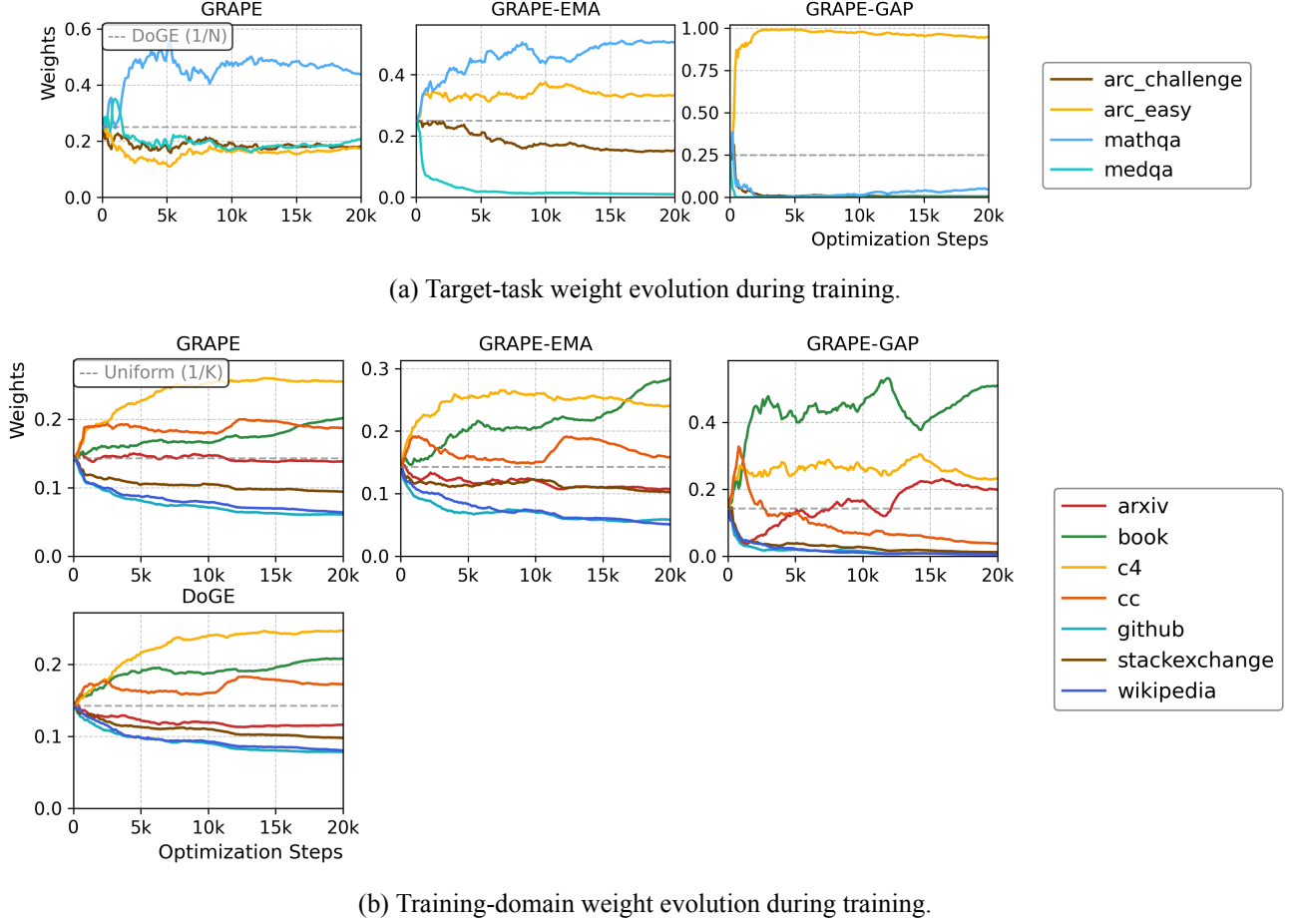


Figure A.22: Target Tasks Set $\{\text{ARC-C, ARC-E, HellaSwag, LogiQA, MathQA, MedQA}\}$: weight dynamics. Top—target-task weights z ; bottom—training-domain weights α . Larger weights indicate greater emphasis during training.

T11	SciQ	PIQA	MathQA	MedQA	COPA	MMLU	OpenBookQA	RACE	WinoGrande	Avg
Uniform	52.40	55.50	20.97	24.59	56.00	25.28	13.20	23.73	51.30	32.53
DoGE	52.40	56.31	19.40	<u>22.80</u>	<u>56.00</u>	25.77	14.00	24.02	49.41	35.57
GRAPE	<u>56.60</u>	<u>58.22</u>	<u>20.70</u>	21.60	60.00	24.37	13.60	25.45	51.30	<u>36.87</u>
GRAPE-GAP	55.40	58.43	21.30	22.00	<u>56.00</u>	<u>25.68</u>	<u>15.00</u>	<u>25.36</u>	52.72	36.88
GRAPE-EMA	58.90	57.40	20.50	23.70	54.00	24.94	15.20	24.59	<u>51.54</u>	36.75

Table A.25: Target Tasks Set $\{\text{ARC-C, ARC-E, HellaSwag, LogiQA, MathQA, MedQA}\}$: 5-shot accuracy $\uparrow$ on held-out tasks (%).

Task combination T12: HellaSwag, LogiQA, MathQA, MedQA

Method	HellaSwag	LogiQA	MathQA	MedQA	Avg
Log Perplexity					
Uniform	4.56	4.08	4.88	4.60	4.53
DoGE	4.37	3.88	4.76	4.32	4.33
GRAPE	<u>4.11</u>	<u>3.57</u>	<u>4.51</u>	<u>2.61</u>	<u>3.70</u>
GRAPE-GAP	3.97	3.50	4.50	2.40	3.59
GRAPE-EMA	4.20	3.63	4.56	<u>2.54</u>	3.73
5-shot exact-match accuracy scores (%)					
Uniform	26.44	26.42	20.97	24.59	24.61
DoGE	26.00	27.50	19.90	21.80	23.80
GRAPE	26.82	<u>28.11</u>	<u>20.30</u>	22.50	<u>24.44</u>
GRAPE-GAP	<u>26.67</u>	26.27	20.20	<u>23.20</u>	24.09
GRAPE-EMA	26.51	28.42	19.50	22.10	24.13

Table A.26: Target Tasks Set {HellaSwag, LogiQA, MathQA, MedQA} : log-perplexity ↓ and 5-shot exact-match accuracy ↑ at 0.66B training tokens.

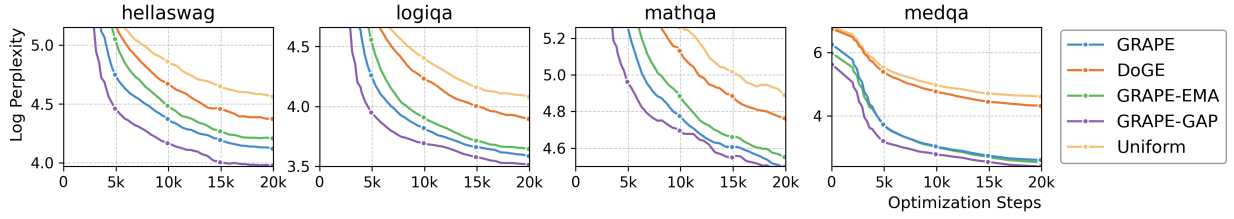


Figure A.23: Target Tasks Set {HellaSwag, LogiQA, MathQA, MedQA} : log-perplexity on target tasks across methods (20k steps, 0.66B tokens).

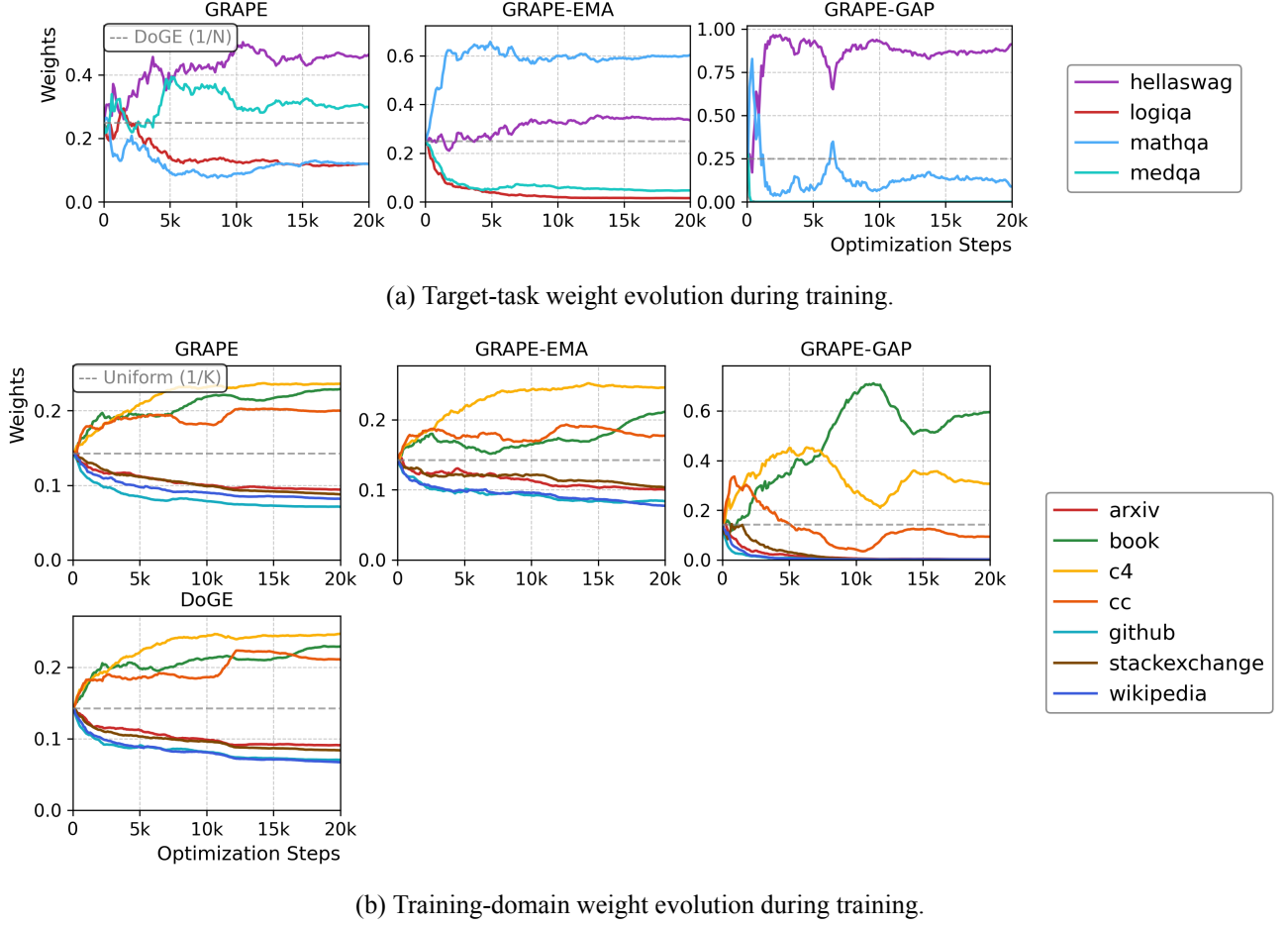


Figure A.24: Target Tasks Set {HellaSwag, LogiQA, MathQA, MedQA} : weight dynamics. Top—target-task weights z ; bottom—training-domain weights α . Larger weights indicate greater emphasis during training.

T12	ARC-C	ARC-E	SciQ	PIQA	COPA	MMLU	OpenBookQA	RACE	WinoGrande	Avg
Uniform	21.50	32.45	52.40	55.50	56.00	25.28	13.20	23.73	51.30	33.15
DoGE	21.08	31.52	48.50	56.47	56.00	24.15	14.00	24.40	51.85	36.44
GRAPE	22.10	34.13	56.50	57.73	54.00	25.05	13.60	25.26	52.17	40.06
GRAPE-GAP	21.84	35.02	57.00	58.92	63.00	24.62	13.40	25.36	52.17	40.15
GRAPE-EMA	21.16	32.28	54.30	58.11	59.00	24.04	13.60	23.92	52.64	39.78

Table A.27: Target Tasks Set {HellaSwag, LogiQA, MathQA, MedQA} : 5-shot accuracy $\uparrow$ on held-out tasks (%).

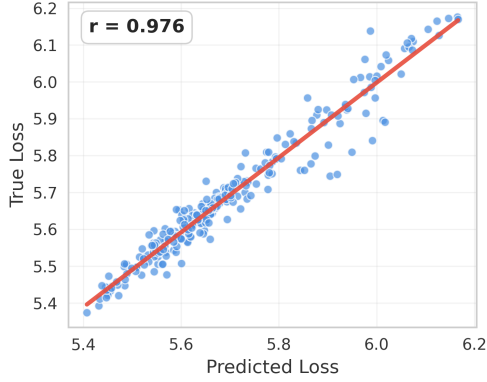
Appendix B

Implementation Details and Baseline Specifications

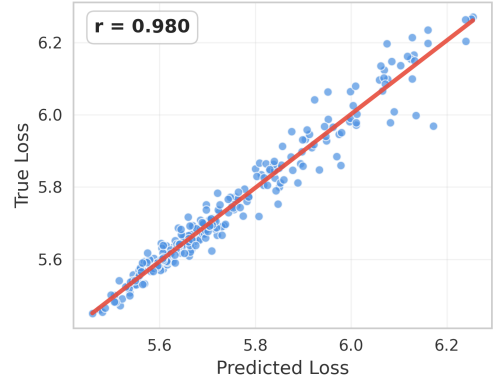
B.1 Implementation Details for RegMix Baseline

We implement the RegMix baseline method [54] to predict optimal domain mixtures for target task performance. We train 768 small proxy models (125M parameters) with randomly sampled domain mixture configurations: 512 for training and 256 for testing. For each proxy model, we record the domain mixture proportions (20 domains) and evaluate validation losses on target tasks after training. We then use LightGBM gradient boosting to fit a regression model that predicts average target task loss from domain mixture weights. Specifically, we compute the mean validation loss across all target tasks $\bar{\mathcal{L}}_{\text{target}} = \frac{1}{|T|} \sum_{t \in T} \mathcal{L}_t$, where T is the set of target tasks. The gradient boosting model is trained on the 512 training configurations with GBDT boosting, L1/L2 loss metrics, 1000 iterations, learning rate of 0.01, and early stopping with 3 rounds. This non-linear approach captures complex interactions between domain mixtures and task performance beyond simple linear combinations.

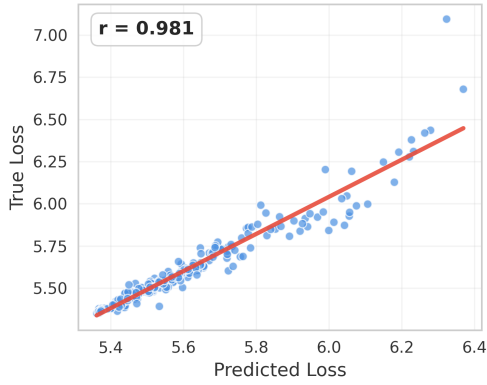
The RegMix regression achieves strong predictive performance across all three experimental settings, with Spearman correlations consistently above 0.94. Each figure in Figure B.1 displays a scatter plot of predicted versus true average validation losses across the 256 held-out test configurations. Points near the diagonal ($y = x$) indicate accurate predictions, while deviations reveal where the regression model fails to capture the relationship between domain mixtures and target performance. The dashed red line shows the linear fit, and the Spearman correlation coefficient (displayed as “Corr”) quantifies the monotonic relationship between predictions and ground truth, with values near 1.0 indicating strong predictive power. Each point represents one of the 256 test proxy models, each trained with a different domain mixture. We show results for two target configurations: the base 6-task setup (ARC-E, ARC-C, HellaSwag, SciQ, PIQA, LogiQA) and an expanded 8-task setup that additionally includes MathQA and MedQA to test mathematical and medical reasoning. The relatively tight clustering of points around the diagonal suggests that RegMix provides reliable predictions for identifying data mixtures, though some prediction error remains as evidenced by the scatter around the fit line. These results indicate that the gradient boosting model captures most of the variance in the relationship between domain mixtures and target task performance, validating the use of RegMix as a baseline for data mixture optimization.



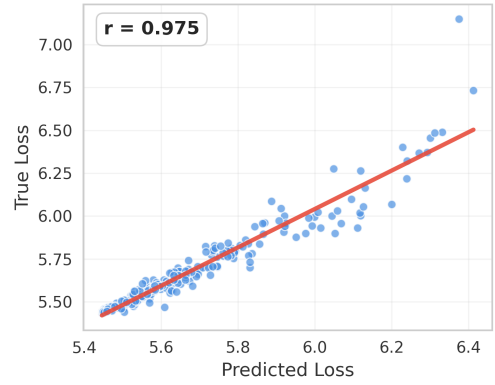
(a) Climblab domains, 6 targets: ARC-E, ARC-C, Hel-laSwag, SciQ, PIQA, LogiQA. Spearman correlation: 0.976.



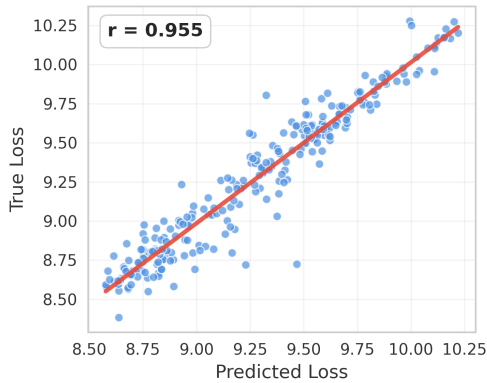
(b) Climblab domains, 8 targets: 6 tasks + MathQA, MedQA. Spearman correlation: 0.980.



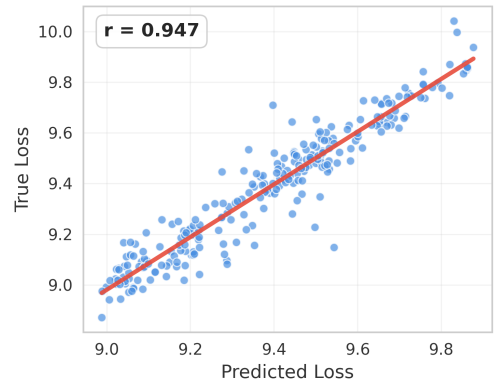
(c) SlimPajama domains (ArXiv, Books, C4, GitHub, StackExchange, Wikipedia), 6 targets. Spearman correlation: 0.982.



(d) SlimPajama domains, 8 targets. Spearman correlation: 0.975.



(e) Wiki40B source languages, 4 targets: Catalan, Danish, Romanian, Ukrainian. Spearman correlation: 0.955.



(f) Wiki40B source languages, 8 targets: 4 targets + Polish, Portuguese, Turkish, Dutch. Spearman correlation: 0.947.

Figure B.1: RegMix regression fits across three experimental settings. Each scatter plot shows predicted vs. true validation loss across 256 test proxy models, with Spearman correlation indicating prediction quality.

B.2 Training Configuration for GRAPE

Hardware and compute. All experiments are conducted on $2 \times$ H100 80GB GPUs.

Optimization. Table B.1 summarizes the training configuration for the Small and Large models. **Cosine decay** is used for the Small model for fast, smooth convergence in shorter runs. **WSD schedule**¹ is used for the Large model to stabilize optimization by preventing premature decay.

	Small (125M)	Large (774M)
Training tokens	1.6B	7.8B (ClimbLab); 6.8B (SlimPajama)
Optimizer	AdamW (default hyperparameters)	AdamW (default hyperparameters)
Batch size	64	128
Initial LR	1.5×10^{-4}	1.5×10^{-4}
Final LR	1.5×10^{-5}	1.5×10^{-5}
LR schedule	Cosine decay	WSD schedule

Table B.1: Training configuration for Small and Large models.

GRAPE hyperparameters. Task weights z and domain weights α are updated every $\Delta T_z = \Delta T_\alpha = 100$ steps. Regularization coefficients control update responsiveness: task weights use $\gamma/\mu_z = 10$ for responsive adaptation to performance gaps, while domain weights use $\gamma/\mu_\alpha = 1.5$ for more conservative mixture adjustments. This asymmetry ensures the algorithm remains sensitive to lagging tasks while maintaining stable training dynamics.

Supervised fine-tuning. For fine-tuning experiments on ClimbLab, we perform supervised fine-tuning on each target task. All pretrained models we fine-tune (Uniform, DoGE, GRAPE-pretrained models) use identical data and optimization settings to ensure controlled comparison: 5 epochs with an effective batch size of 64, a learning rate of 1×10^{-5} , and a linear scheduler with 30% warmup followed by decay to 3×10^{-6} . The best checkpoint is selected by validation performance per task.

¹**WSD**: Warmup-Static-Decay; holds learning rate constant for most of training, then linearly decays over the final 20% of steps [40].

Appendix C

Theoretical Derivation and Analysis

C.1 Derivation of GRAPE

We derive the multiplicative (mirror-descent/ascent) updates for the task weights $\mathbf{z}$ and domain weights $\boldsymbol{\alpha}$ that were stated in Section 3.1, starting directly from the regularized saddle-point objective in Eq. (3.4) and using the RoI alignment proxy of Eq. (3.3). Throughout, we keep the notation of Sections 3–3.1: ℓ_n denotes the (strictly positive) loss on target task T_n , $\mathbf{g}_k$ the per-domain training gradient, and $\mathbf{d}_t = \sum_{k=1}^K a_k^{(t)} \mathbf{g}_k(\boldsymbol{\theta}_t)$ the mixture gradient.

Regularized value function. Substituting the alignment form of the expected RoI into Eq. (3.4) yields the time- t value function

$$\max_{\boldsymbol{\alpha} \in \Delta_K} \min_{\mathbf{z} \in \Delta_N} \underbrace{\gamma_t \sum_{k=1}^K \sum_{n=1}^N a_k z_n \langle \nabla_{\boldsymbol{\theta}} \log \ell_n(\boldsymbol{\theta}_t), \mathbf{g}_k(\boldsymbol{\theta}_t) \rangle}_{\text{RoI alignment term}} - \mu_{\alpha} D_{\text{KL}}(\boldsymbol{\alpha} \| \boldsymbol{\alpha}^{(t-1)}) + \mu_z D_{\text{KL}}(\mathbf{z} \| \mathbf{z}^{(t-1)}), \quad (\text{C.1})$$

where $D_{\text{KL}}(p \| q) = \sum_i p_i \log \frac{p_i}{q_i}$. Define the alignment matrix entries

$$f_{nk}(\boldsymbol{\theta}_t) := \langle \nabla_{\boldsymbol{\theta}} \log \ell_n(\boldsymbol{\theta}_t), \mathbf{g}_k(\boldsymbol{\theta}_t) \rangle. \quad (\text{C.2})$$

Then the bilinear part of (C.1) becomes $\gamma_t \boldsymbol{\alpha}^{\top} F \mathbf{z}$ with $(F)_{kn} = f_{nk}$.

Inner minimization over $\mathbf{z}$

Fix $\boldsymbol{\alpha}$. Let $\phi_n := \sum_{k=1}^K a_k f_{nk}$ denote the (current) alignment between task n and the mixture direction $\mathbf{d}_t$. The inner problem reads

$$\min_{\mathbf{z} \in \Delta_N} \gamma_t \sum_{n=1}^N z_n \phi_n + \mu_z D_{\text{KL}}(\mathbf{z} \| \mathbf{z}^{(t-1)}). \quad (\text{C.3})$$

Introducing a Lagrange multiplier λ for $\sum_n z_n = 1$ and differentiating coordinate-wise gives

$$0 = \partial_{z_n} \mathcal{L} = \gamma_t \phi_n + \mu_z \left(1 + \log \frac{z_n}{z_n^{(t-1)}} \right) + \lambda.$$

Solving for z_n and normalizing to the simplex yields the multiplicative-weights update

$$z_n^{(t)} = \frac{z_n^{(t-1)} \exp\left(-\frac{\gamma_t}{\mu_z} \phi_n\right)}{\sum_{m=1}^N z_m^{(t-1)} \exp\left(-\frac{\gamma_t}{\mu_z} \phi_m\right)} = \frac{z_n^{(t-1)} \exp\left(-\frac{\gamma_t}{\mu_z} \sum_{k=1}^K a_k^{(t-1)} f_{nk}(\boldsymbol{\theta}_t)\right)}{\sum_{m=1}^N z_m^{(t-1)} \exp\left(-\frac{\gamma_t}{\mu_z} \sum_{k=1}^K a_k^{(t-1)} f_{mk}(\boldsymbol{\theta}_t)\right)}. \quad (\text{C.4})$$

Sign convention. The negative sign in the exponent reflects the inner minimization over $\mathbf{z}$.

Outer maximization over α

Plugging $\mathbf{z}^{(t)}$ into (C.1), the outer problem becomes

$$\max_{\alpha \in \Delta_K} \gamma_t \sum_{k=1}^K a_k \sum_{n=1}^N z_n^{(t)} f_{nk} - \mu_\alpha D_{\text{KL}}(\alpha \| \alpha^{(t-1)}). \quad (\text{C.5})$$

With a multiplier ν for $\sum_k a_k = 1$, the first-order condition per coordinate is

$$0 = \partial_{a_k} \mathcal{L} = \gamma_t \sum_{n=1}^N z_n^{(t)} f_{nk} - \mu_\alpha \left(1 + \log \frac{a_k}{a_k^{(t-1)}} \right) + \nu.$$

Solving and renormalizing gives the exponentiated-gradient ascent step

$$a_k^{(t)} = \frac{a_k^{(t-1)} \exp\left(\frac{\gamma_t}{\mu_\alpha} \sum_{n=1}^N z_n^{(t)} f_{nk}(\theta_t)\right)}{\sum_{j=1}^K a_j^{(t-1)} \exp\left(\frac{\gamma_t}{\mu_\alpha} \sum_{n=1}^N z_n^{(t)} f_{nj}(\theta_t)\right)}. \quad (\text{C.6})$$

C.2 Theoretical Analysis

We study convergence of the θ -updates driven by the *adaptive* mixture direction $\mathbf{d}_t = \sum_{j=1}^K \alpha_j^{(t)} \mathbf{g}_j(\theta_t)$, where $(\alpha^{(t)}, \mathbf{z}^{(t)})$ are given by (C.4)–(C.6).

Assumptions

1. **Smoothness and bounded gradients.** Each task loss l_i is L -smooth and its (stochastic) gradient has bounded second moment: $\mathbb{E} \|\nabla l_i(\theta)\|^2 \leq \mathcal{G}^2$. Consequently, there exists $G > 0$ such that $\mathbb{E} \|\mathbf{d}_t\|^2 \leq G^2$ for all t .
2. **Stepsizes.** $0 < \gamma_t \leq 1/L$. Define $\eta_{\mathbf{z}} = \gamma_t / \mu_{\mathbf{z}}$ and $\eta_\alpha = \gamma_t / \mu_\alpha$ with $\mu_{\mathbf{z}}, \mu_\alpha > 0$.
3. **Reference objective.** Let $\mathcal{L}(\theta) := \max_{i \in [N]} l_i(\theta)$ and $\mathcal{L}^* := \min_\theta \mathcal{L}(\theta)$.

C.2.1 Proof of the Convergence Theorem

Theorem C.1 (Convergence of GRAPE). *Let the loss functions $l_n(\theta)$ be L -smooth for all $n \in [N]$ and the norm of stochastic gradients be upper-bounded by $\mathcal{G}$. If the learning rate γ_t satisfies $\gamma_t \leq \frac{1}{L}$ and the regularization parameters $\mu_\alpha, \mu_{\mathbf{z}}$ are chosen such that $\mu_\alpha > 0$ and $\mu_{\mathbf{z}} > 0$, then the GRAPE algorithm with update rules given by the above equations converges to a neighborhood of the Pareto optimal solution at a rate of $\mathcal{O}(1/T)$, where T is the number of training iterations. Specifically, for any $\epsilon > 0$, there exists T_0 such that for all $T > T_0$:*

$$\min_{t \in [T]} \left\{ \max_{n \in [N]} \mathbb{E}[l_n(\theta_t)] - \min_\theta \max_{n \in [N]} l_n(\theta) \right\} \leq \epsilon$$

Proof. We establish the convergence of GRAPE by analyzing the dynamics of the minimax optimization problem and demonstrating that the algorithm makes consistent progress toward Pareto optimal solutions while balancing performance across tasks.

First, we analyze the task weight updates. Given that $\mathbf{z}$ is updated according to:

$$\mathbf{z}_n^t = \mathbf{z}_n^{t-1} \cdot \exp\left(-\frac{\gamma_t}{\mu_{\mathbf{z}}} \langle \mathbf{d}_t, \nabla_\theta \log l_n(\theta_t) \rangle\right) \quad (\text{C.7})$$

The task weights adjust to increase focus on tasks with slower improvement rates. For any task i where $\langle \mathbf{d}_t, \nabla_{\theta} \log l_n(\theta_t) \rangle < 0$ (indicating poor alignment between the current update direction and task gradient), z_n^t increases proportionally. This ensures more attention to tasks making less progress.

For each iteration t , let $\mathcal{L}(\theta_t) = \max_{n \in [N]} l_n(\theta_t)$ represent our worst-case objective. Let $n_t = \arg \max_{n \in [N]} l_n(\theta_t)$ be the index of the task with highest loss at iteration t . Due to the exponential update rule, as t increases, $z_{n_t}^t$ approaches 1, while other weights approach 0, directing optimization toward the current worst-case task.

Considering the domain weight updates:

$$\alpha_k^t = \alpha_k^{t-1} \cdot \exp \left(\gamma_t \sum_{n=1}^N \frac{z_n^{t-1}}{\mu_{\alpha}} \langle \mathbf{g}_k(\theta_t), \nabla_{\theta} \log l_n(\theta_t) \rangle \right)$$

These updates increase weights for domains whose gradients align well with the task gradients weighted by z . Since z concentrates on the worst-performing tasks, α increasingly favors domains beneficial to these tasks.

Given the L -smoothness of the loss functions, we can upper-bound the progress on the worst-case objective:

$$\mathcal{L}(\theta_{t+1}) - \mathcal{L}(\theta_t) \leq \langle \nabla_{\theta} l_{n_t}(\theta_t), -\gamma_t \mathbf{d}_t \rangle + \frac{L\gamma_t^2}{2} \|\mathbf{d}_t\|^2$$

Let $\mathcal{Q}_t = \langle \nabla_{\theta} l_{n_t}(\theta_t), \mathbf{d}_t \rangle$. Due to our update rules for z and α , as t increases, $\mathcal{Q}_t$ becomes increasingly positive (as domain weights shift toward domains beneficial for the worst-performing task). With the condition $\gamma_t \leq \frac{1}{L}$, we have:

$$\mathcal{L}(\theta_{t+1}) - \mathcal{L}(\theta_t) \leq -\gamma_t \mathcal{Q}_t + \frac{\gamma_t}{2} \|\mathbf{d}_t\|^2$$

Since the domain weights are probability distributions and gradients are bounded (due to L -smoothness), $\|\mathbf{d}_t\|$ is bounded by some constant G . Therefore:

$$\mathcal{L}(\theta_{t+1}) - \mathcal{L}(\theta_t) \leq -\gamma_t \mathcal{Q}_t + \frac{\gamma_t G^2}{2}$$

As optimization progresses and $\mathcal{Q}_t$ increases due to our reweighting strategy, we eventually reach a point where $\mathcal{Q}_t > \frac{G^2}{2}$, ensuring consistent progress on the worst-case objective.

Summing over iterations and applying the μ -strong convexity, we obtain:

$$\sum_{t=1}^T (\mathcal{L}(\theta_{t+1}) - \mathcal{L}(\theta_t)) \leq \sum_{t=1}^T \left(-\gamma_t \mathcal{Q}_t + \frac{\gamma_t G^2}{2} \right)$$

This implies:

$$\mathcal{L}(\theta_{T+1}) - \mathcal{L}(\theta_1) \leq \sum_{t=1}^T \gamma_t \left(-\mathcal{Q}_t + \frac{G^2}{2} \right)$$

Given our reweighting strategy ensures $\mathcal{Q}_t > \frac{G^2}{2}$ after sufficient iterations, this difference becomes negative, establishing convergence.

For the convergence rate, with constant step size $\gamma_t = \frac{1}{L}$, we can show:

$$\min_{t \in [T]} \{\mathcal{L}(\theta_t) - \mathcal{L}^*\} \leq \frac{L\|\theta_1 - \theta^*\|^2}{2T} + \frac{G^2}{2L} - \frac{1}{T} \sum_{t=1}^T \mathcal{Q}_t$$

Where $\mathcal{L}^* = \min_{\theta} \max_{n \in [N]} l_n(\theta)$ and θ^* is the corresponding optimal parameter. As $T \rightarrow \infty$, this bound approaches $\frac{G^2}{2L} - \frac{\bar{\mathcal{Q}}}{L}$, where $\bar{\mathcal{Q}}$ is the average alignment.

For the variance reduction claim, let $\sigma_t^2 = \text{Var}_{n \in [N]}(l_n(\theta_t))$. The exponential reweighting of tasks directly targets tasks with higher losses, ensuring faster progress on these tasks compared to better-performing ones. This leads to a natural decrease in the spread of task performances.

Specifically, for any pair of tasks i, j with $l_i(\boldsymbol{\theta}_t) > l_j(\boldsymbol{\theta}_t)$, the ratio of their weights $\frac{z_i^t}{z_j^t}$ increases over time, directing more optimization effort toward task i . Consequently, l_i decreases faster than l_j , reducing the performance gap. This mechanism ensures that $\sigma_{t+1}^2 < \sigma_t^2$ until all tasks reach similar performance levels, at which point the variance stabilizes at a minimal value.

Therefore, GRAPE converges to a neighborhood of the Pareto optimal solution at a rate of $\mathcal{O}(1/T)$. $\square$

C.2.2 Proof of the Variance Reduction Theorem

Theorem C.2 (Monotonic Variance Reduction of Task Performance). *Let $\sigma_t^2 = \text{Var}_{n \in [N]}(l_n(\boldsymbol{\theta}_t))$ denote the variance of task performances at iteration t . Let the loss functions $l_n(\boldsymbol{\theta})$ be L -smooth for all $n \in [N]$ and the norm of stochastic gradients be upper-bounded by $\mathcal{G}$, and assuming the task losses are L -smooth and μ -strongly convex, the variance decreases monotonically until reaching a minimal basin, i.e., $\sigma_{t+1}^2 \leq \sigma_t^2$ for all $t \geq T_0$ for some finite T_0 .*

Proof. According to the definition of Rate-of-Improvement (RoI) for task i at iteration t :

$$r_i^{(t)} = \frac{l_i(\boldsymbol{\theta}_t) - l_i(\boldsymbol{\theta}_{t+1})}{l_i(\boldsymbol{\theta}_t)}$$

For any task i , by the L -smoothness assumption, we can bound the change in loss:

$$l_i(\boldsymbol{\theta}_{t+1}) - l_i(\boldsymbol{\theta}_t) \leq -\gamma_t \langle \nabla_{\boldsymbol{\theta}} l_i(\boldsymbol{\theta}_t), \mathbf{d}_t \rangle + \frac{L\gamma_t^2}{2} |\mathbf{d}_t|^2$$

Let $\Delta l_i(t) = l_i(\boldsymbol{\theta}_{t+1}) - l_i(\boldsymbol{\theta}_t)$ represent the change in loss for task i .

First, we establish a direct relationship between task weights and improvement rates. At each iteration, GRAPE updates task weights according to:

$$z_i^t \propto z_i^{t-1} \cdot \exp \left(-\frac{\gamma_t}{\mu_z} \langle \mathbf{d}_t, \nabla_{\boldsymbol{\theta}} \log l_i(\boldsymbol{\theta}_t) \rangle \right)$$

it reflecting the principle that tasks with lower RoI receive higher weights in the next iteration. When domain weights are subsequently updated, domains that yield better improvements for these higher-weighted (struggling) tasks are upweighted:

$$\alpha_j^t \propto \alpha_j^{t-1} \cdot \exp \left(\gamma_t \sum_{i=1}^N \frac{z_i^{t-1}}{\mu_\alpha} \langle \mathbf{g}_j(\boldsymbol{\theta}_t), \nabla_{\boldsymbol{\theta}} \log l_i(\boldsymbol{\theta}_t) \rangle \right)$$

This creates a feedback mechanism that specifically targets and improves tasks with lower RoI values. This indicates that tasks with lower RoI experience proportionally greater increases in their improvement rates. We can formalize this in **Lemma 1**:

Lemma 1: There exists a constant $\beta > 0$ such that if $r_i^{(t)} < r_j^{(t)}$ for tasks i and j , then after GRAPE's reweighting mechanism, the expected improvement in the next iteration satisfies: $\mathbb{E}[r_i^{(t+1)}] - \mathbb{E}[r_i^{(t)}] > \mathbb{E}[r_j^{(t+1)}] - \mathbb{E}[r_j^{(t)}] + \beta(r_j^{(t)} - r_i^{(t)})$

We further define the normalized task losses:

$$\hat{l}_i(t) = \frac{l_i(\boldsymbol{\theta}_t) - \min_j l_j(\boldsymbol{\theta}_t)}{\max_j l_j(\boldsymbol{\theta}_t) - \min_j l_j(\boldsymbol{\theta}_t)}$$

We show that the variance of normalized loss $\hat{l}_i(t)$ is proportional to the variance of losses:

Lemma 2 (Variance of Normalized Losses): The variance of normalized losses $\hat{\sigma}_t^2 = \text{Var}_{i \in [N]}(\hat{l}_i(t))$ is directly proportional to the variance of original losses $\sigma_t^2 = \text{Var}_{i \in [N]}(l_i(\boldsymbol{\theta}_t))$.

Proof. We begin with the definition of normalized losses:

$$\hat{l}_i(t) = \frac{l_i(\boldsymbol{\theta}_t) - \min_n l_n(\boldsymbol{\theta}_t)}{\max_n l_n(\boldsymbol{\theta}_t) - \min_n l_n(\boldsymbol{\theta}_t)}$$

Let's denote $a_t = \min_n l_n(\boldsymbol{\theta}_t)$ and $b_t = \max_n l_n(\boldsymbol{\theta}_t)$ for brevity. The normalized loss can be written as:

$$\hat{l}_i(t) = \frac{l_i(\boldsymbol{\theta}_t) - a_t}{b_t - a_t}$$

This is an affine transformation of the original losses. By the properties of variance for affine transformations, for any random variable X and constants c and d , we have:

$$\text{Var}(cX + d) = c^2 \text{Var}(X)$$

In our case, viewing the task losses as samples from a distribution, we have $c = \frac{1}{b_t - a_t}$ and $d = \frac{-a_t}{b_t - a_t}$. Thus, we have,

$$\hat{\sigma}_t^2 = \text{Var}_{i \in [N]}(\hat{l}_i(t)) = \text{Var}_{i \in [N]} \left(\frac{l_i(\boldsymbol{\theta}_t) - a_t}{b_t - a_t} \right) = \frac{1}{(b_t - a_t)^2} \text{Var}_{i \in [N]}(l_i(\boldsymbol{\theta}_t)) = \frac{\sigma_t^2}{(b_t - a_t)^2}$$

Therefore:

$$\hat{\sigma}_t^2 = \frac{\sigma_t^2}{(b_t - a_t)^2}$$

Which establishes a direct proportionality between $\hat{\sigma}_t^2$ and σ_t^2 , with the proportionality constant $\frac{1}{(b_t - a_t)^2}$. $\square$

Given the relationship between task weights and RoI established in **Lemma 1**, tasks with higher normalized losses tend to have lower RoI values and thus receive higher weights in GRAPE. This leads to a correlation between $\hat{l}_i(t)$ and the subsequent change in normalized loss $\Delta \hat{l}_i(t) = \hat{l}_i(t+1) - \hat{l}_i(t)$. Specifically, for any pair of tasks i and j where $\hat{l}_i(t) > \hat{l}_j(t)$, GRAPE's reweighting mechanism ensures:

$$\mathbb{E}[\Delta \hat{l}_i(t)] < \mathbb{E}[\Delta \hat{l}_j(t)]$$

Therefore, we can express the change in variance:

$$\begin{aligned} \hat{\sigma}_{t+1}^2 - \hat{\sigma}_t^2 &= \frac{1}{N} \sum_{i=1}^N (\hat{l}_i(t+1) - \bar{\hat{l}}_{t+1})^2 - \frac{1}{N} \sum_{i=1}^N (\hat{l}_i(t) - \bar{\hat{l}}_t)^2 \\ &= \frac{1}{N} \sum_{i=1}^N [(\hat{l}_i(t) + \Delta \hat{l}_i(t) - \bar{\hat{l}}_t - \Delta \bar{\hat{l}}_t)^2 - (\hat{l}_i(t) - \bar{\hat{l}}_t)^2] \\ &= \hat{\sigma}_{t+1}^2 - \hat{\sigma}_t^2 = \frac{1}{N} \sum_{i=1}^N [2(\hat{l}_i(t) - \bar{\hat{l}}_t)(\Delta \hat{l}_i(t) - \Delta \bar{\hat{l}}_t) + (\Delta \hat{l}_i(t) - \Delta \bar{\hat{l}}_t)^2] \end{aligned}$$

The key term is the correlation between current deviation and improvement. Given that tasks with higher normalized loss experience greater improvements under GRAPE, this correlation term is negative:

$$\frac{1}{N} \sum_{i=1}^N (\hat{l}_i(t) - \bar{\hat{l}}_t)(\Delta \hat{l}_i(t) - \Delta \bar{\hat{l}}_t) \leq -\kappa \hat{\sigma}_t^2$$

for some constant $\kappa > 0$, representing the strength of GRAPE's balancing effect.

The second term, $\frac{1}{N} \sum_{i=1}^N (\Delta \hat{l}_i(t) - \Delta \bar{\hat{l}}_t)^2$, represents the variance of the changes in normalized losses, which

is bounded by $O(\gamma_t^2)$ under the smoothness assumption. Thus:

$$\hat{\sigma}_{t+1}^2 - \hat{\sigma}_t^2 \leq -2\kappa\hat{\sigma}_t^2 + O(\gamma_t^2)$$

For sufficiently small γ_t , this difference becomes negative, establishing that $\hat{\sigma}_{t+1}^2 < \hat{\sigma}_t^2$. Following **Lemma 2**, it indicates that $\sigma_{t+1}^2 < \sigma_t^2$. $\square$

Bibliography

- [1] Alon Albalak, Liangming Pan, Colin Raffel, and William Yang Wang. Efficient online data mixing for language model pre-training, 2023. URL <https://arxiv.org/abs/2312.02406>.
- [2] Alon Albalak, Hila Gonen, Haggai Roitman, and Omer Levy. D4: Improving llm pretraining via document decontamination and deduplication. In *International Conference on Learning Representations (ICLR)*, 2024.
- [3] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav, Joshua Lochner, Caleb Fahlgren, Xuan-Son Nguyen, Clémentine Fourier, Ben Burtenshaw, Hugo Larcher, Haojun Zhao, Cyril Zakka, Mathieu Morlon, Colin Raffel, Leandro von Werra, and Thomas Wolf. Smollm2: When smol goes big – data-centric training of a small language model, 2025. URL <https://arxiv.org/abs/2502.02737>.
- [4] Aida Amini, Saadia Gabriel, Peter Lin, Rik Koncel-Kedziorski, Yejin Choi, and Hannaneh Hajishirzi. Mathqa: Towards interpretable math word problem solving with operation-based formalisms, 2019. URL <https://arxiv.org/abs/1905.13319>.
- [5] A. Author, B. Author, and C. Author. Llm-eval: Unified benchmarking framework for evaluating large language models. *arXiv preprint arXiv:2401.12345*, 2024.
- [6] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*, 2022.
- [7] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. In *arXiv preprint arXiv:2004.05150*, 2020.
- [8] Emily M Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. On the dangers of stochastic parrots: Can language models be too big? *ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2021.
- [9] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning, 2009. ICML 2009.
- [10] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. Piqa: Reasoning about physical commonsense in natural language, 2019. URL <https://arxiv.org/abs/1911.11641>.
- [11] Christian Bohn, Ido Freeman, Hasan Tercan, and Tobias Meisen. Task weighting through gradient projection for multitask learning, 2024. URL <https://arxiv.org/abs/2409.01793>.
- [12] Rishi Bommasani, Drew A Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill, et al. On the opportunities and risks of foundation models. In *arXiv preprint arXiv:2108.07258*, 2021.

- [13] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [14] Mayee F. Chen, Nicholas Roberts, Kush Bhatia, Jue Wang, Ce Zhang, Frederic Sala, and Christopher Ré. Skill-it! a data-driven skills framework for understanding and training language models, 2023. URL <https://arxiv.org/abs/2307.14430>.
- [15] Zhao Chen, Vijay Badrinarayanan, Chen-Yu Lee, and Andrew Rabinovich. Gradnorm: Gradient normalization for adaptive loss balancing in deep multitask networks, 2018. ICML 2018.
- [16] Zhao Chen, Vijay Badrinarayanan, Chen-Yu Lee, and Andrew Rabinovich. Gradnorm: Gradient normalization for adaptive loss balancing in deep multitask networks, 2018. URL <https://arxiv.org/abs/1711.02257>.
- [17] Krzysztof Choromanski, Valerii Likhoshesterov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. Rethinking attention with performers. In *International Conference on Learning Representations (ICLR)*, 2021.
- [18] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: Scaling language modeling with pathways. In *International Conference on Machine Learning (ICML)*, 2022.
- [19] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge, 2018. URL <https://arxiv.org/abs/1803.05457>.
- [20] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- [21] Tri Dao, Daniel Y Fu, Stefano Ermon, Atri Rudra, and Christopher Re. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [22] Jean-Antoine Désidéri. Multiple-gradient descent algorithm (MGDA) for multiobjective optimization. *Comptes Rendus Mathématique*, 350(5-6):313–318, 2012.
- [23] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *North American Chapter of the Association for Computational Linguistics (NAACL)*, 2019.
- [24] Shizhe Diao, Yu Yang, Yonggan Fu, Xin Dong, Dan Su, Markus Kliegl, Zijia Chen, Peter Belcak, Yoshi Suhara, Hongxu Yin, Mostofa Patwary, Yingyan, Lin, Jan Kautz, and Pavlo Molchanov. Climb: Clustering-based iterative data mixture bootstrapping for language model pre-training, 2025. URL <https://arxiv.org/abs/2504.13161>.
- [25] Shizhe Diao et al. Climblab: Clustering-based iterative data mixing, 2025. URL <https://climblab.ai>. Accessed: 2025.
- [26] John Duchi and Hongseok Namkoong. Learning models with uniform performance via distributionally robust optimization, 2020. URL <https://arxiv.org/abs/1810.08750>.
- [27] John C. Duchi and Hongseok Namkoong. Statistics of robust optimization: A generalized empirical likelihood approach, 2021. *Annals of Statistics* 2021.

- [28] Simin Fan, David Grangier, and Pierre Ablin. Dynamic gradient alignment for online data mixing, 2024. URL <https://arxiv.org/abs/2410.02498>.
- [29] Simin Fan, Matteo Pagliardini, and Martin Jaggi. Doge: Domain reweighting with generalization estimation, 2024. URL <https://arxiv.org/abs/2310.15393>.
- [30] Simin Fan, Maria Ios Glarou, and Martin Jaggi. Grape: Optimize data mixture for group robust multi-target adaptive pretraining, 2025. URL <https://arxiv.org/abs/2505.20380>.
- [31] Simin Fan, Maria Ios Glarou, and Martin Jaggi. Grape: Group-robust multi-target adaptive pretraining, 2025. This work.
- [32] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The pile: An 800gb dataset of diverse text for language modeling, 2020.
- [33] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N Dauphin. Convolutional sequence to sequence learning. In *International Conference on Machine Learning (ICML)*, 2017.
- [34] Thomas George, César Laurent, Xavier Bouthillier, Nicolas Ballas, and Pascal Vincent. Fast approximate natural gradient descent in a kronecker factored eigenbasis, 2018. NeurIPS 2018.
- [35] Amirata Ghorbani, Michael Kim, and James Zou. Data shapley: Equitable valuation of data for machine learning, 2019. ICML 2019.
- [36] David Grangier, Simin Fan, Skyler Seto, and Pierre Ablin. Task-adaptive pretrained language models via clustered-importance sampling, 2025. URL <https://arxiv.org/abs/2410.03735>.
- [37] Mandy Guo, Zihang Dai, Denny Vrandečić, and Rami Al-Rfou. Wiki-40B: Multilingual language model dataset. In Nicoletta Calzolari, Frédéric Béchet, Philippe Blache, Khalid Choukri, Christopher Cieri, Thierry Declerck, Sara Goggi, Hitoshi Isahara, Bente Maegaard, Joseph Mariani, Hélène Mazo, Asuncion Moreno, Jan Odijk, and Stelios Piperidis, editors, *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 2440–2452, Marseille, France, May 2020. European Language Resources Association. ISBN 979-10-95546-34-4. URL <https://aclanthology.org/2020.lrec-1.297/>.
- [38] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.
- [39] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. In *arXiv preprint arXiv:2203.15556*, 2022.
- [40] Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, Xinrong Zhang, Zheng Leng Thai, Kaihuo Zhang, Chongyi Wang, Yuan Yao, Chenyang Zhao, Jie Zhou, Jie Cai, Zhongwu Zhai, Ning Ding, Chao Jia, Guoyang Zeng, Dahai Li, Zhiyuan Liu, and Maosong Sun. Minicpm: Unveiling the potential of small language models with scalable training strategies, 2024. URL <https://arxiv.org/abs/2404.06395>.
- [41] Ruoxi Jia, Tri Dao, Sham M. Wang, Florian Hubis, Wesley Gurnee, Alex Kazachkov, Christopher Liaw, Bryan Wang, and Ce Zhang. Towards efficient data valuation based on the shapley value, 2019. AISTATS 2019.
- [42] Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. What disease does this patient have? a large-scale open domain question answering dataset from medical exams, 2020. URL <https://arxiv.org/abs/2009.13081>.

- [43] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. In *arXiv preprint arXiv:2001.08361*, 2020.
- [44] Krishna C. Killamsetty, Sonaal Durga, Ganesh Ramakrishnan, Abir De, and Rishabh Iyer. Glisten: Generalization based data subset selection for efficient and robust learning, 2021. AAAI 2021.
- [45] Krishna C. Killamsetty, Dilip Sivasubramanian, and Rishabh Iyer. Gradmatch: Gradient matching based data subset selection for efficient deep model training, 2021. ICML 2021.
- [46] Pang Wei Koh and Percy Liang. Understanding black-box predictions via influence functions, 2017. ICML 2017.
- [47] Yongchan Kwon, Joongbo Kim, Yongchan Park, Dongjun Joo, and Il-Chul Moon. Beta shapley: A unified and noise-reduced data valuation framework for machine learning, 2021. NeurIPS 2021.
- [48] Guokun Lai, Qizhe Xie, Hanxiao Liu, Yiming Yang, and Eduard Hovy. Race: Large-scale reading comprehension dataset from examinations. *arXiv preprint arXiv:1704.04683*, 2017.
- [49] Katherine Lee, Tianjian Gao, Adam Roberts, Noam Shazeer, Phillip Isola, Quoc V Le, Peter J Liu, and Colin Raffel. Deduplicating training data makes language models better. *arXiv preprint arXiv:2107.06499*, 2022.
- [50] Bo Liu, Xiang Zhu, Jinliang Li, Xiaotong Liu, Yanfeng Sun, and Bolei Zhou. Conflict-averse gradient descent for multi-task learning, 2021. NeurIPS 2021.
- [51] Bo Liu, Yihao Feng, Peter Stone, and Qiang Liu. Famo: Fast adaptive multitask optimization, 2023. URL <https://arxiv.org/abs/2306.03792>.
- [52] Bo Liu, Xingchao Liu, Xiaojie Jin, Peter Stone, and Qiang Liu. Conflict-averse gradient descent for multi-task learning, 2024. URL <https://arxiv.org/abs/2110.14048>.
- [53] Jian Liu, Leyang Cui, Hanmeng Liu, Dandan Huang, Yile Wang, and Yue Zhang. Logiqa: A challenge dataset for machine reading comprehension with logical reasoning, 2020. URL <https://arxiv.org/abs/2007.08124>.
- [54] Qian Liu, Xiaosen Zheng, Niklas Muennighoff, Guangtao Zeng, Longxu Dou, Tianyu Pang, Jing Jiang, and Min Lin. Regmix: Data mixture as regression for language model pre-training, 2025. URL <https://arxiv.org/abs/2407.01492>.
- [55] Kyle Lo, Lucy Lu Wang, Mark Neumann, Rodney Kinney, and Daniel S Weld. S2orc: The semantic scholar open research corpus. In *Association for Computational Linguistics (ACL)*, 2019.
- [56] James Martens and Roger Grosse. Optimizing neural networks with kronecker-factored approximate curvature, 2015. ICML 2015.
- [57] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. *arXiv preprint arXiv:1809.02789*, 2018.
- [58] Baharan Mirzasoleiman, Jeff Bilmes, Jure Leskovec, and Cho-Jui Hsieh. Coresets for data-efficient training of machine learning models, 2020. ICML 2020.
- [59] Hongseok Namkoong and John C Duchi. Stochastic gradient methods for distributionally robust optimization with f-divergences. In D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc., 2016. URL https://proceedings.neurips.cc/paper_files/paper/2016/file/4588e674d3f0faf985047d4c3f13ed0d-Paper.pdf.

- [60] Hongseok Namkoong and John C. Duchi. Variance-based regularization with convex objectives, 2017. NeurIPS 2017.
- [61] Aviv Navon, Aviv Shamsian, Idan Achituve, Haggai Maron, Kenji Kawaguchi, Gal Chechik, and Ethan Fetaya. Multi-task learning as a bargaining game, 2022. URL <https://arxiv.org/abs/2202.01017>.
- [62] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [63] Namkyeong Park, Sanghyuk Choe, Min Kim, Sungsoo Ahn, Se-Young Yun, Minyoung Huh, and Jinwoo Shin. Trak: Attributing model behavior at scale. arXiv:2303.14186, 2023. ICML 2023.
- [64] Guilherme Penedo, Hynek Kydlíček, Vinko Sabolčec, Bettina Messmer, Negar Foroutan, Amir Hossein Kargaran, Colin Raffel, Martin Jaggi, Leandro Von Werra, and Thomas Wolf. Fineweb2: One pipeline to scale them all – adapting pre-training data processing to every language, 2025. URL <https://arxiv.org/abs/2506.20920>.
- [65] Aleksandra Piktus, Gautier Izacard, Lucas Hosseini, and Edouard Grave. The refinedweb dataset for falcon llm: Outperforming curated corpora with web data, and web data only. In *arXiv preprint arXiv:2306.01116*, 2023.
- [66] Gaurav Pruthi, Frederick Liu, Satyen Kale, and Mukund Sundararajan. Estimating training data influence by tracing gradient descent, 2020. NeurIPS 2020.
- [67] Qi Qi, Zhishuai Guo, Yi Xu, Rong Jin, and Tianbao Yang. An online method for a class of distributionally robust optimization with non-convex objectives, 2021. URL <https://arxiv.org/abs/2006.10138>.
- [68] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. *OpenAI preprint*, 2018.
- [69] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI preprint*, 2019.
- [70] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. In *Journal of Machine Learning Research*, 2020.
- [71] Melissa Roemmele, Cosmin Adrian Bejan, and Andrew S. Gordon. Choice of plausible alternatives: An evaluation of commonsense causal reasoning. In *AAAI Spring Symposium on Logical Formalizations of Commonsense Reasoning*, Stanford University, March 2011. URL <https://cdn.aaai.org/ocs/2418/2418-10878-1-PB.pdf>.
- [72] Shiori Sagawa, Pang Wei Koh, Tatsunori Hashimoto, and Percy Liang. Distributionally robust neural networks for group shifts: On the importance of regularization for worst-case generalization. arXiv:1911.08731, 2020. ICLR 2020.
- [73] Shiori Sagawa, Pang Wei Koh, Tatsunori B. Hashimoto, and Percy Liang. Distributionally robust neural networks for group shifts: On the importance of regularization for worst-case generalization, 2020. URL <https://arxiv.org/abs/1911.08731>.
- [74] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *arXiv preprint arXiv:1907.10641*, 2019.
- [75] Ozan Sener and Vladlen Koltun. Multi-task learning as multi-objective optimization, 2018. NeurIPS 2018.

- [76] Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. Self-attention with relative position representations. In *North American Chapter of the Association for Computational Linguistics (NAACL)*, 2018.
- [77] Daria Soboleva, Faisal Al-Khateeb, Robert Myers, Jacob R Steeves, Joel Hestness, and Nolan Dey. SlimPajama: A 627B token cleaned and deduplicated version of RedPajama. <https://www.cerebras.net/blog/slimpajama-a-627b-token-cleaned-and-deduplicated-version-of-redpajama>, June 2023. URL <https://huggingface.co/datasets/cerebras/SlimPajama-627B>.
- [78] Dan Su, Kezhi Kong, Ying Lin, Joseph Jennings, Brandon Norick, Markus Kliegl, Mostofa Patwary, Mohammad Shoeybi, and Bryan Catanzaro. Nemotron-cc: Transforming common crawl into a refined long-horizon pretraining dataset, 2025. URL <https://arxiv.org/abs/2412.02595>.
- [79] Jianlin Su, Yu Lu, Shengfeng Pan, Bo Miao, and Yongdong Zhou. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.
- [80] Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. Commonsenseqa: A question answering challenge targeting commonsense knowledge. *arXiv preprint arXiv:1811.00937*, 2018.
- [81] Yi Tay, Mostafa Dehghani, Vinh Q. Tran, Xavier Garcia, Dara Bahri, Tal Schuster, Huaixiu Steven Zheng, Neil Houlsby, and Donald Metzler. U12: Unifying language learning paradigms, 2022. URL <https://arxiv.org/abs/2205.05131>.
- [82] Together Computer. Redpajama: An open source recipe to reproduce llama training dataset, 2023. URL <https://github.com/togethercomputer/RedPajama-Data>.
- [83] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. In *arXiv preprint arXiv:2302.13971*, 2023.
- [84] Taishi Uchiya, Atsuyoshi Nakamura, and Mineichi Kudo. Algorithms for adversarial bandit problems with multiple plays. In *Algorithmic Learning Theory (ALT 2010)*, volume 6331 of *Lecture Notes in Computer Science*, pages 375–389, Berlin, Heidelberg, 2010. Springer. doi: 10.1007/978-3-642-16108-7_30.
- [85] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [86] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023. URL <https://arxiv.org/abs/1706.03762>.
- [87] Hang Wang, Minghao Xu, Bingbing Ni, and Wenjun Zhang. Learning to combine: Knowledge aggregation for multi-source domain adaptation, 2020. URL <https://arxiv.org/abs/2007.08801>.
- [88] Jiachen T. Wang, Prateek Mittal, Dawn Song, and Ruoxi Jia. Data shapley in one training run. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025. doi: 10.48550/arXiv.2406.11011. URL <https://openreview.net/forum?id=HD6bWcj87Y>. Oral.
- [89] Jiachen T. Wang, Prateek Mittal, Dawn Song, and Ruoxi Jia. Data shapley in one training run, 2025. URL <https://arxiv.org/abs/2406.11011>.
- [90] Jun Wang, Ruoxi Jia, and Ce Zhang. Data banzhaf: A robust data valuation framework for machine learning. *arXiv:2206.08679*, 2023. ICLR 2023.

- [91] Jason Wei, Maarten Bosma, Vincent Zhao, Kelvin Guu, Adams Yu, Brian Lester, Nan Du, Andrew Dai, and Quoc V Le. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations (ICLR)*, 2022.
- [92] Johannes Welbl, Nelson F. Liu, and Matt Gardner. Crowdsourcing multiple choice science questions, 2017. URL <https://arxiv.org/abs/1707.06209>.
- [93] Sang Michael Xie, Hieu Pham, Xuanyi Dong, Nan Du, Hanxiao Liu, Yifeng Lu, Percy Liang, Quoc V. Le, Tengyu Ma, and Adams Wei Yu. Doremi: Optimizing data mixtures speeds up language model pretraining, 2023. URL <https://arxiv.org/abs/2305.10429>.
- [94] Weizhe Xu, Yiming Wang, Jiayu Wu, and Dian Yu. Curriculum learning for natural language understanding. In *Association for Computational Linguistics (ACL)*, 2020.
- [95] Zhangchen Xu, Yang Liu, Yueqin Yin, Mingyuan Zhou, and Radha Poovendran. Kodcode: A diverse, challenging, and verifiable synthetic dataset for coding, 2024. URL <https://arxiv.org/abs/2409.03810>.
- [96] Tang Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning, 2020. NeurIPS 2020.
- [97] Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning, 2020. URL <https://arxiv.org/abs/2001.06782>.
- [98] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence?, 2019. URL <https://arxiv.org/abs/1905.07830>.
- [99] Yukun Zhu, Ryan Kiros, Rich Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. In *International Conference on Computer Vision (ICCV)*, 2015.